

part_2

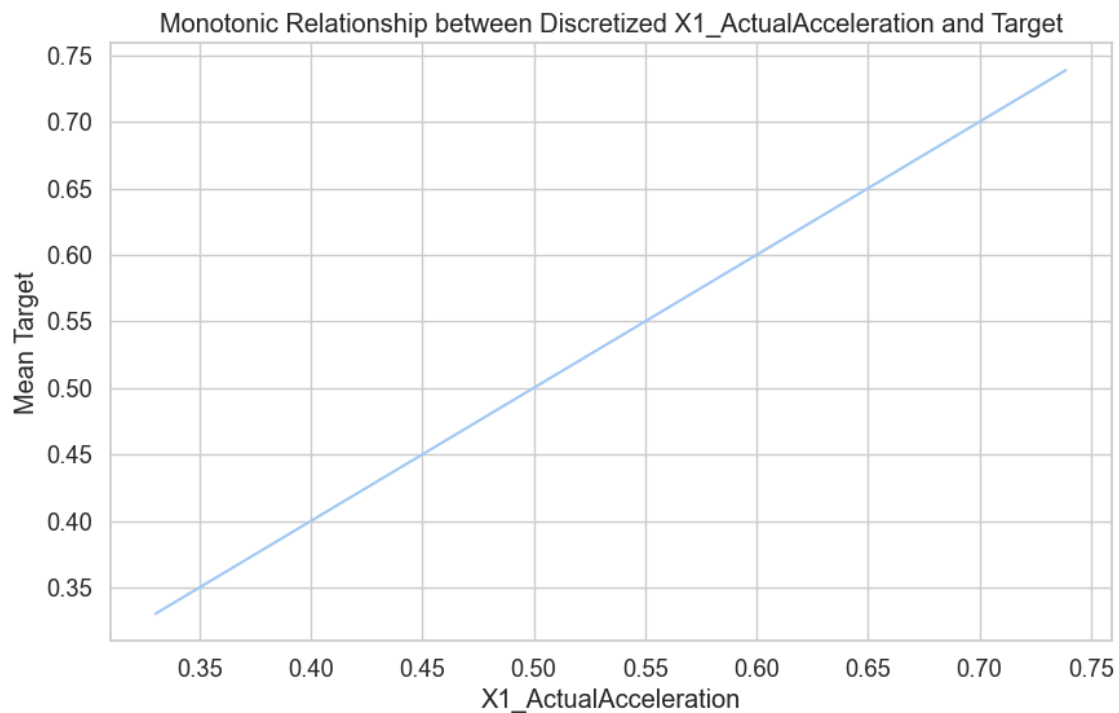
October 25, 2025

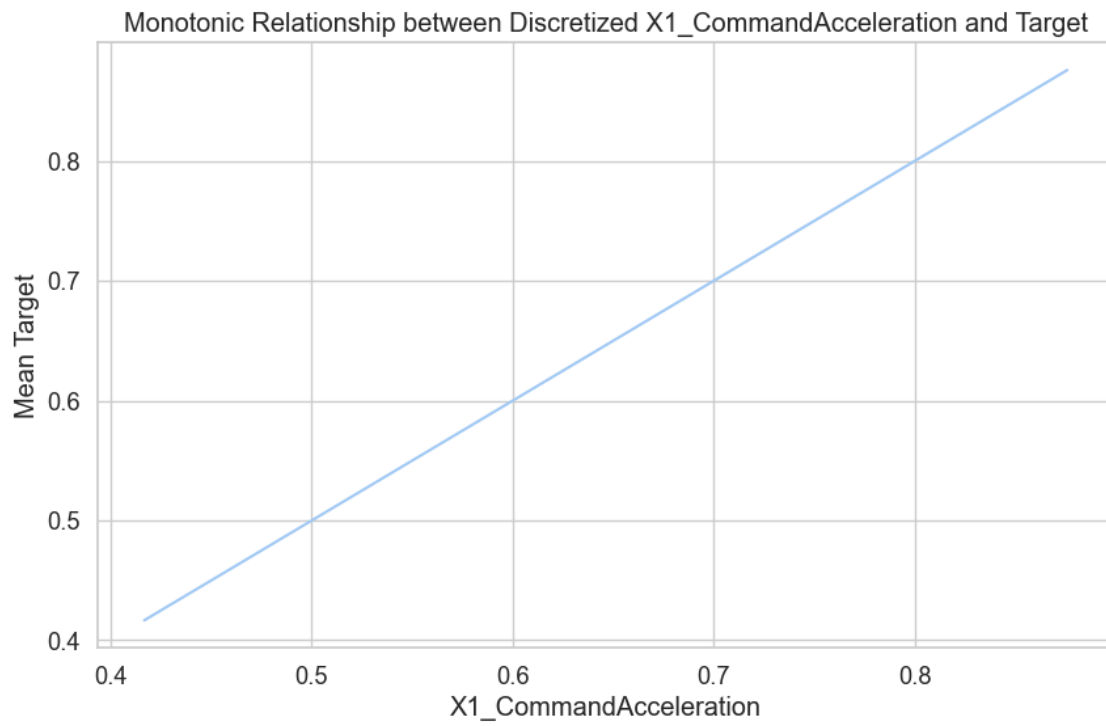
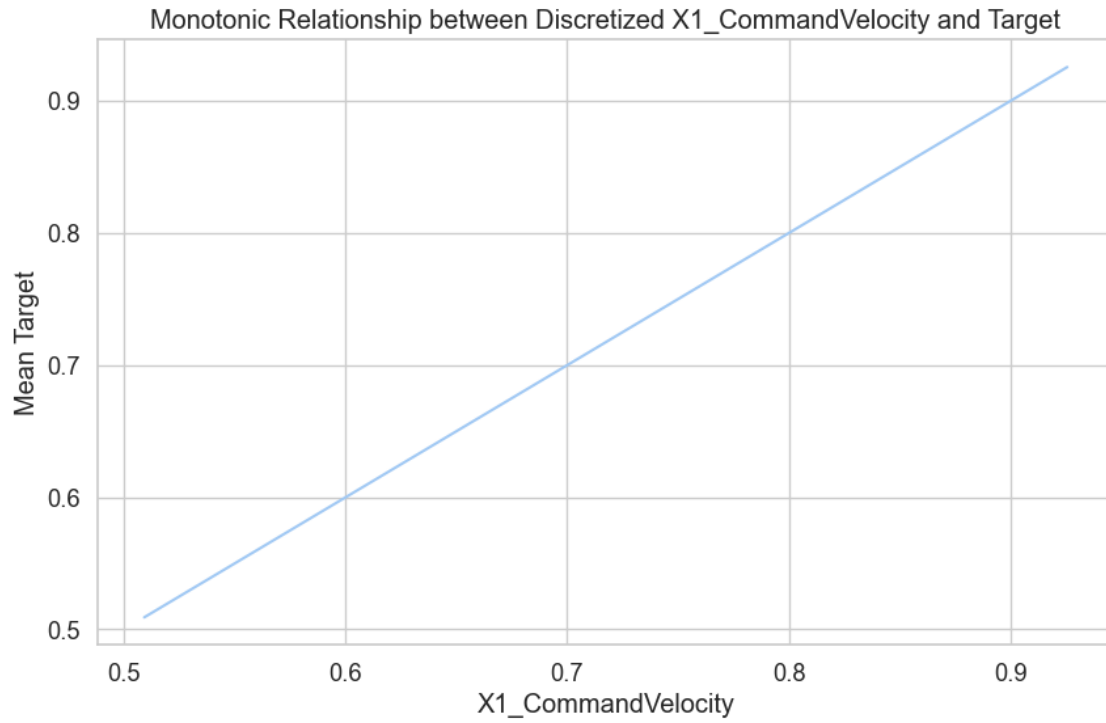
```
[49]: for col in train_t.columns:
    plt.figure(figsize=(10, 6))

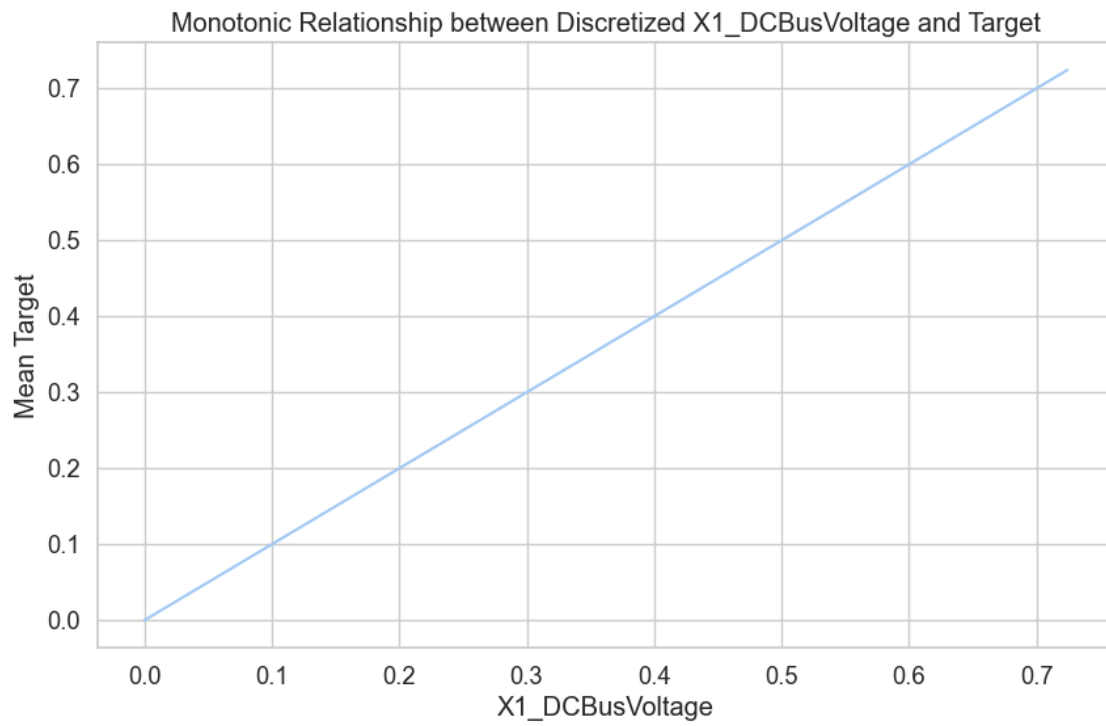
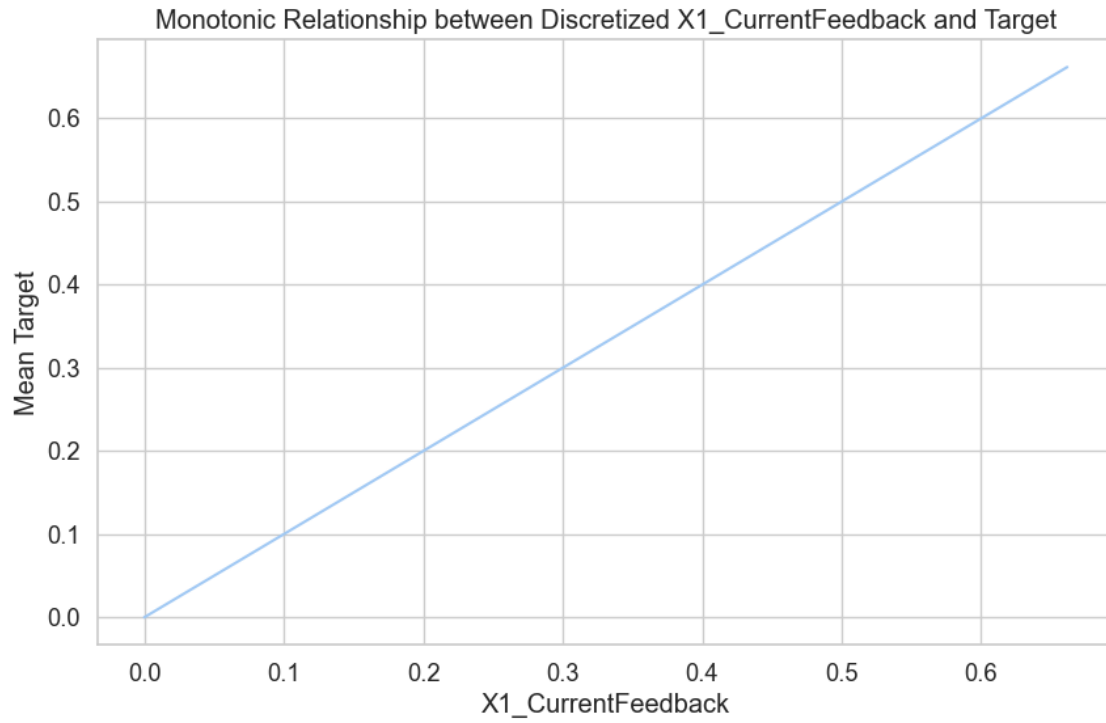
    combined_data = pd.concat([train_t, y_train], axis=1)
    grouped_data = combined_data.groupby(col)['target'].mean()
    grouped_data.plot()

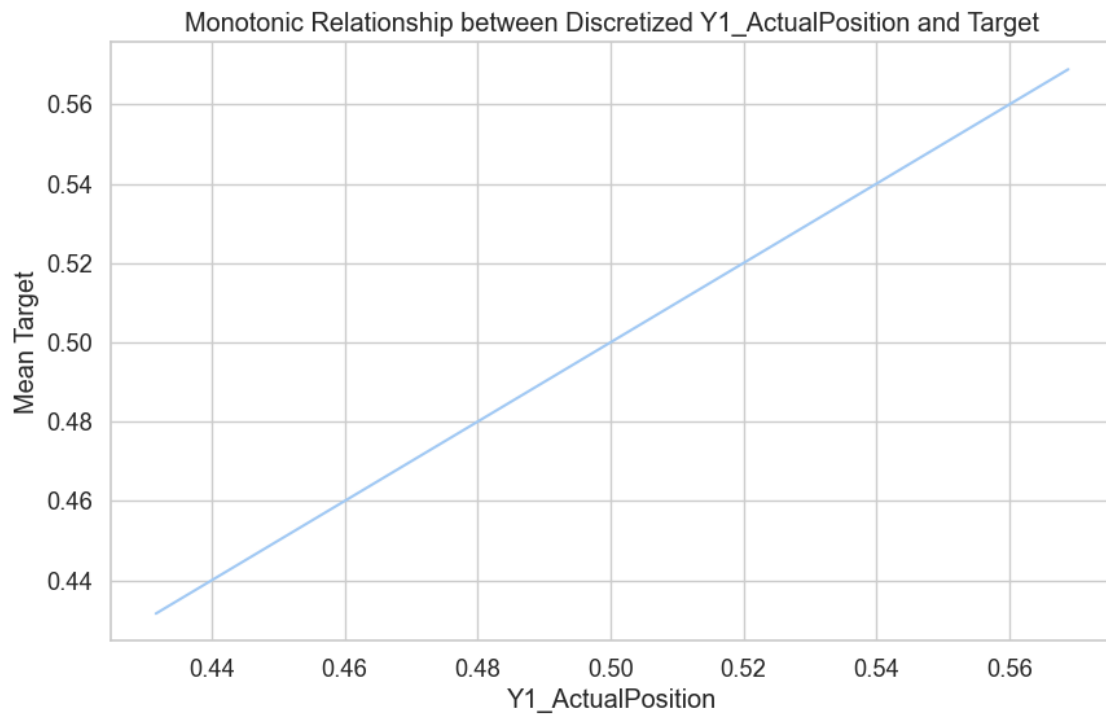
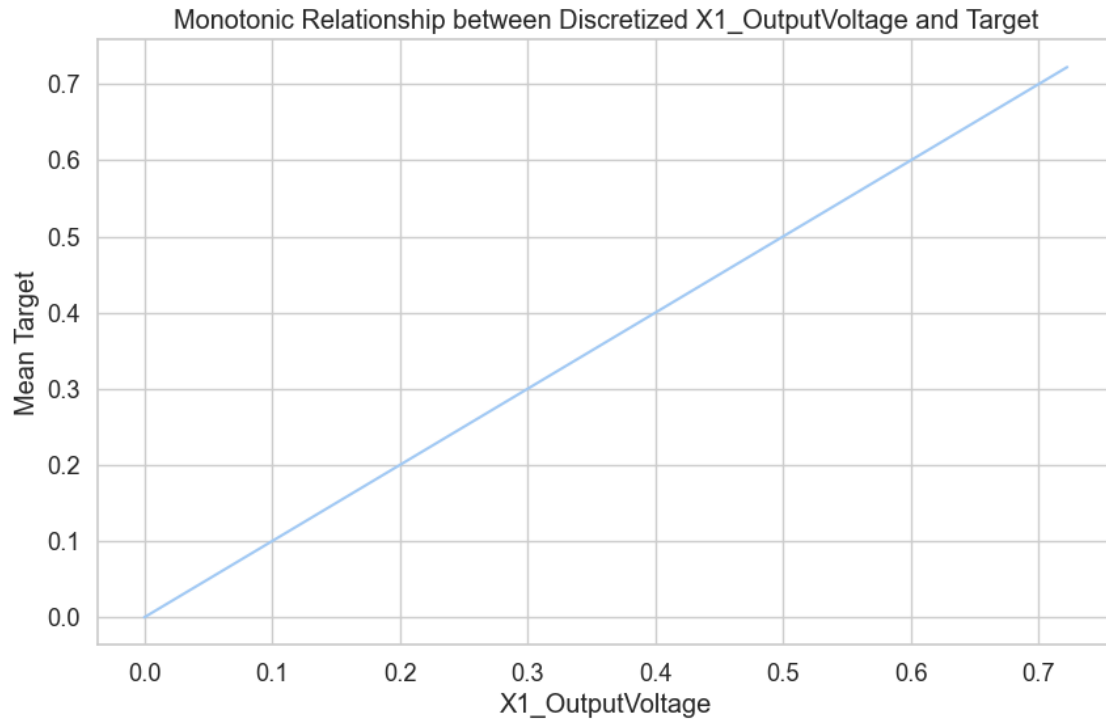
    plt.title(f'Monotonic Relationship between Discretized {col} and Target')
    plt.xlabel(col)
    plt.ylabel('Mean Target')
    plt.grid(True)

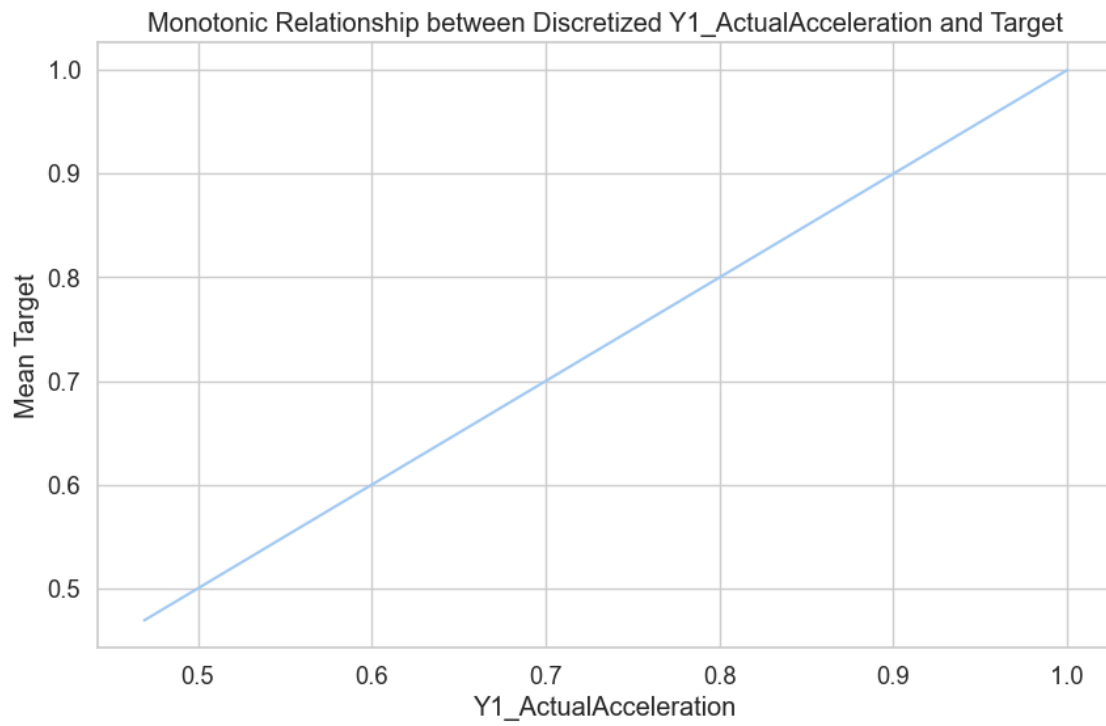
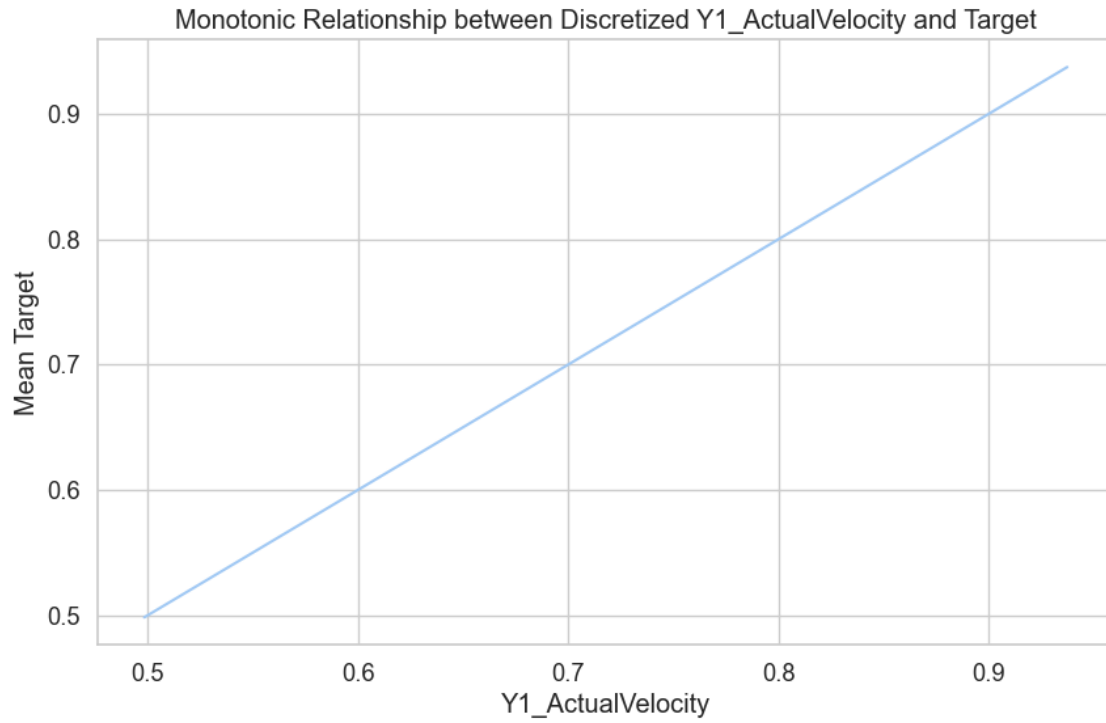
    plt.show()
```

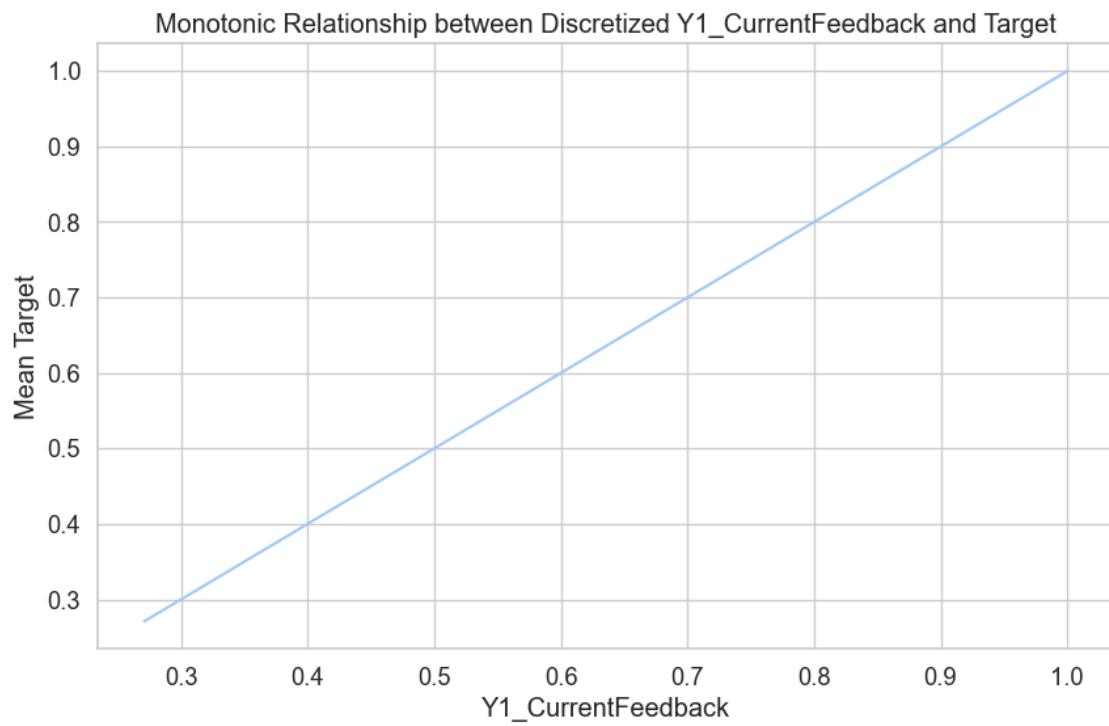
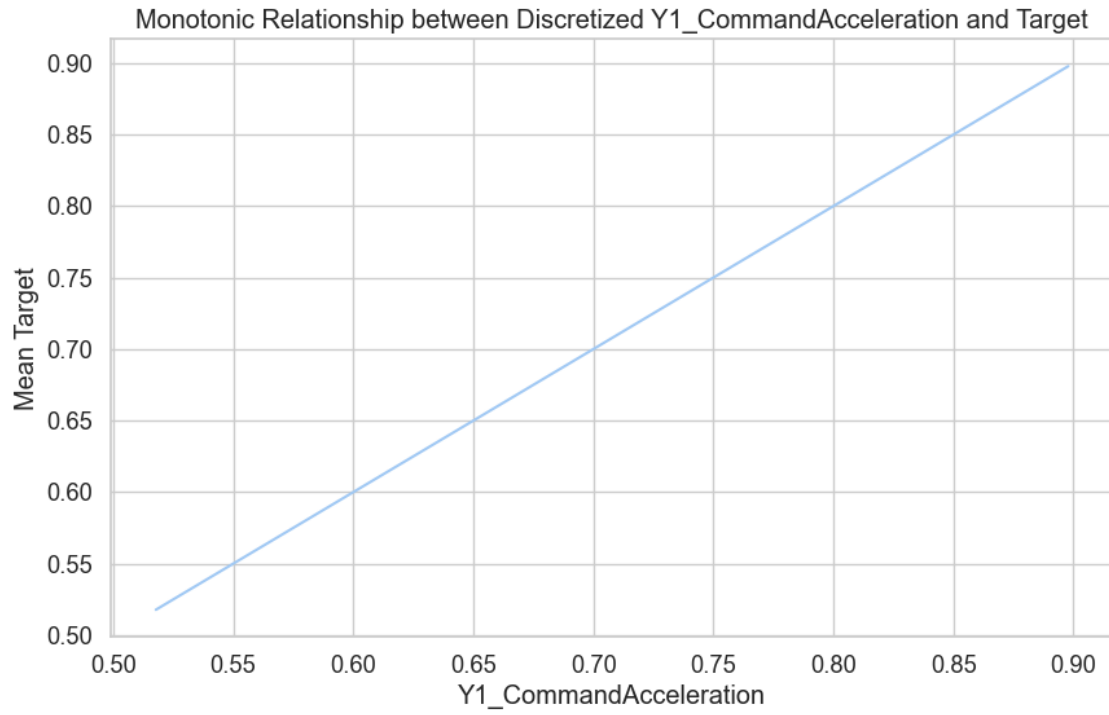


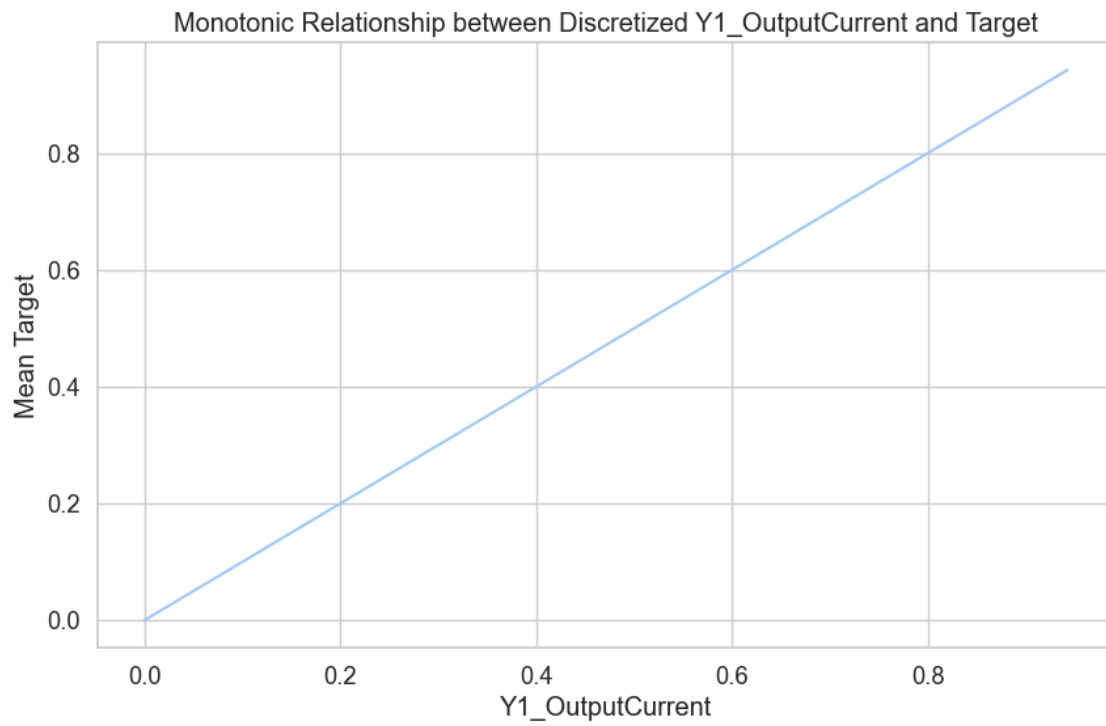
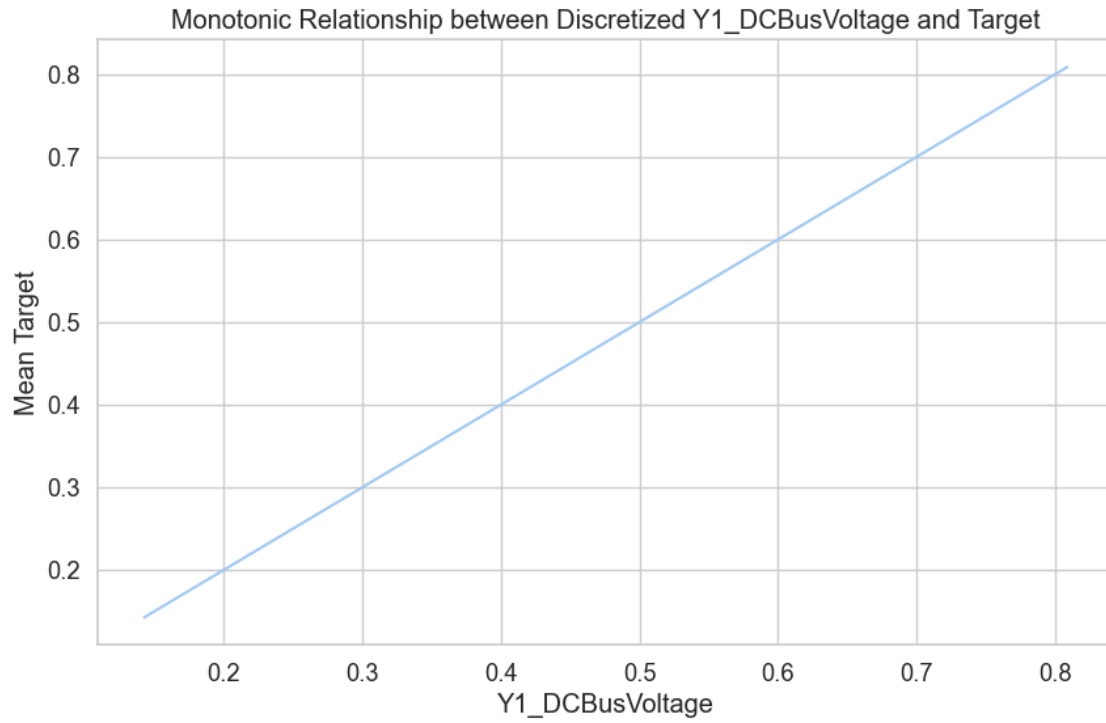


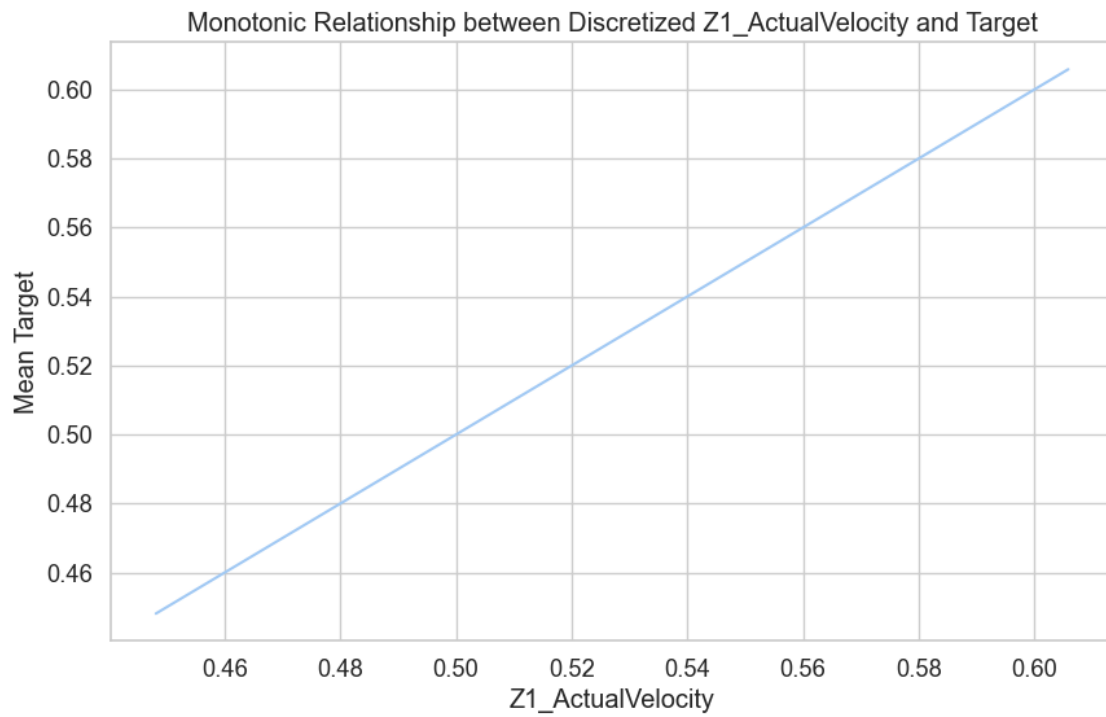
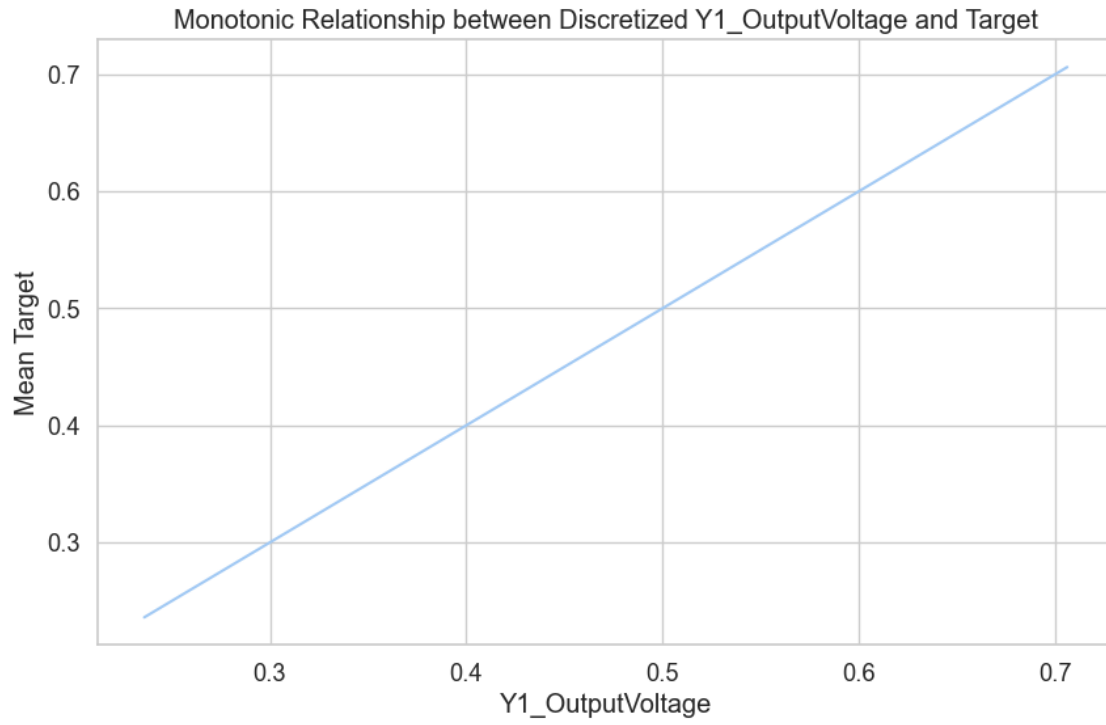


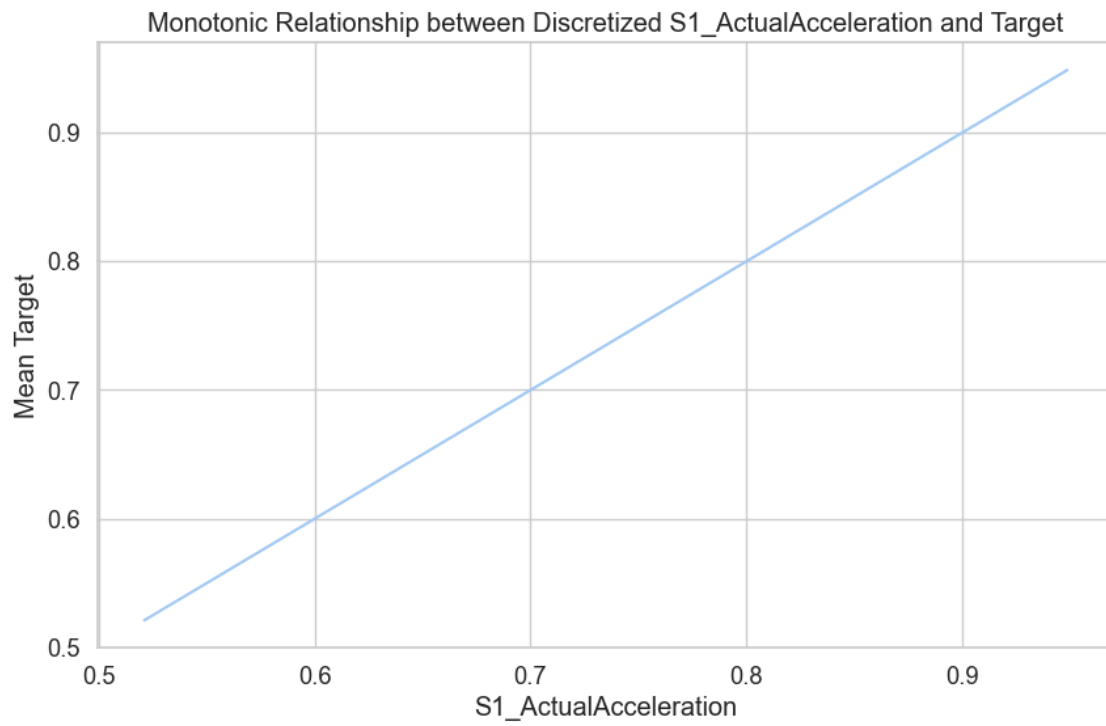
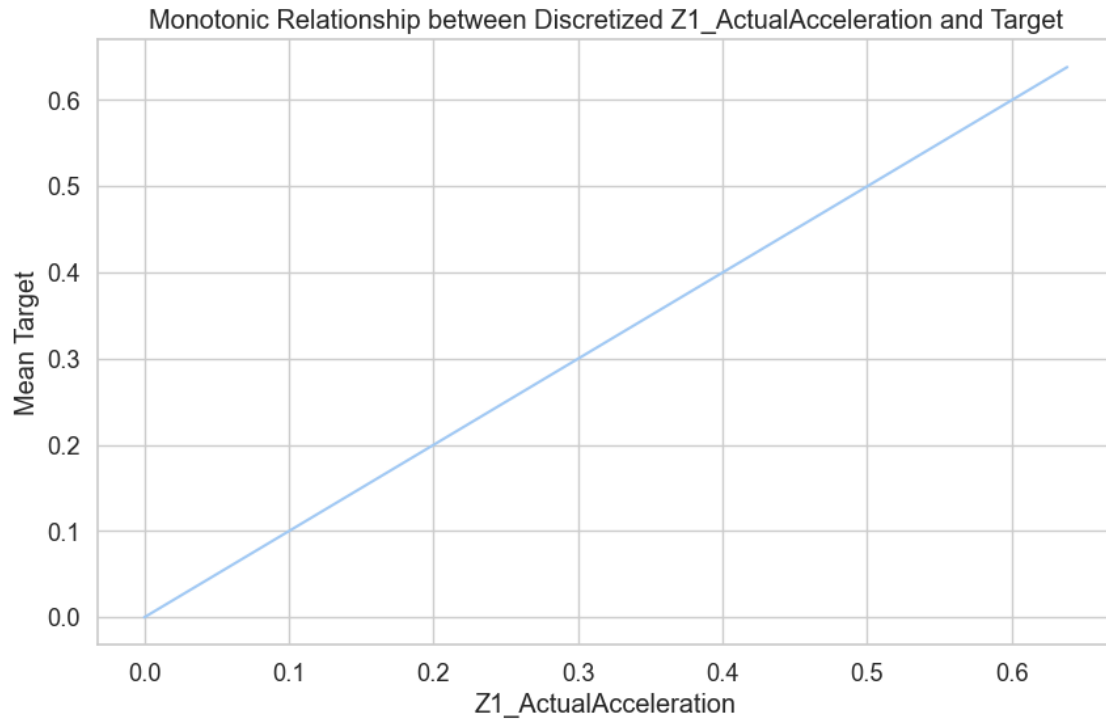


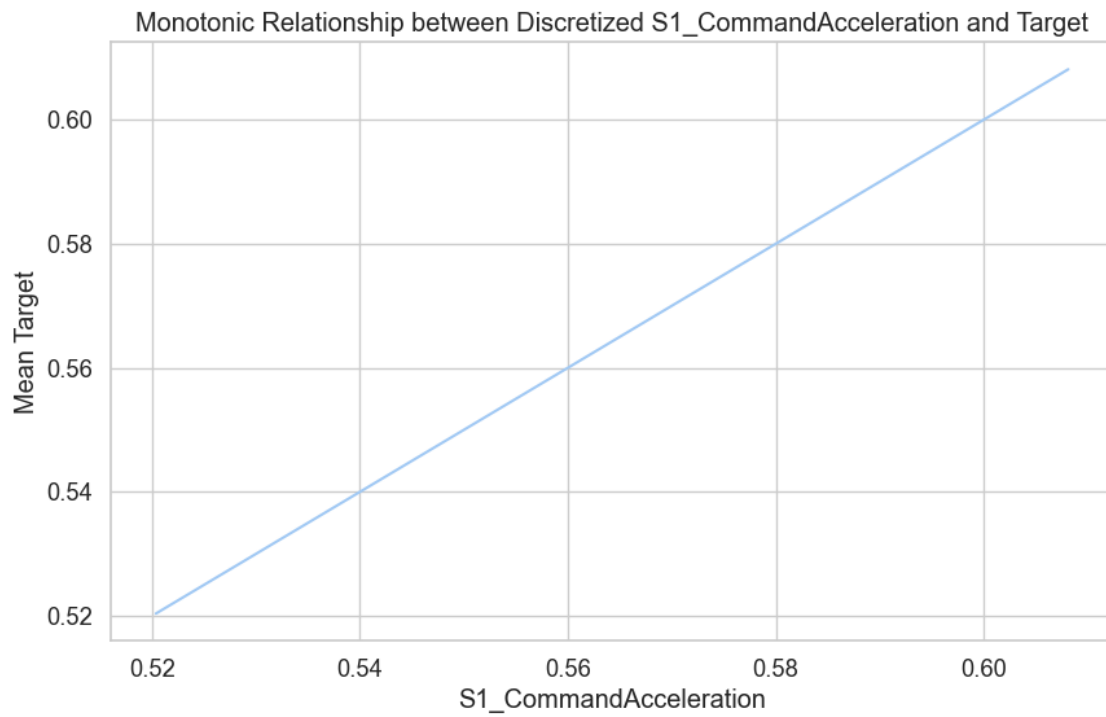
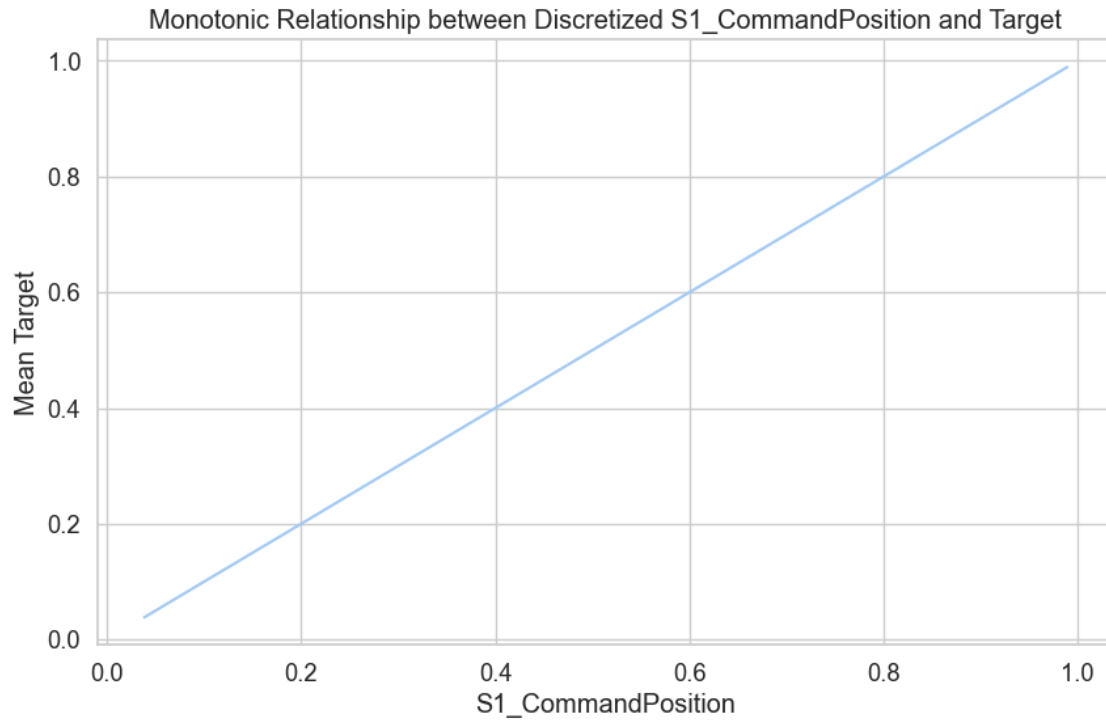


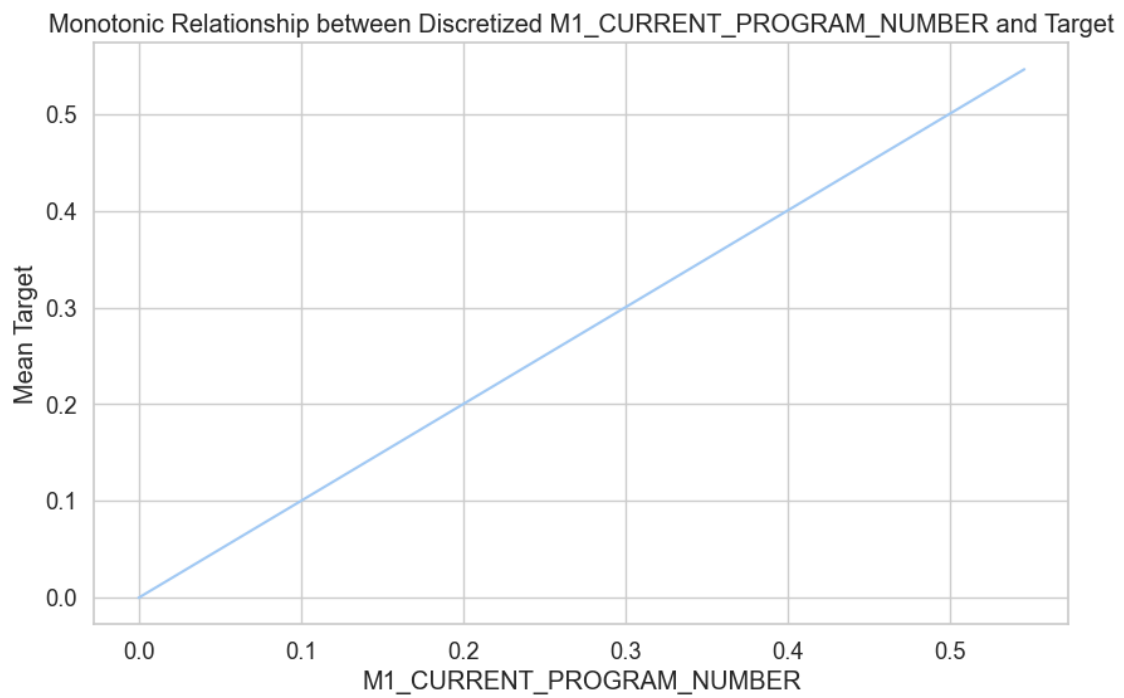
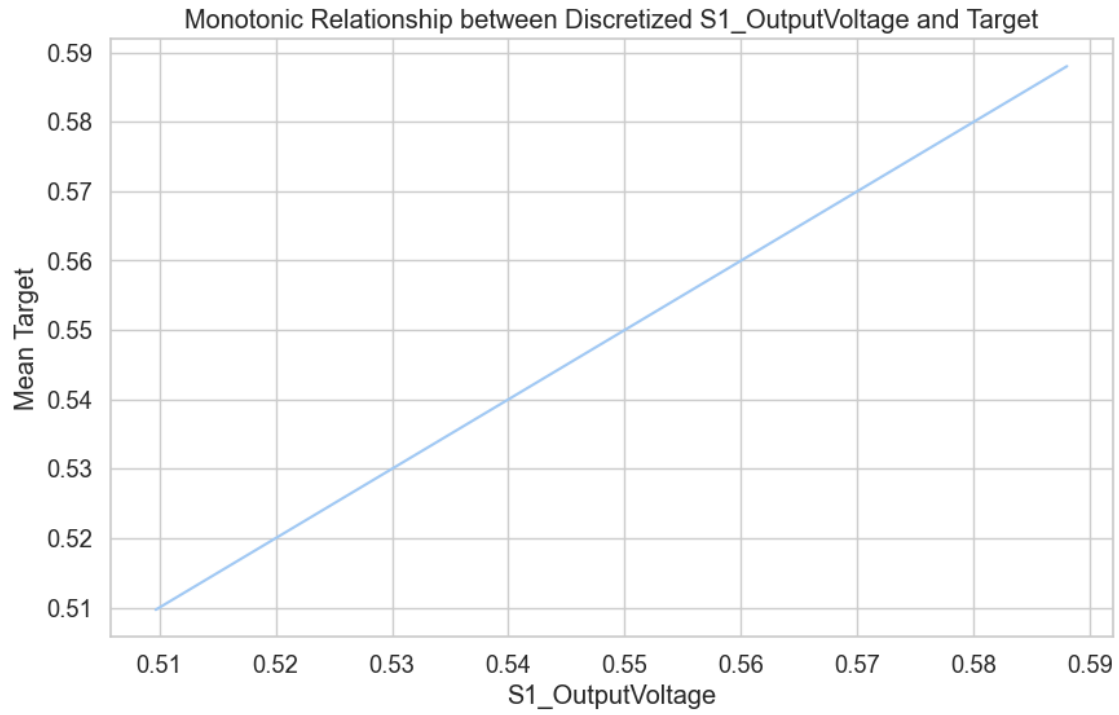


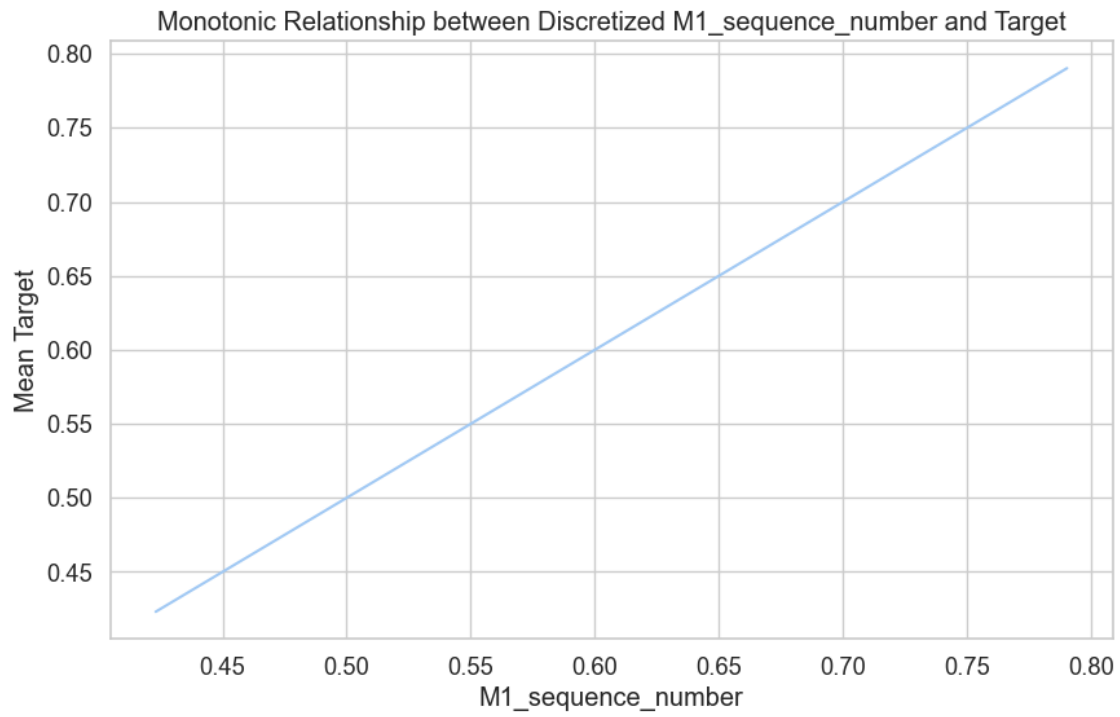


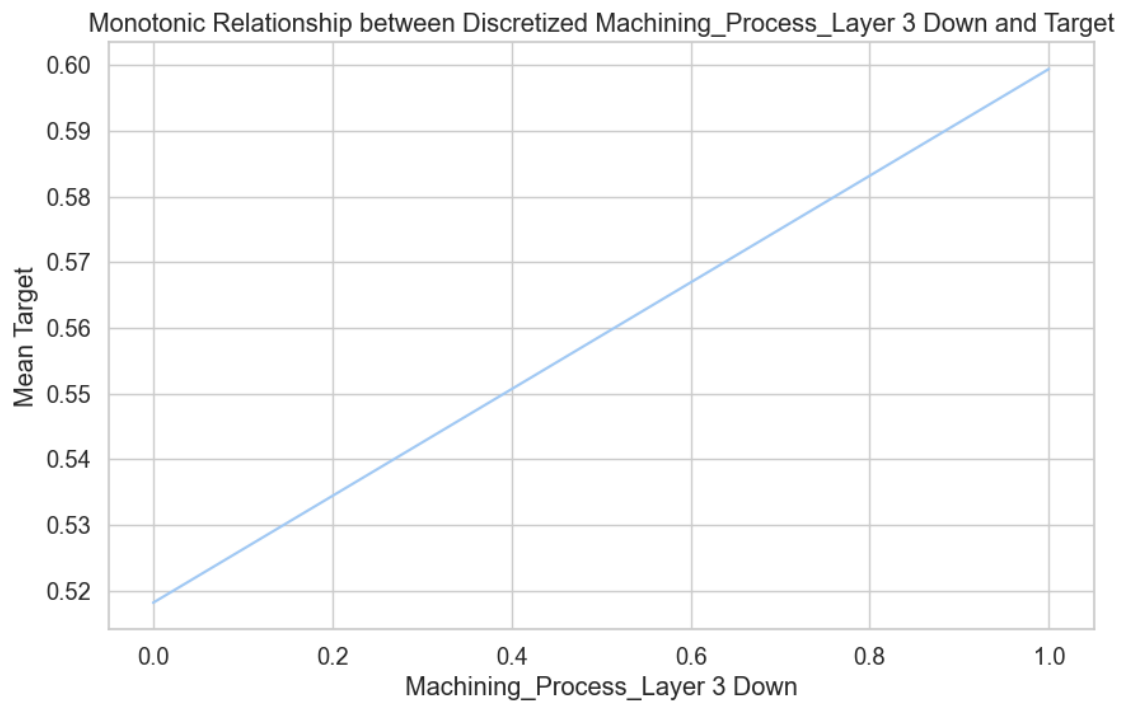
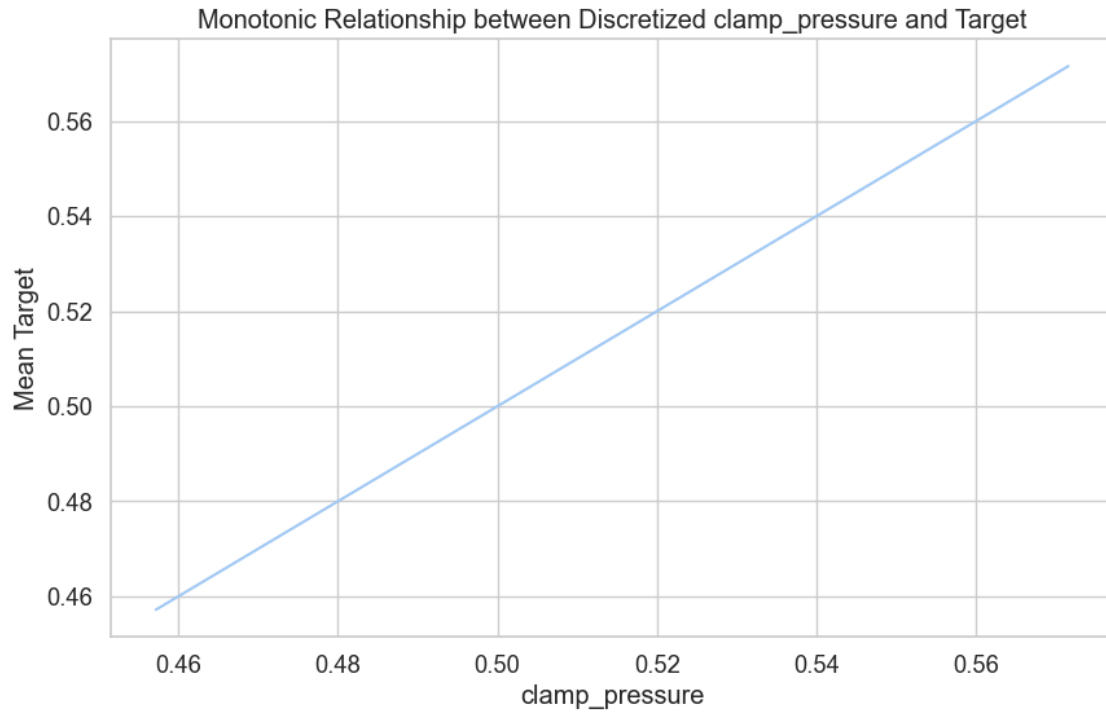


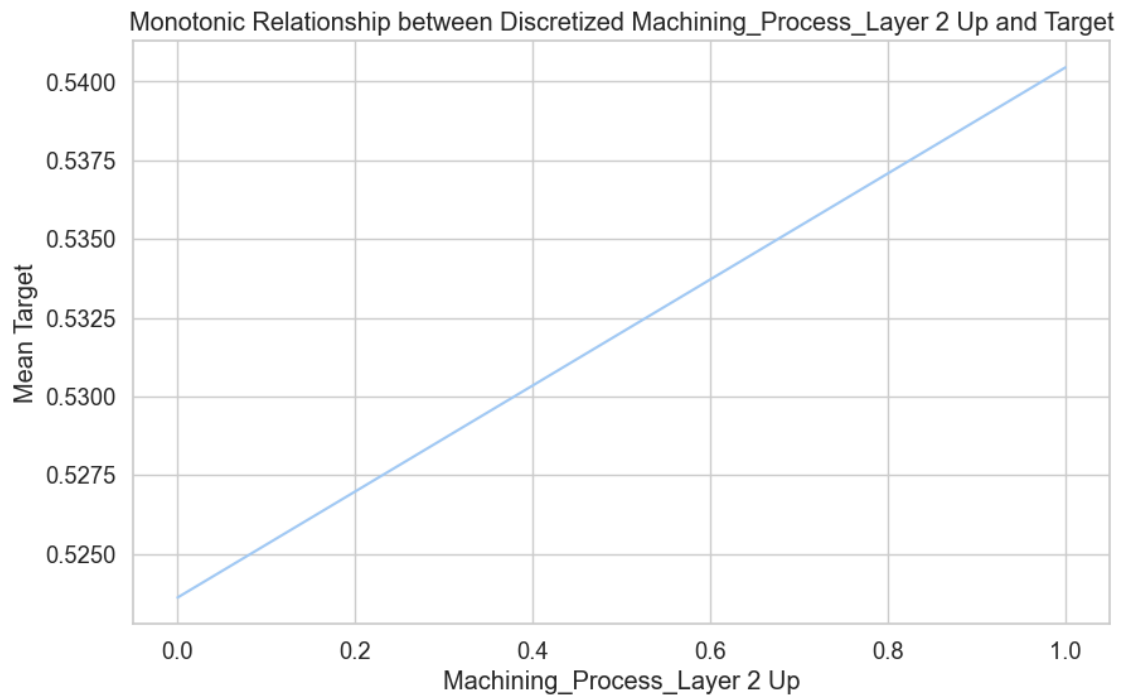
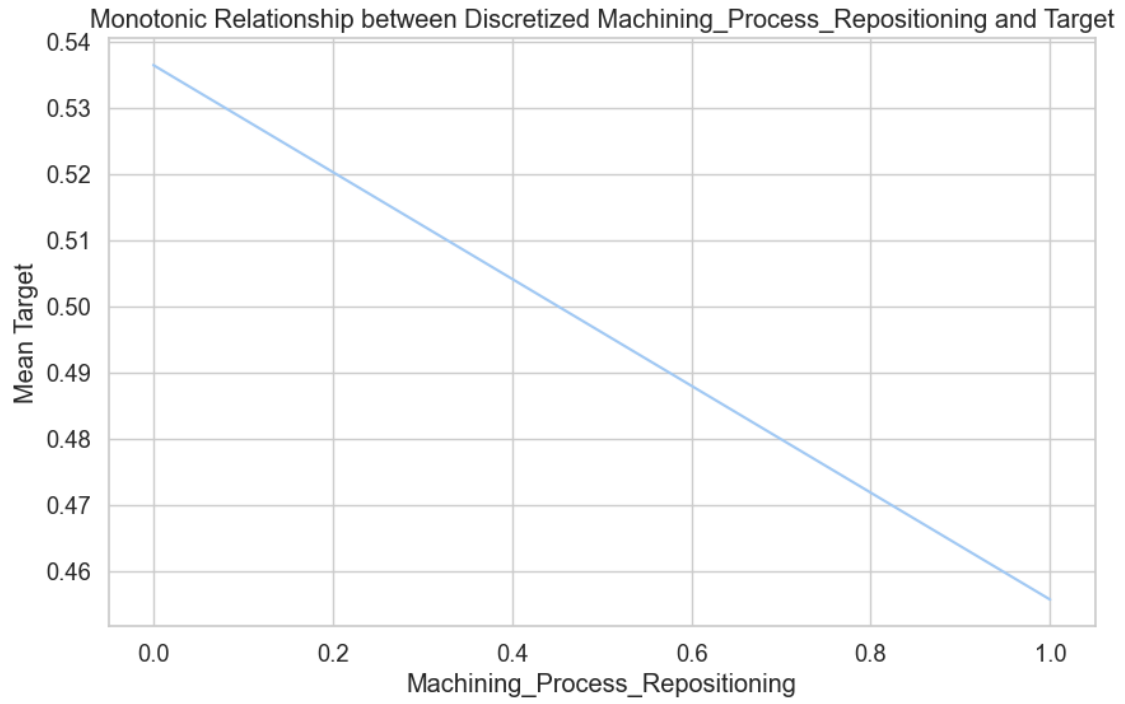


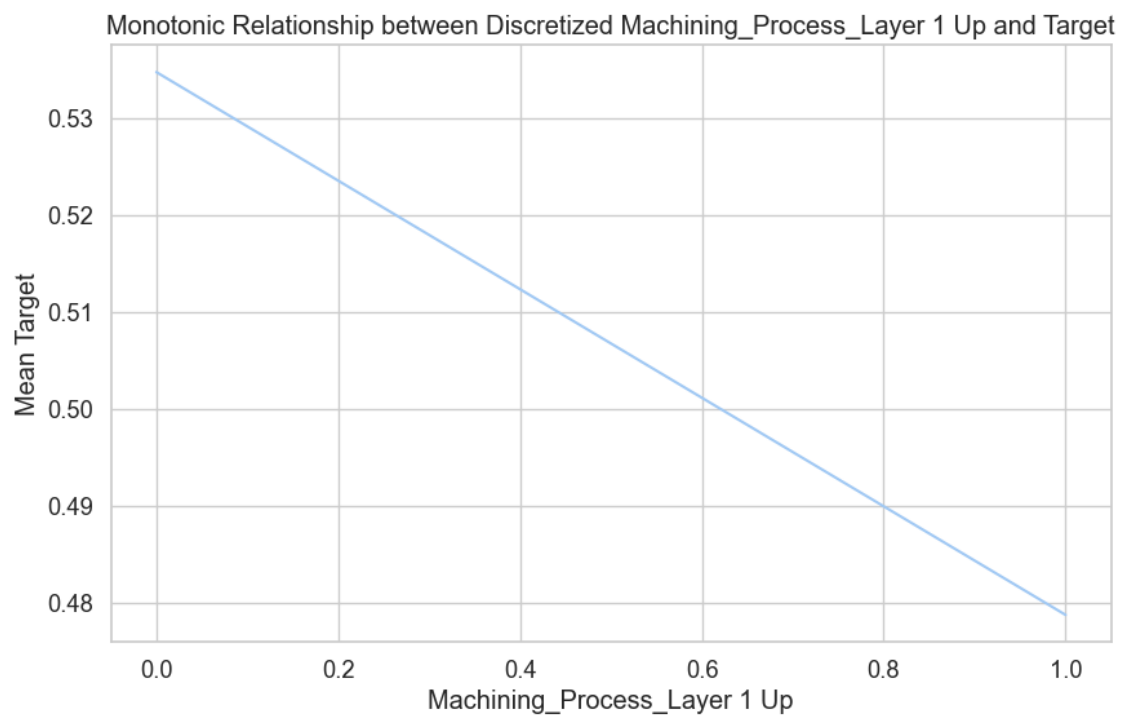
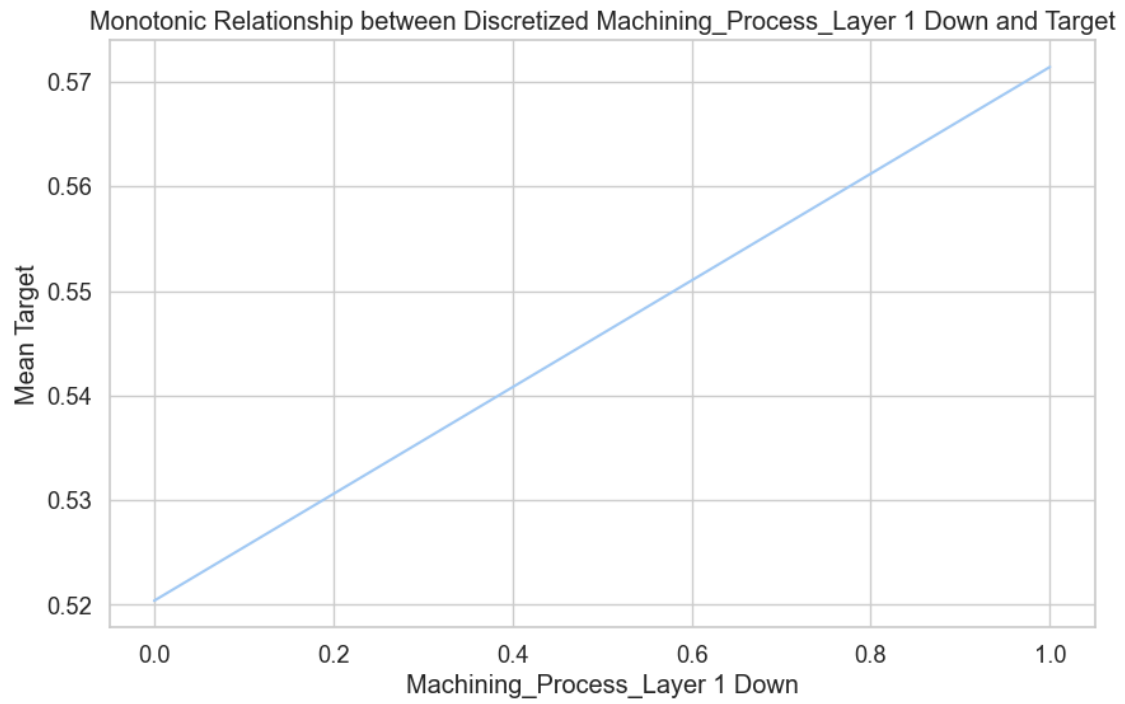


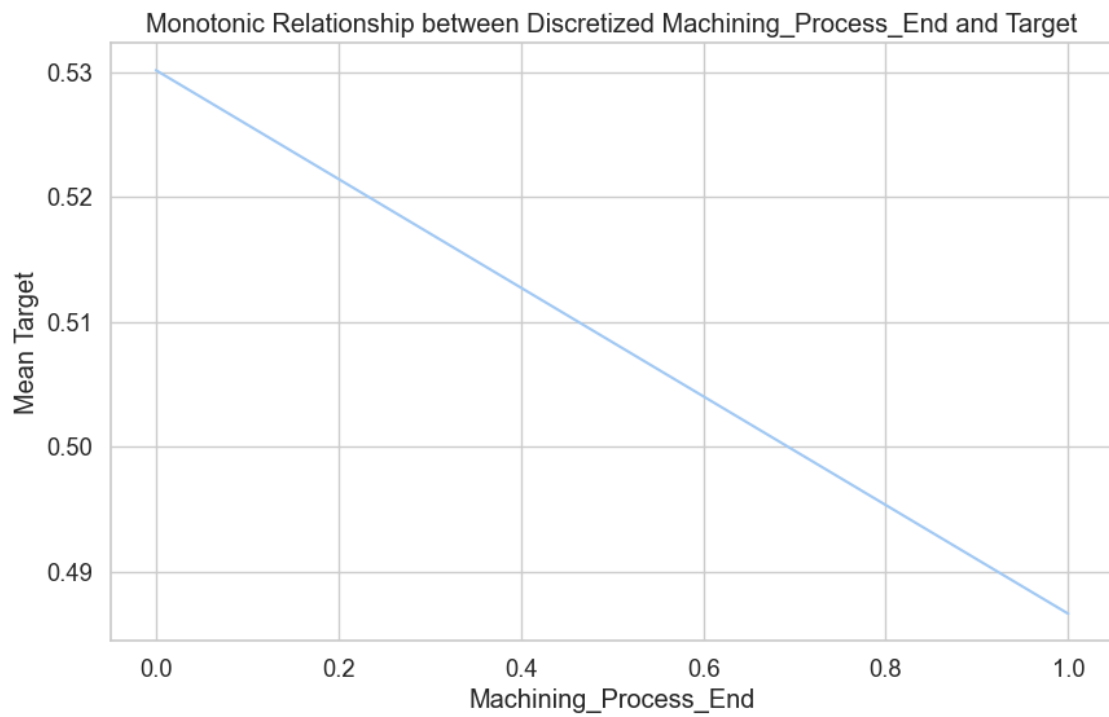
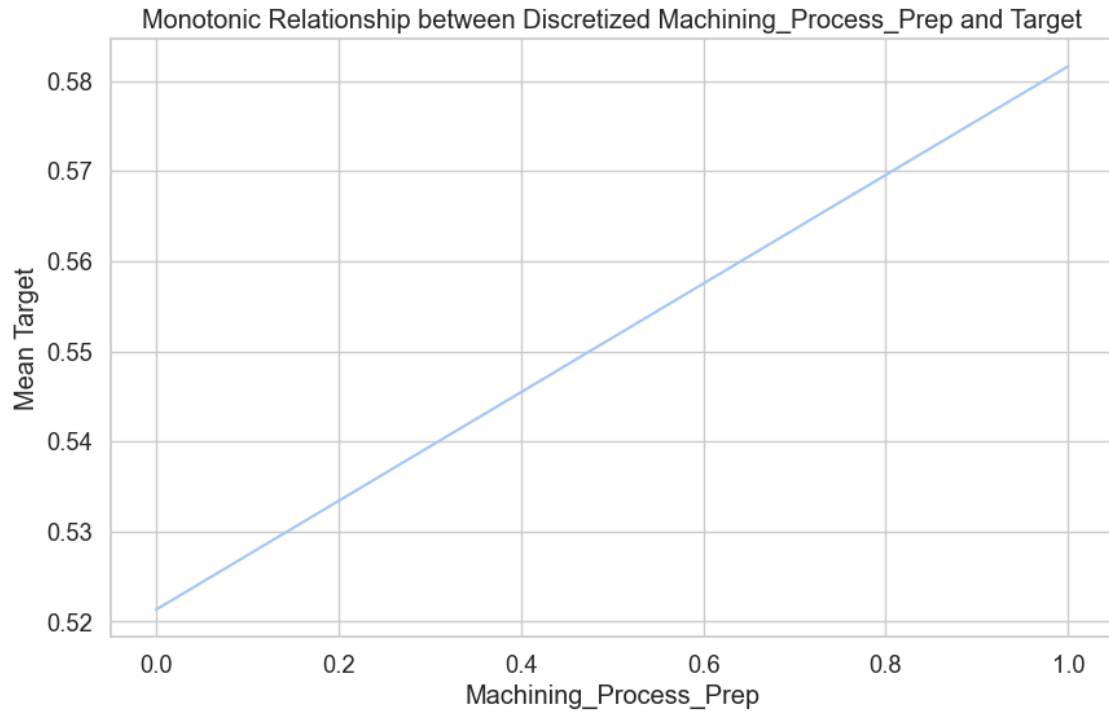


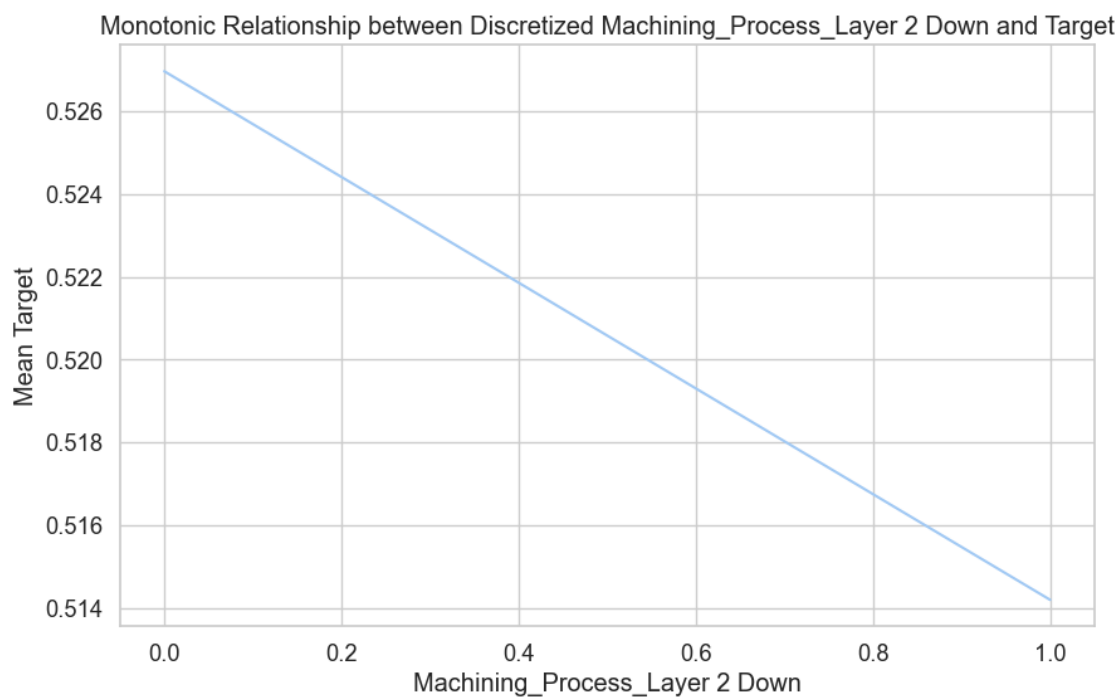
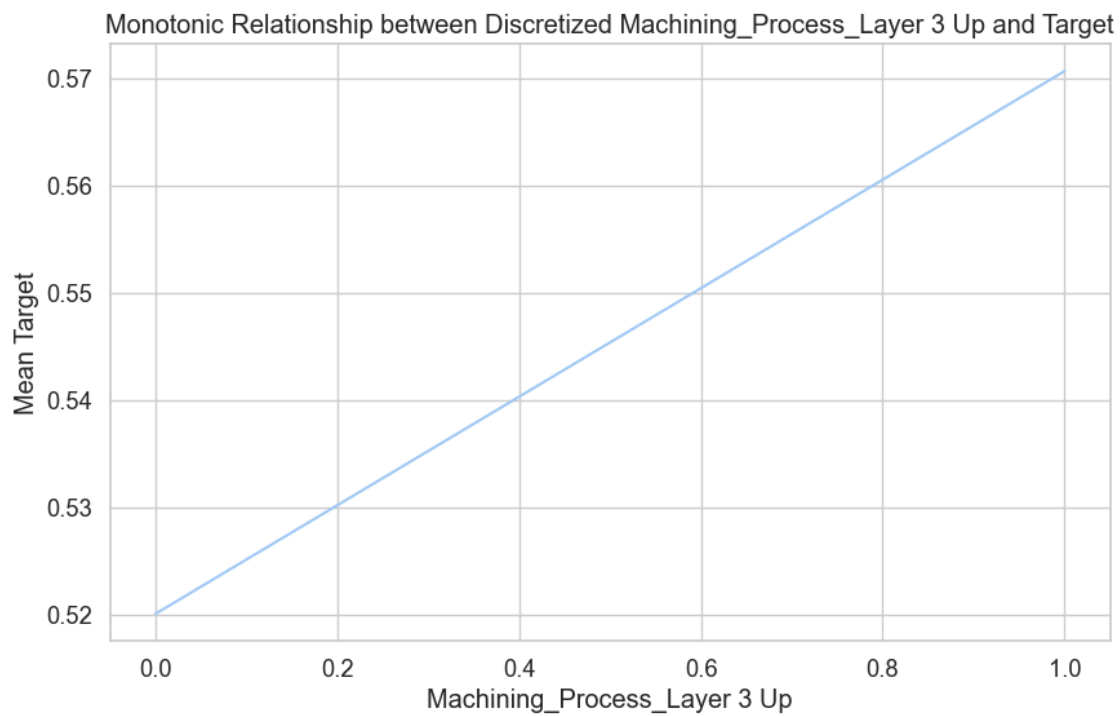


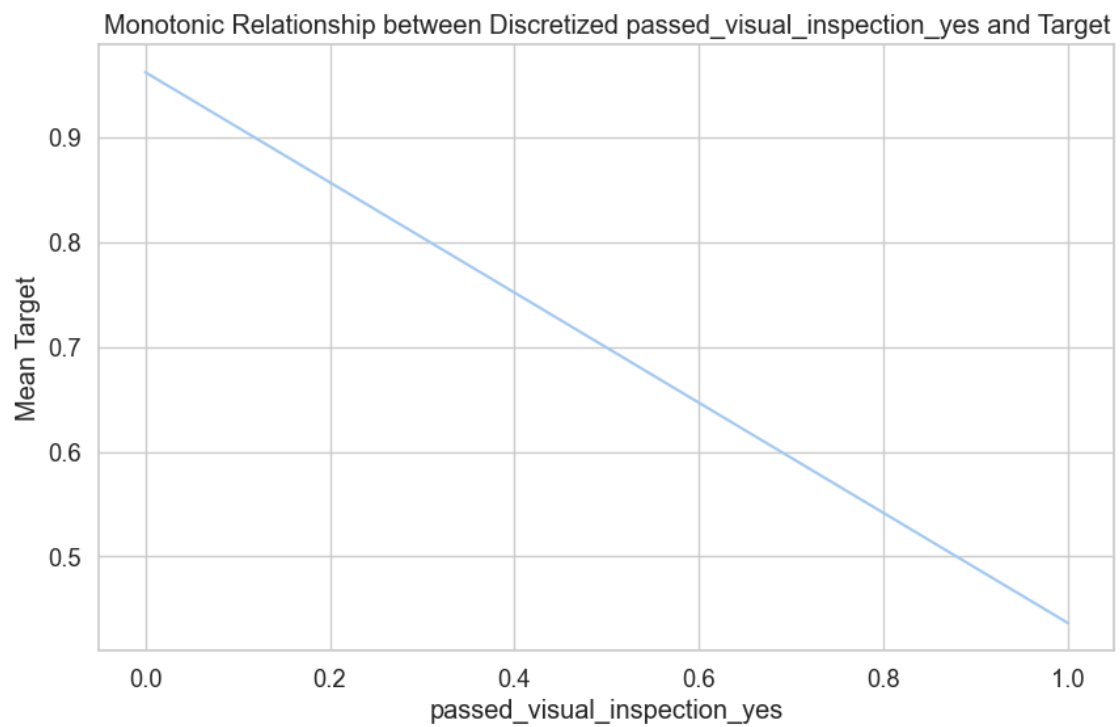
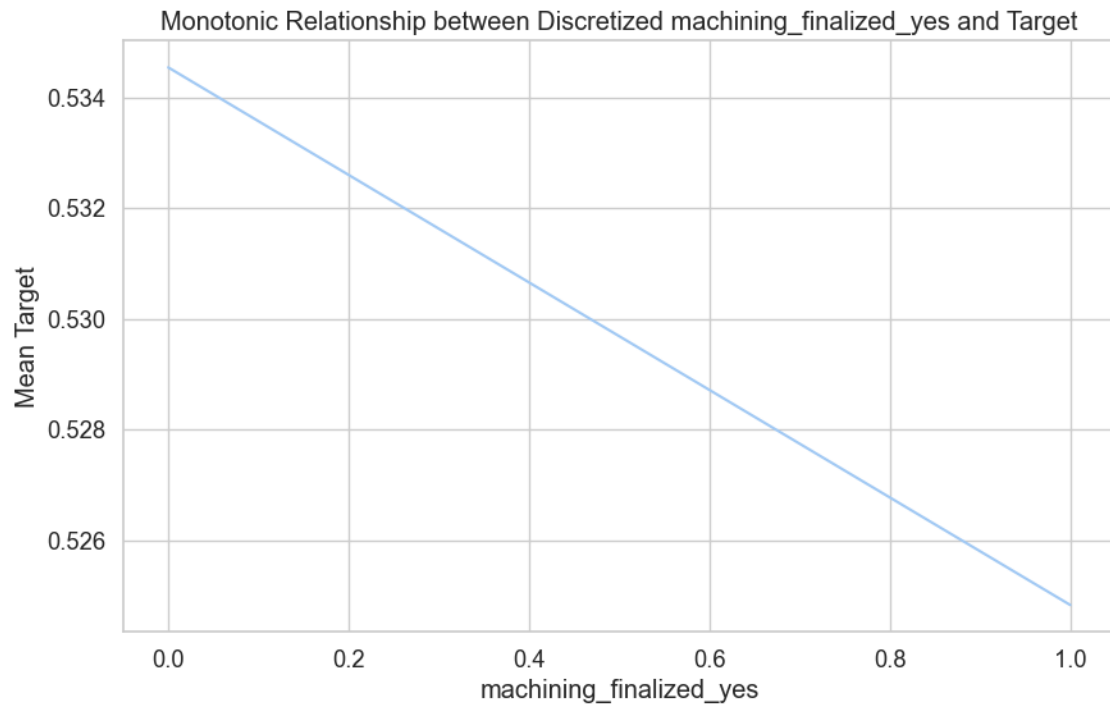










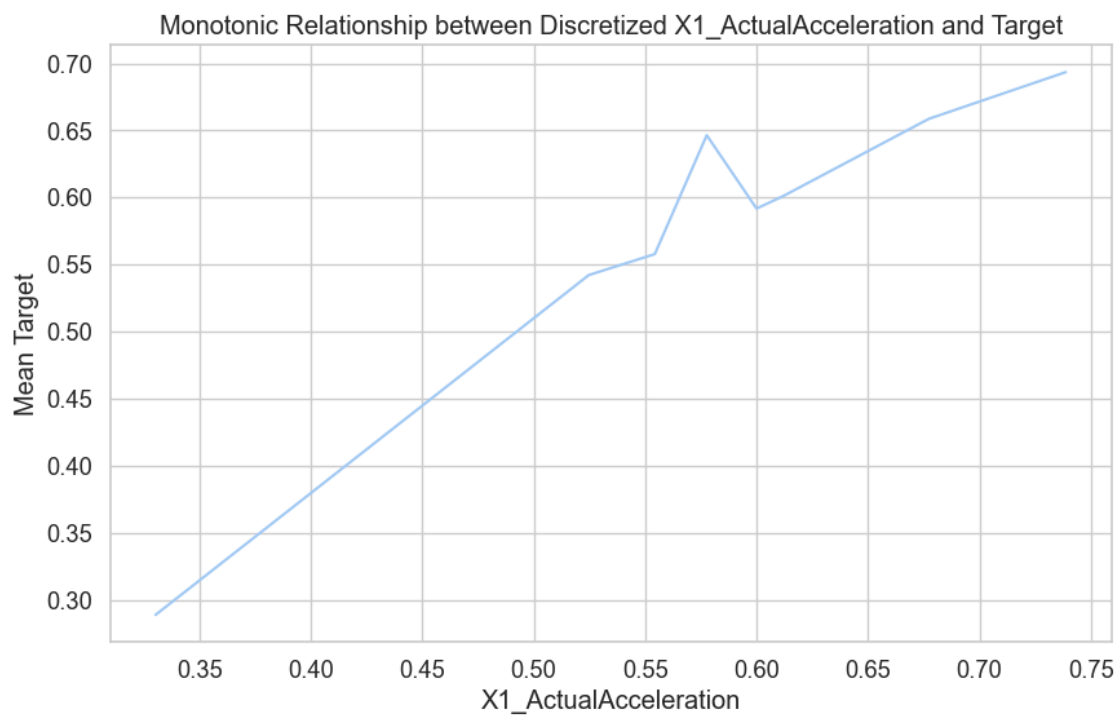


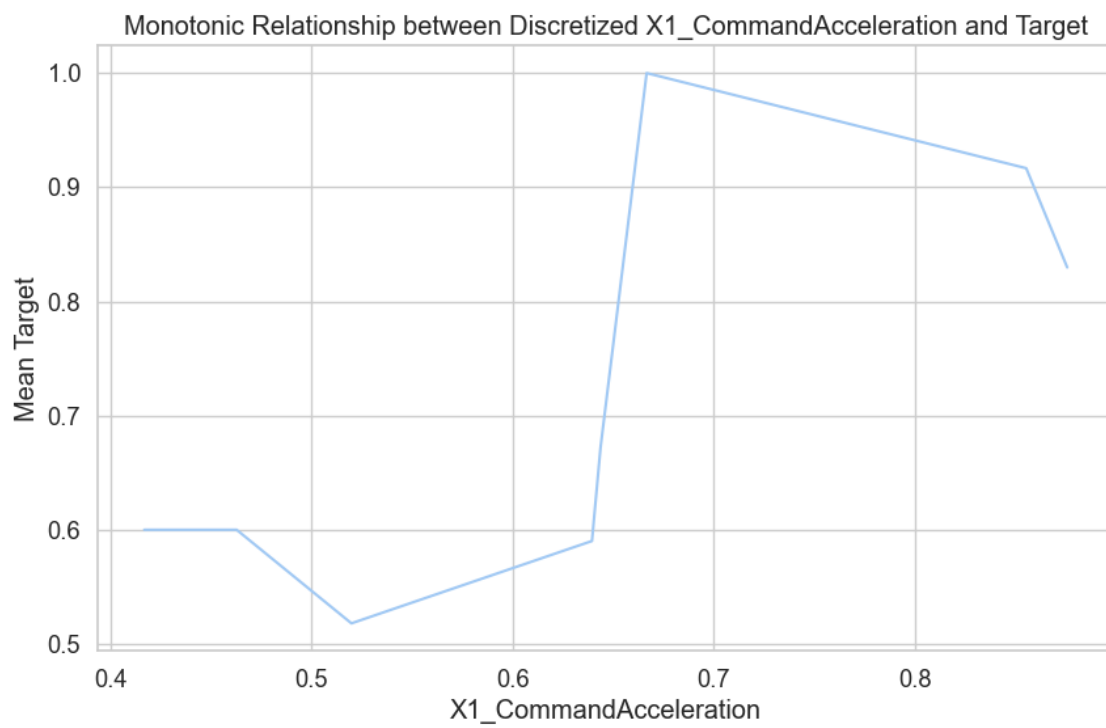
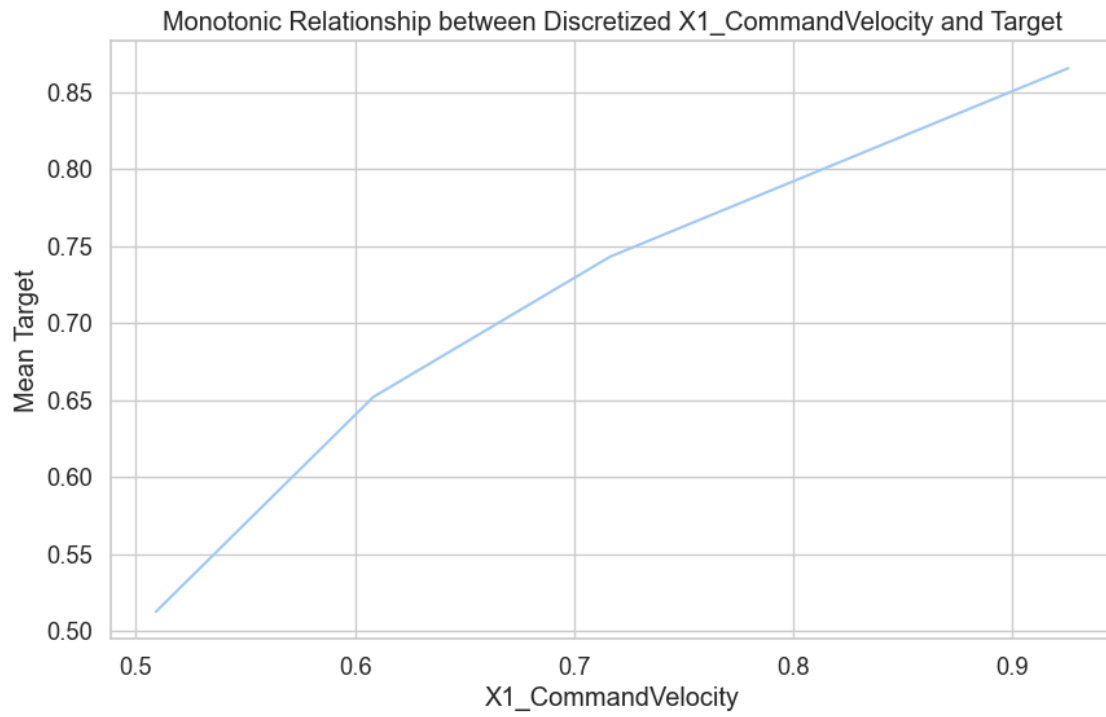
```
[50]: for col in test_t.columns:
    plt.figure(figsize=(10, 6))

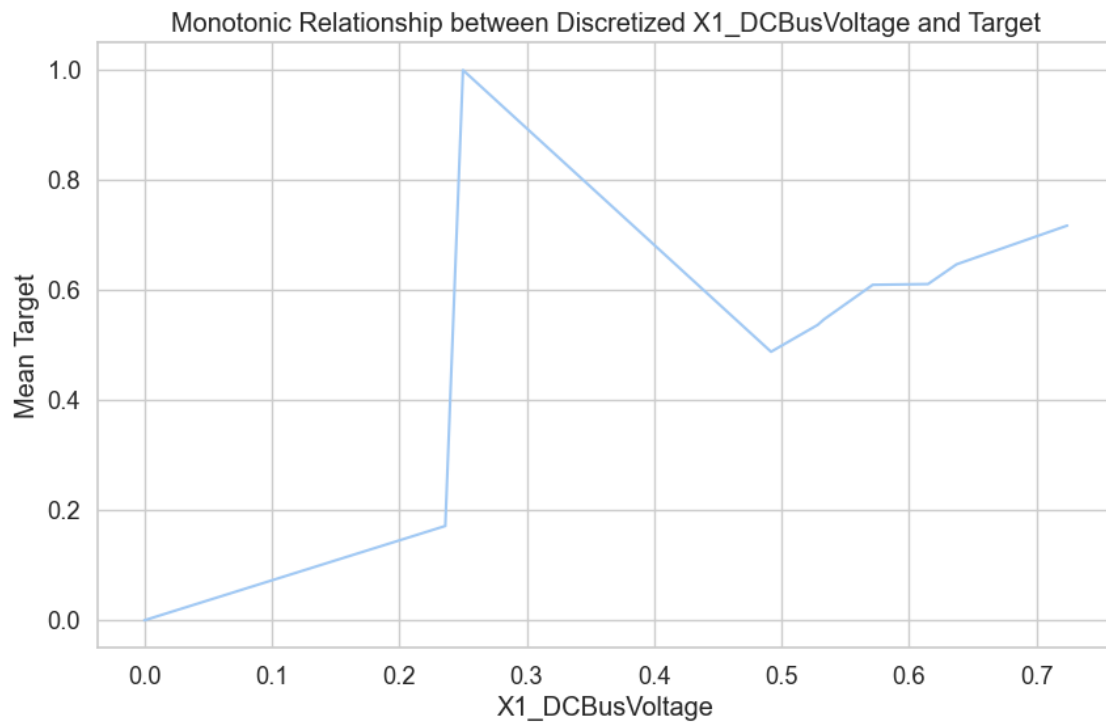
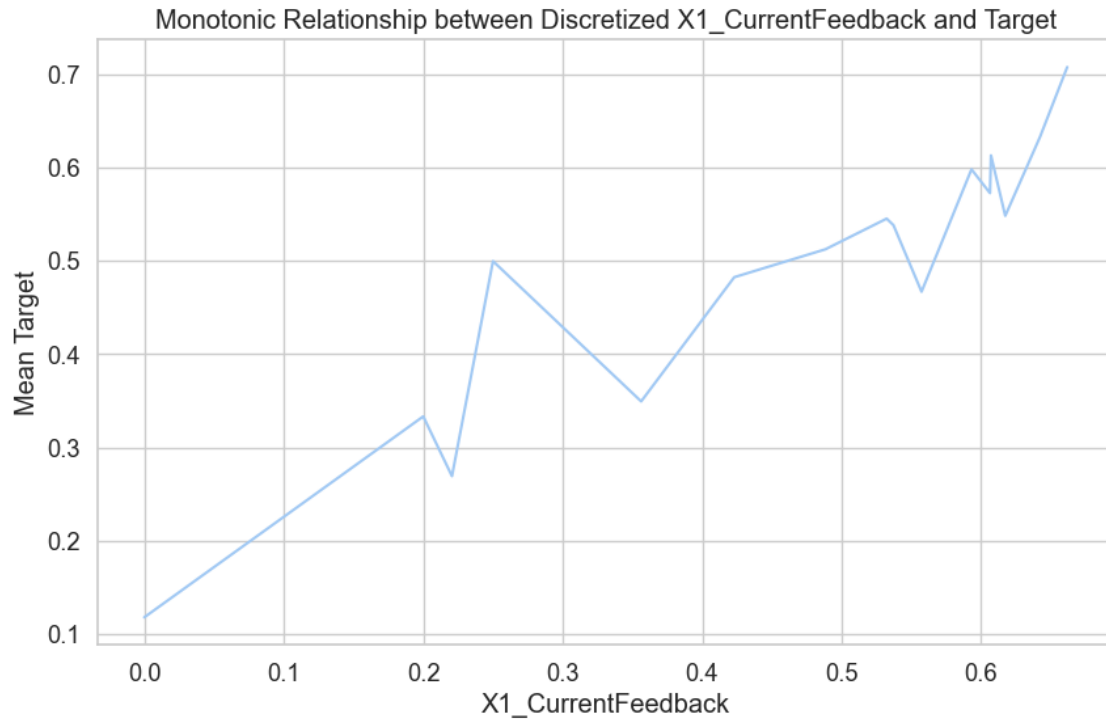
    combined_data = pd.concat([test_t, y_test], axis=1)
    grouped_data = combined_data.groupby(col)['target'].mean()
    grouped_data.plot()

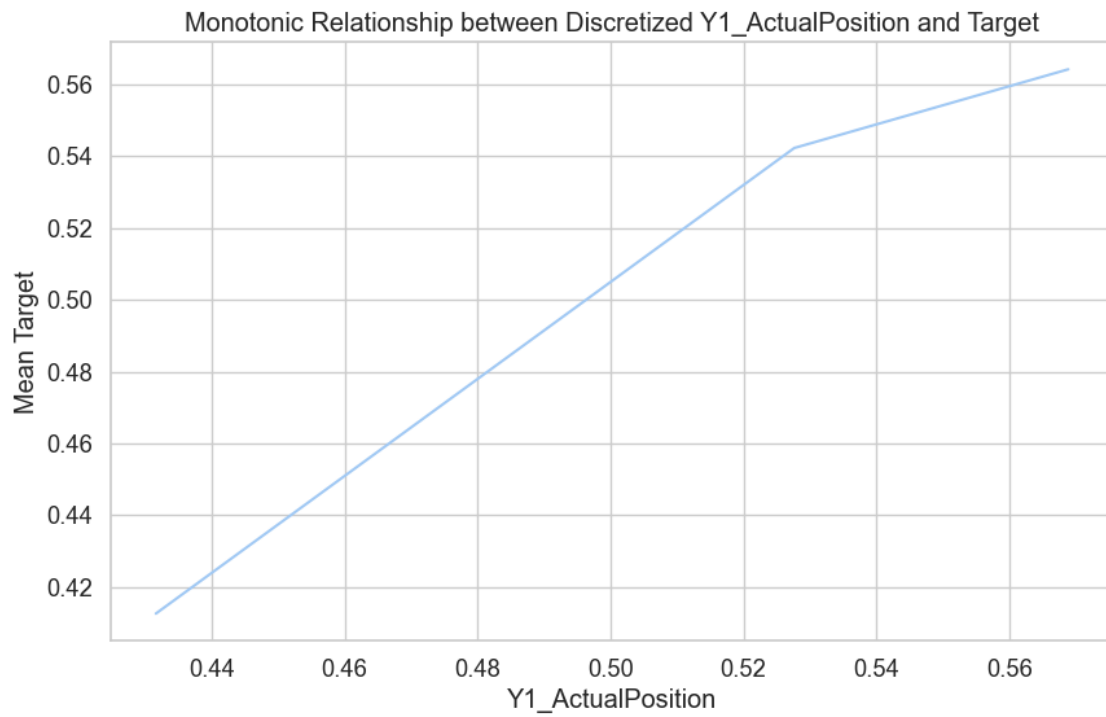
    plt.title(f'Monotonic Relationship between Discretized {col} and Target')
    plt.xlabel(col)
    plt.ylabel('Mean Target')
    plt.grid(True)

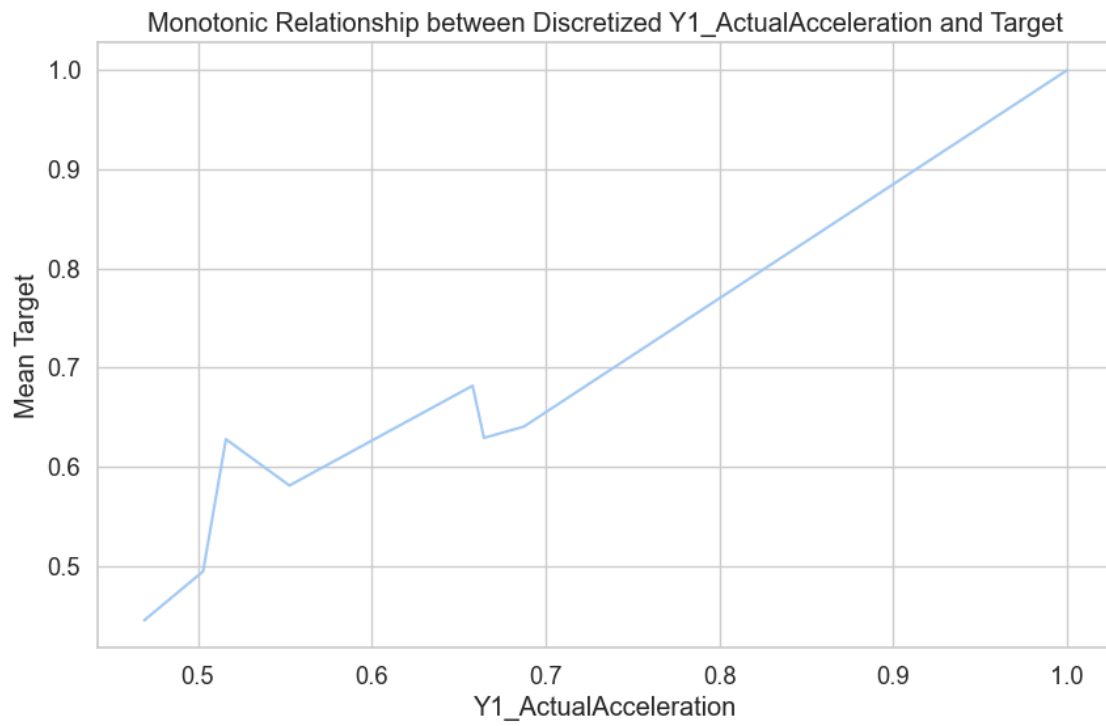
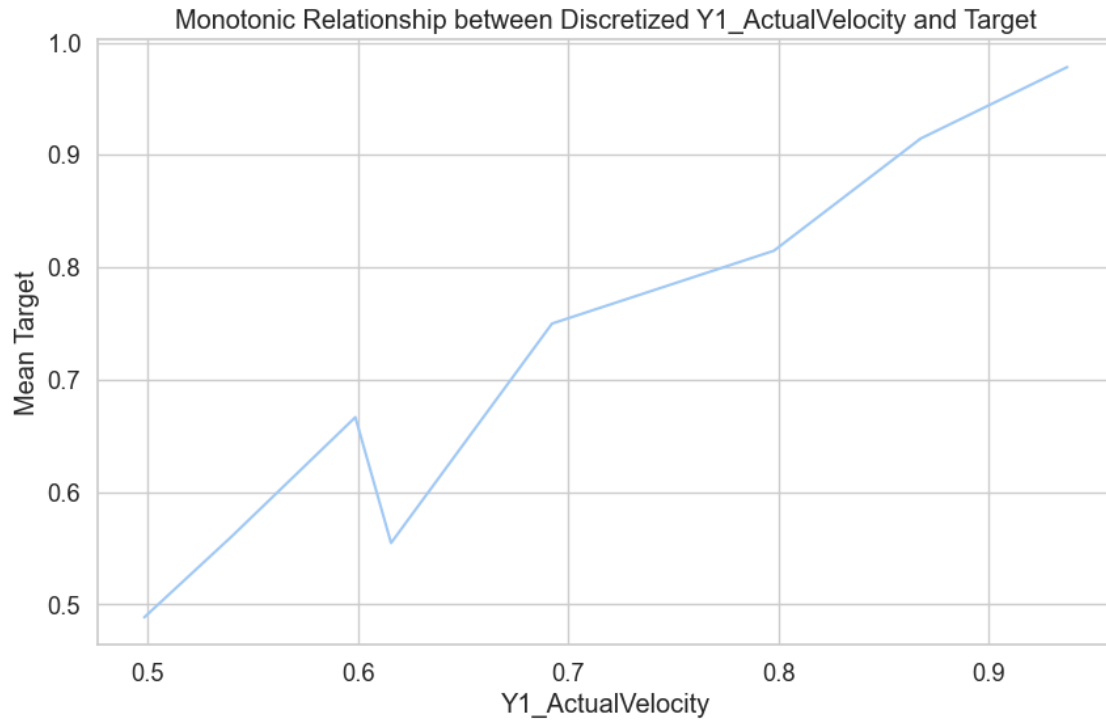
    plt.show()
```

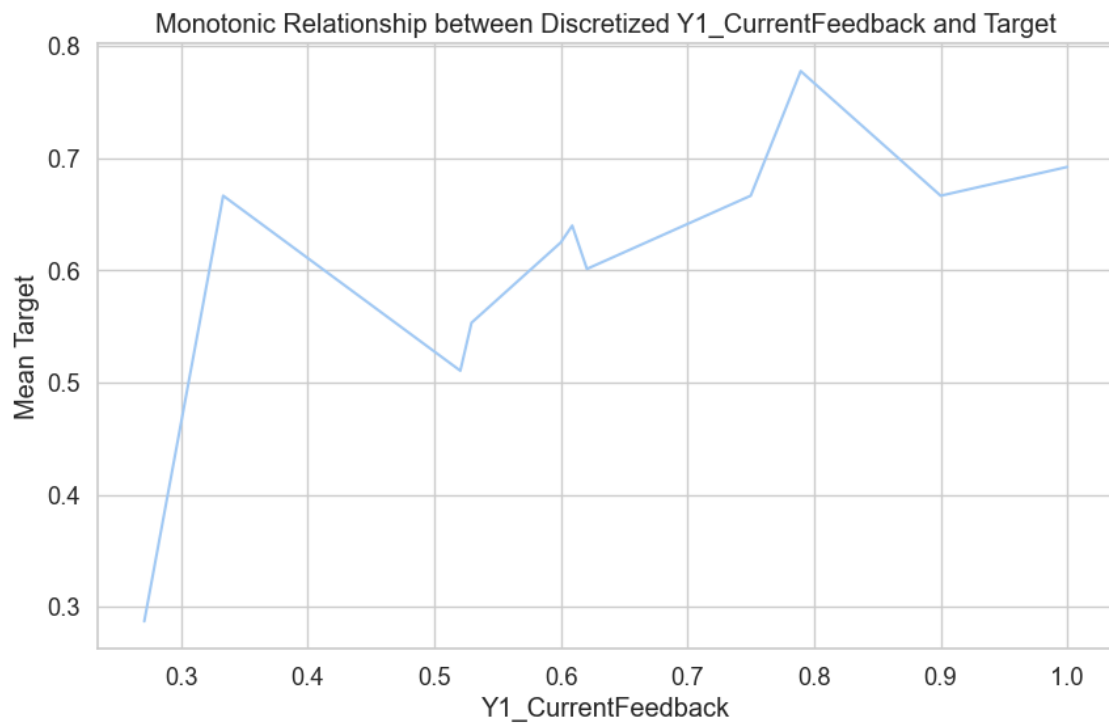
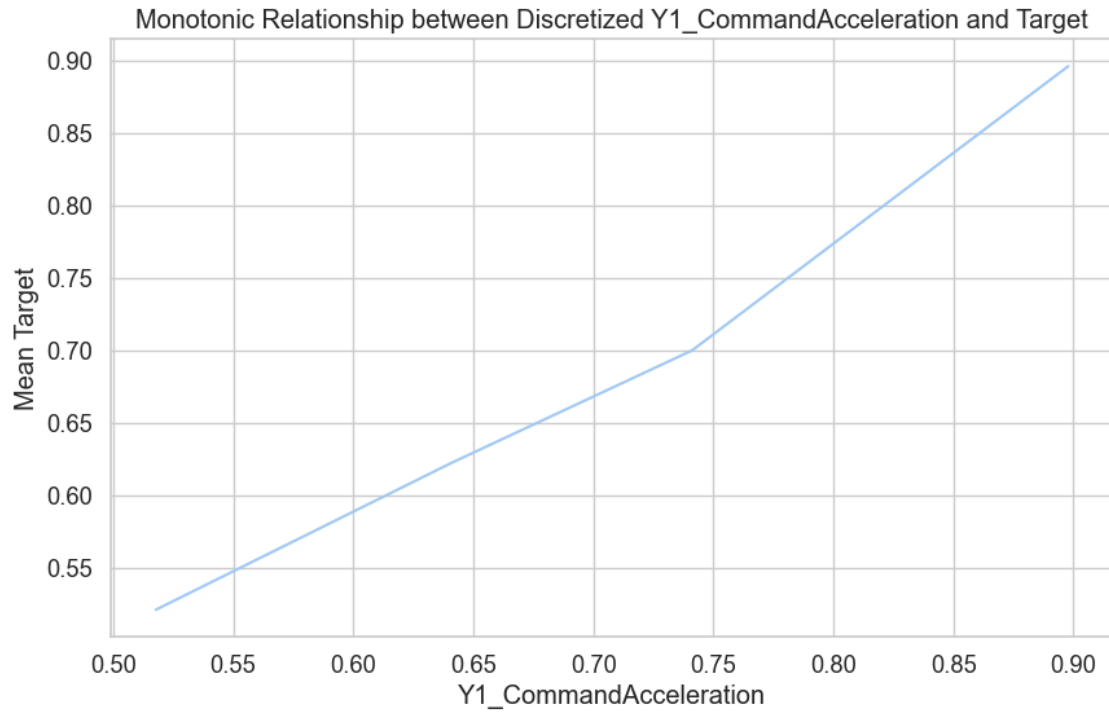


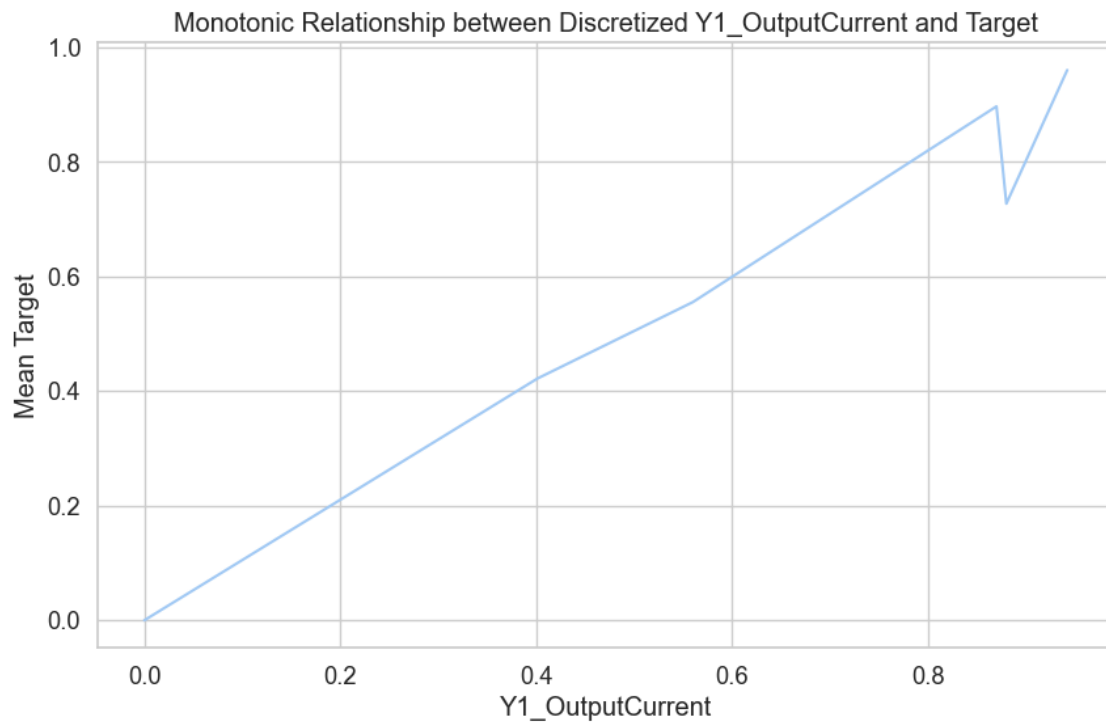
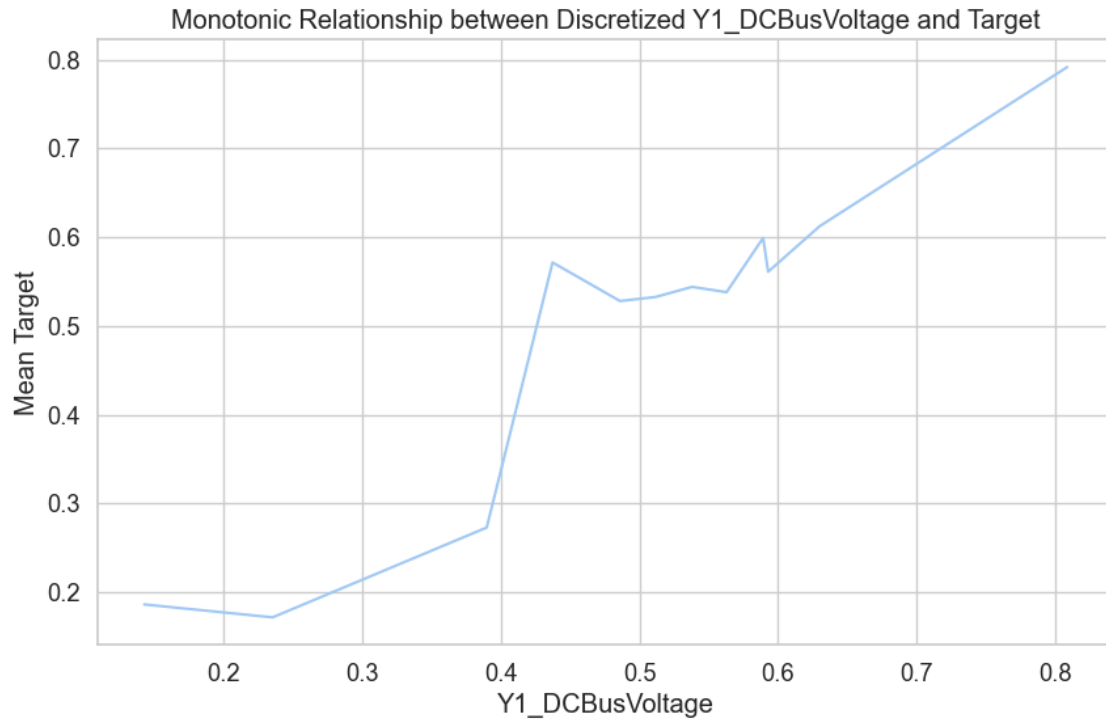


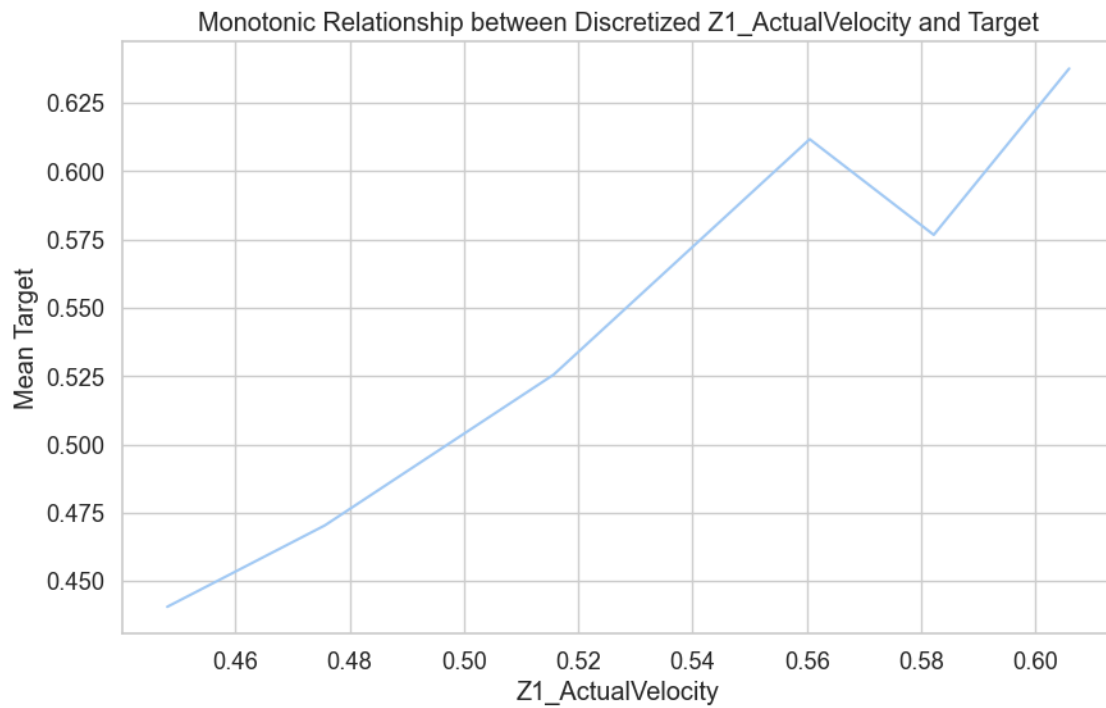
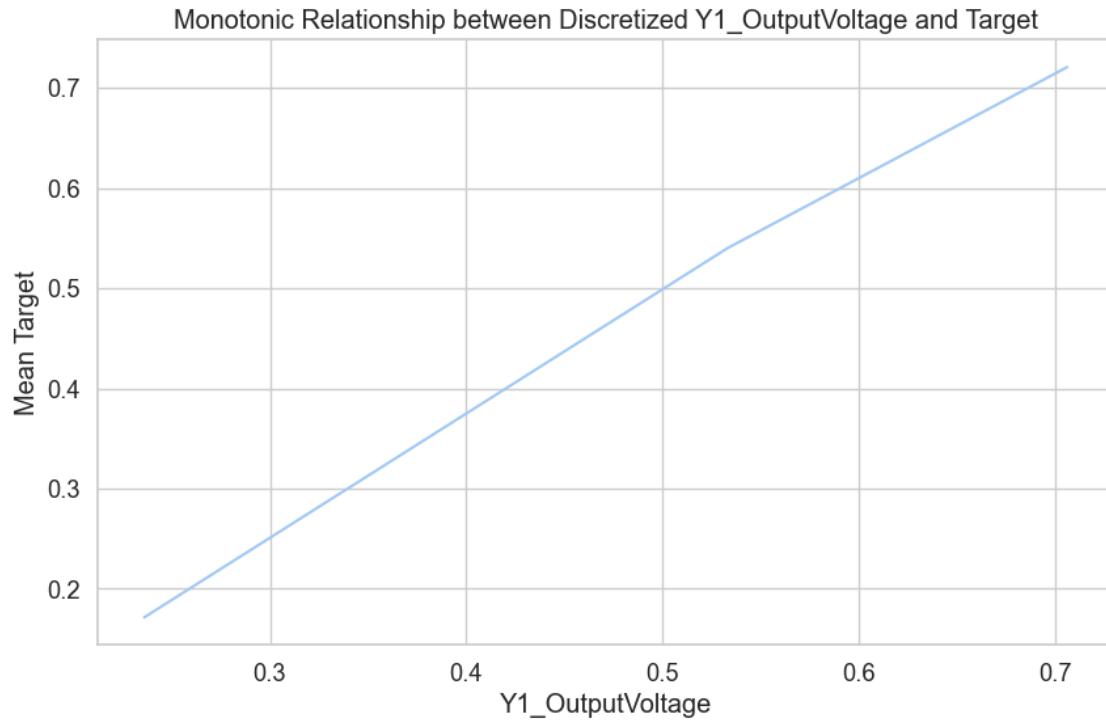


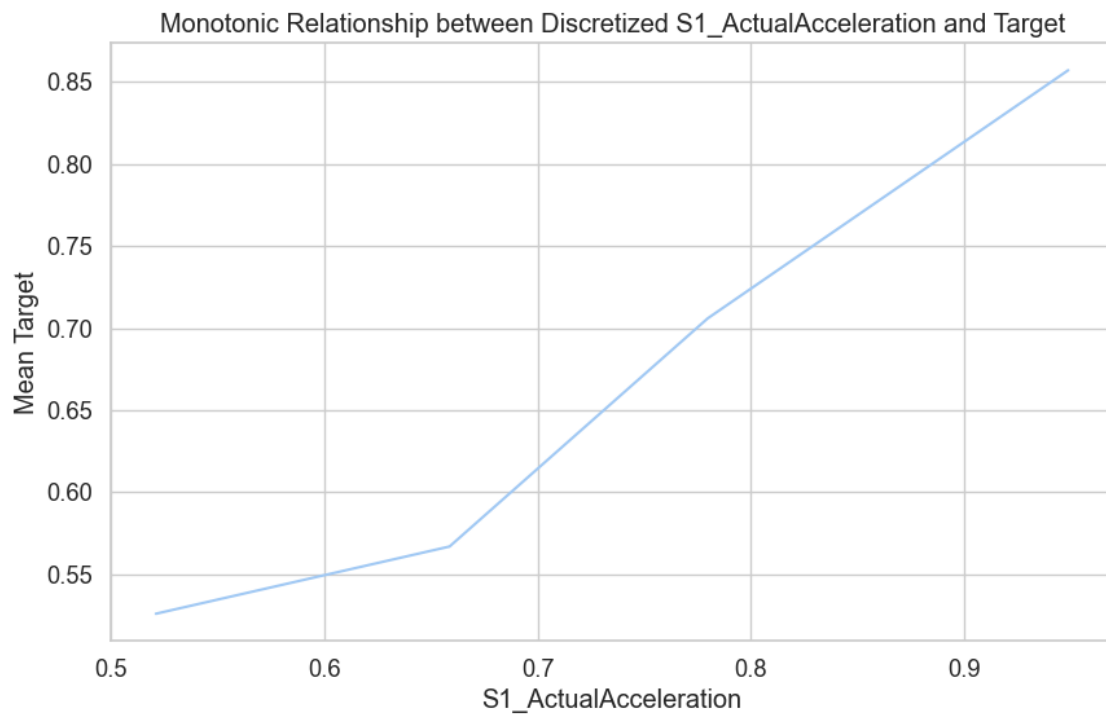
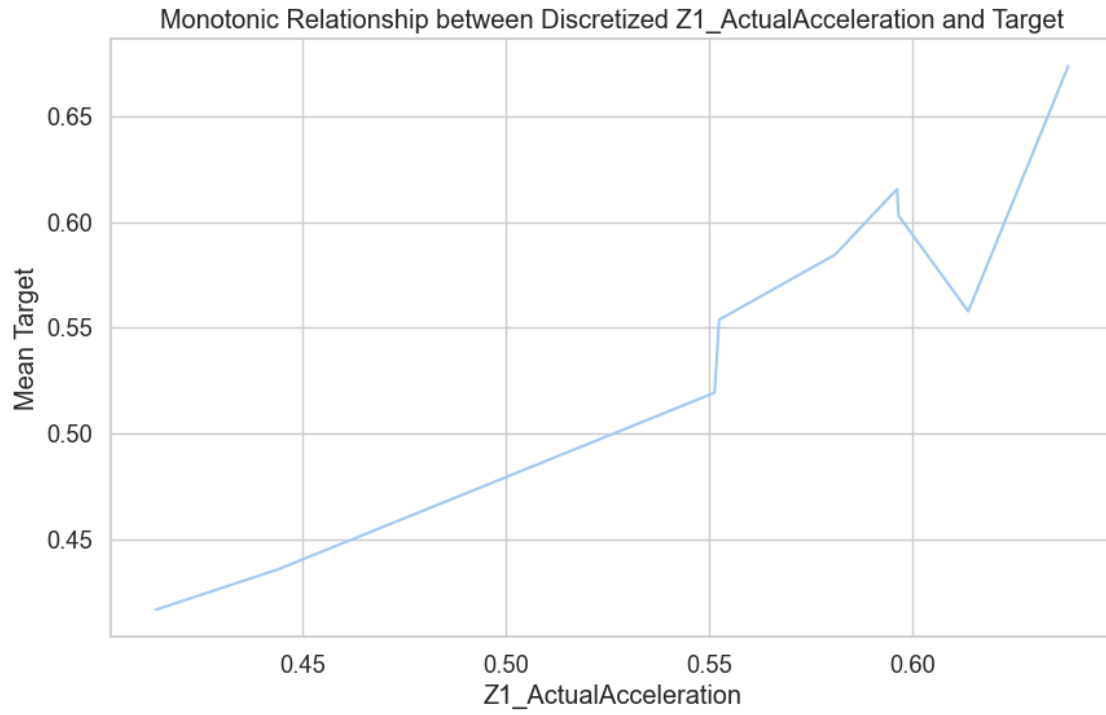


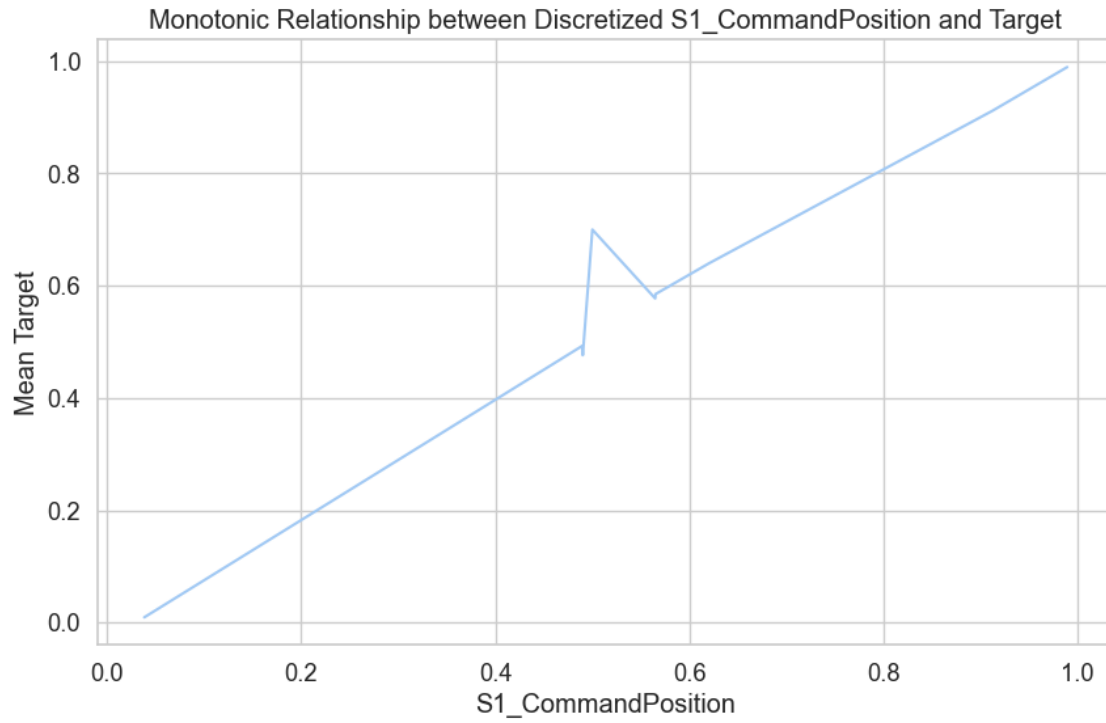


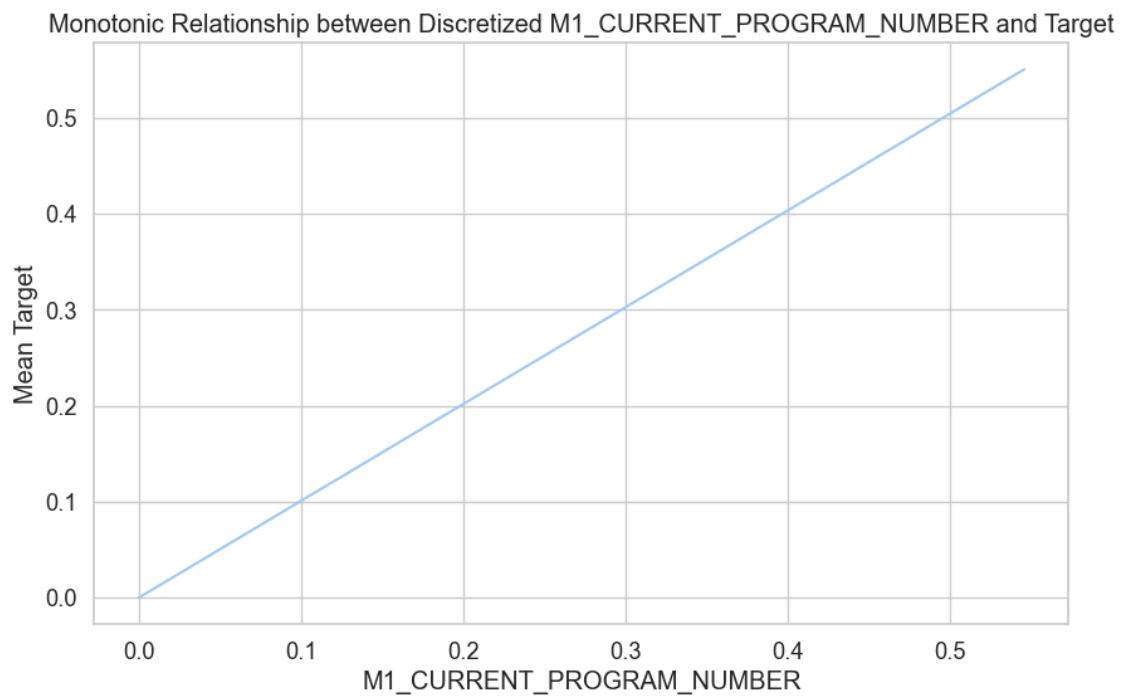
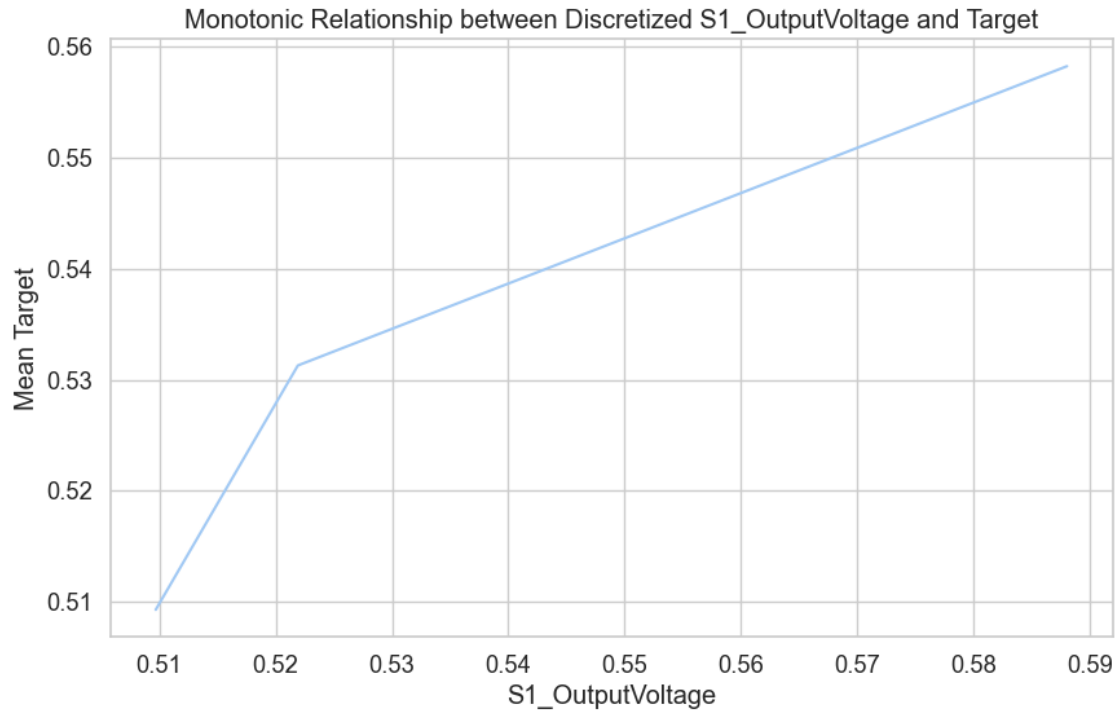


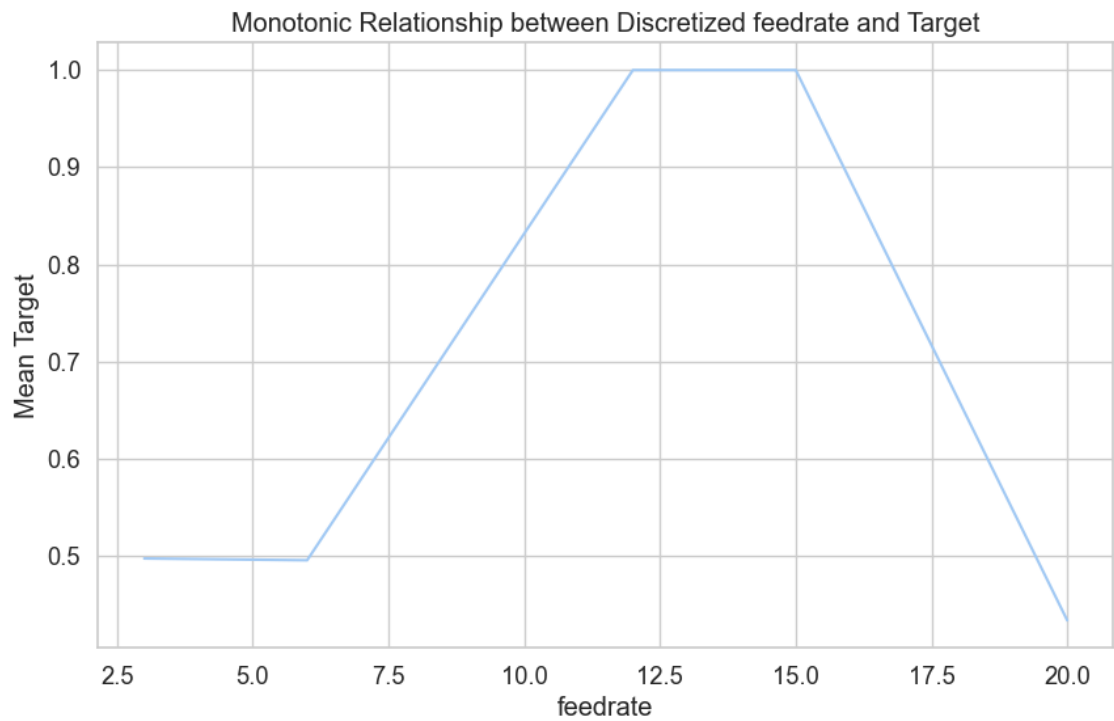
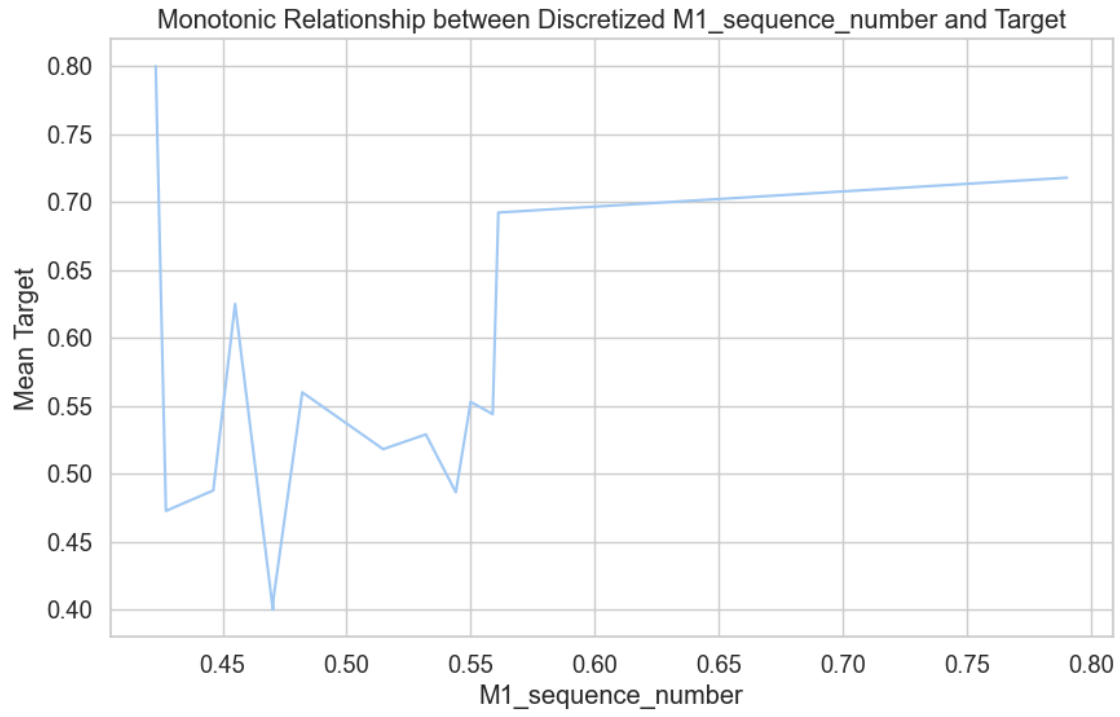


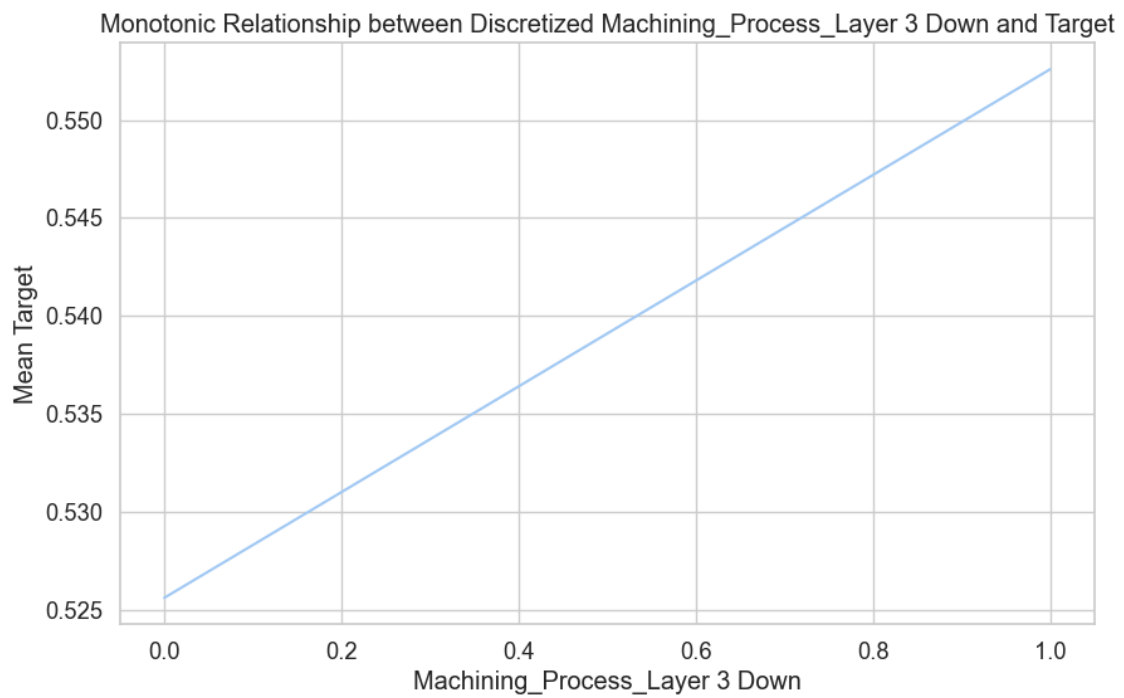
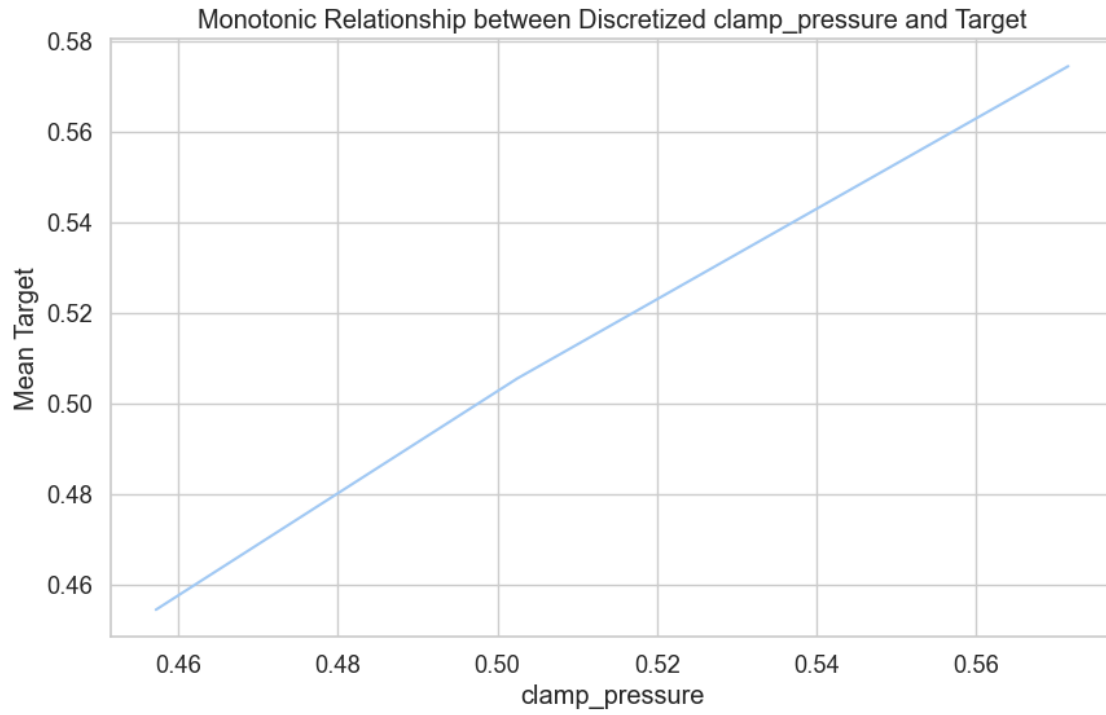


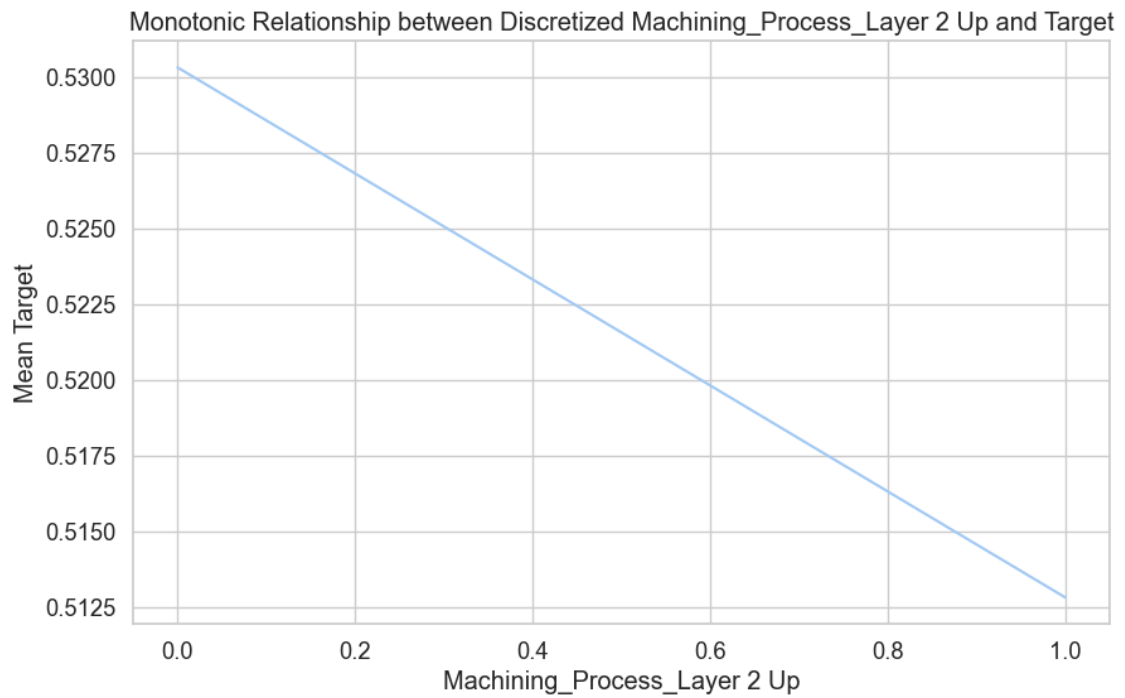
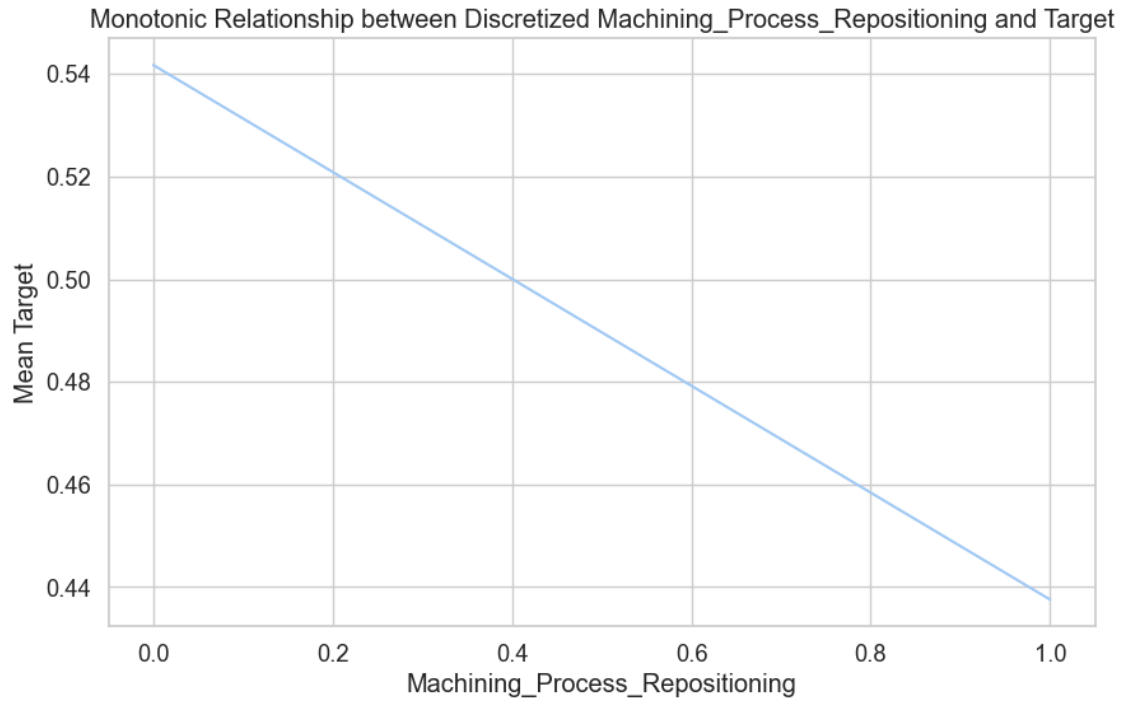


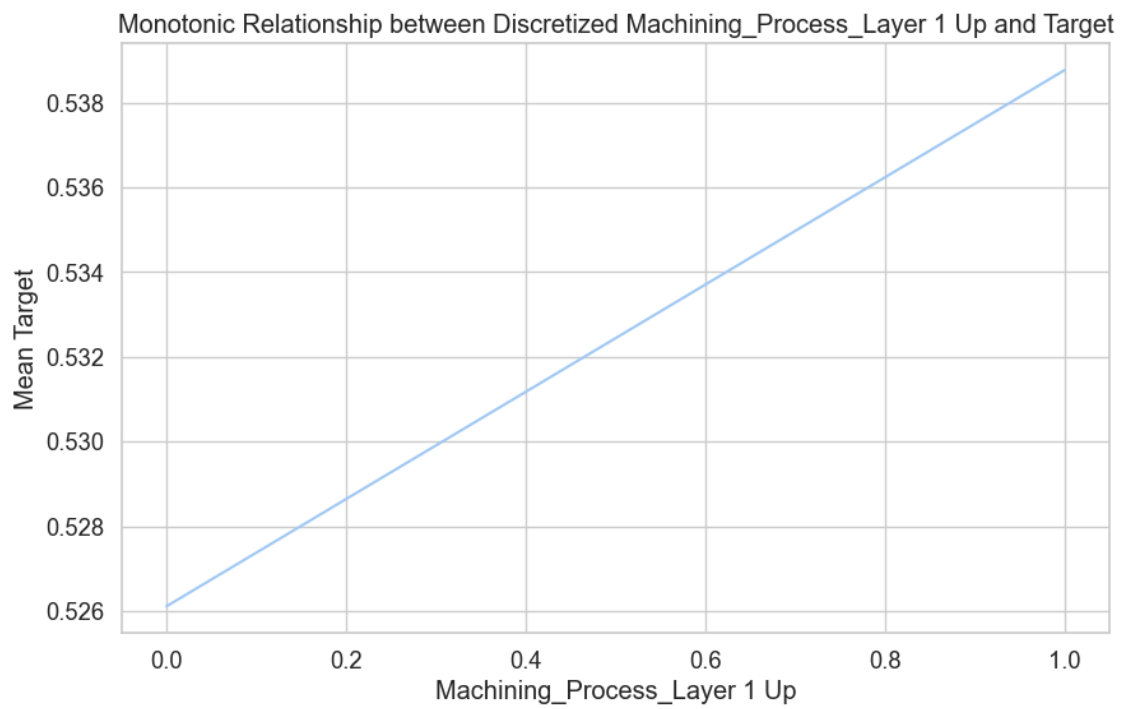
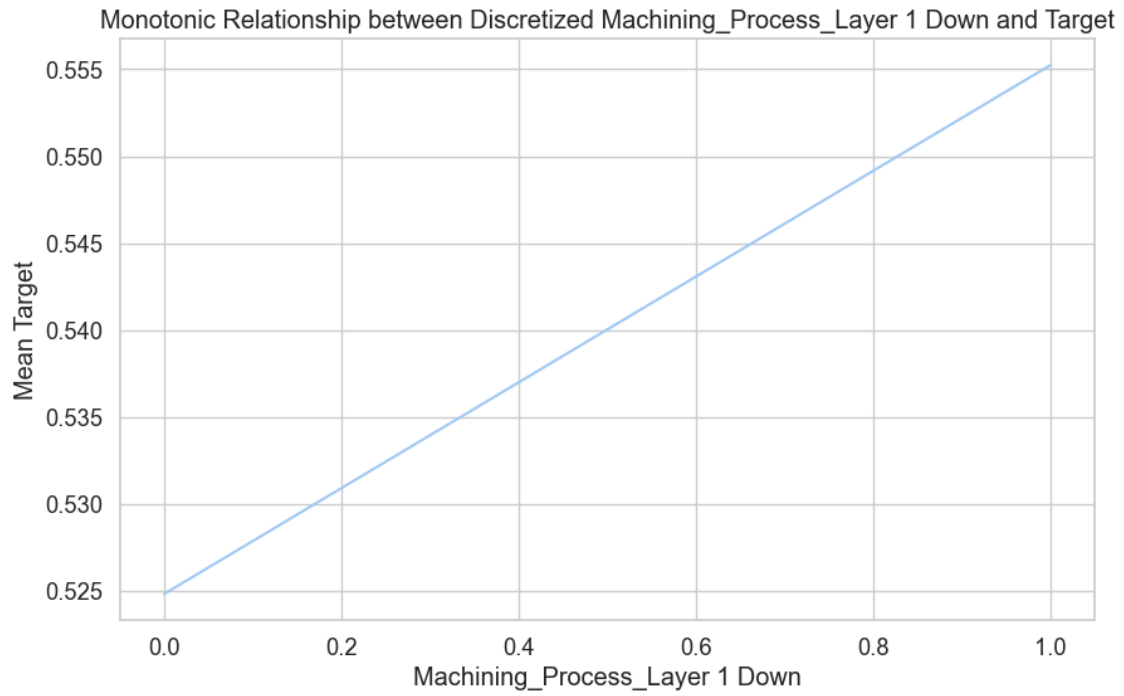


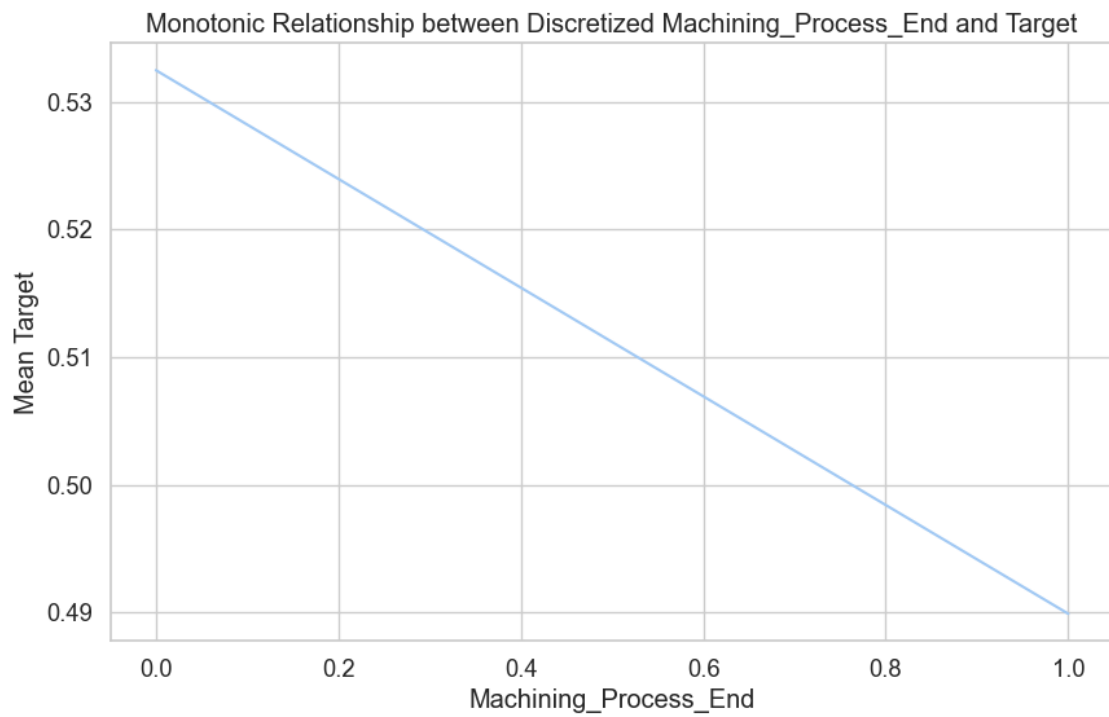
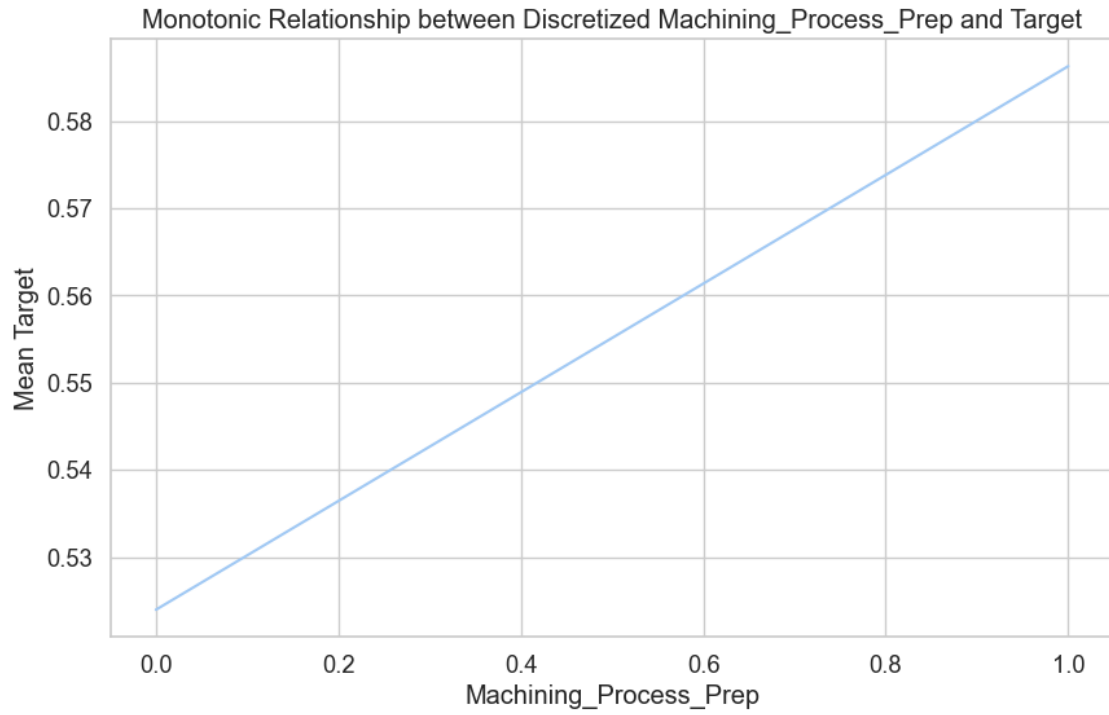


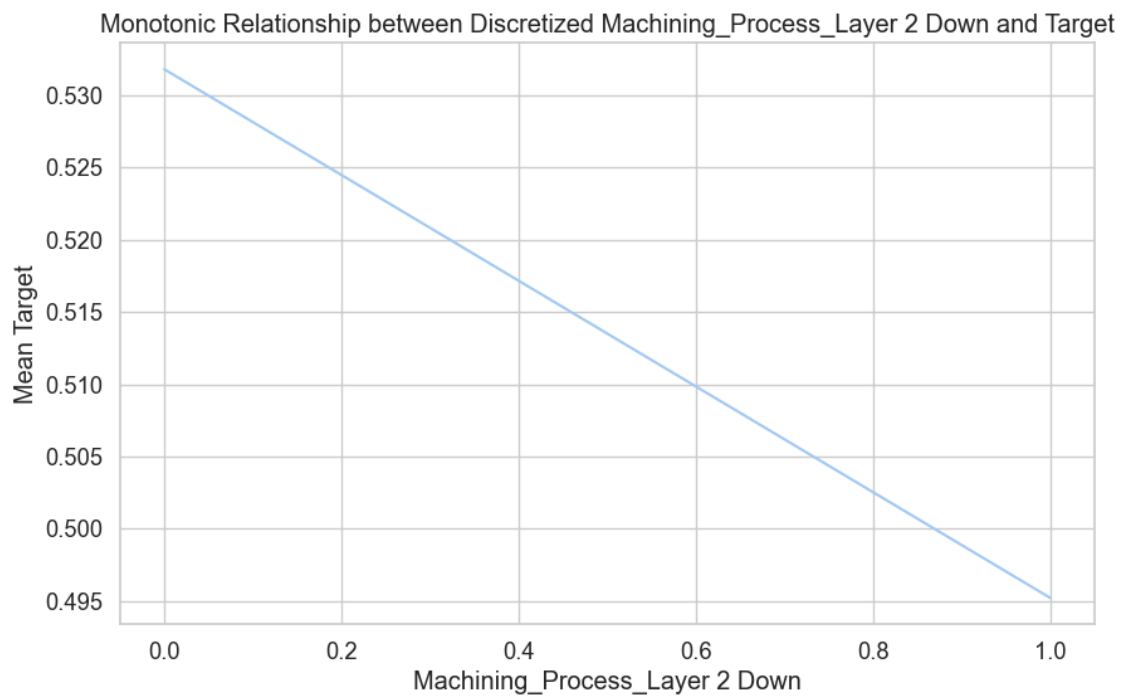
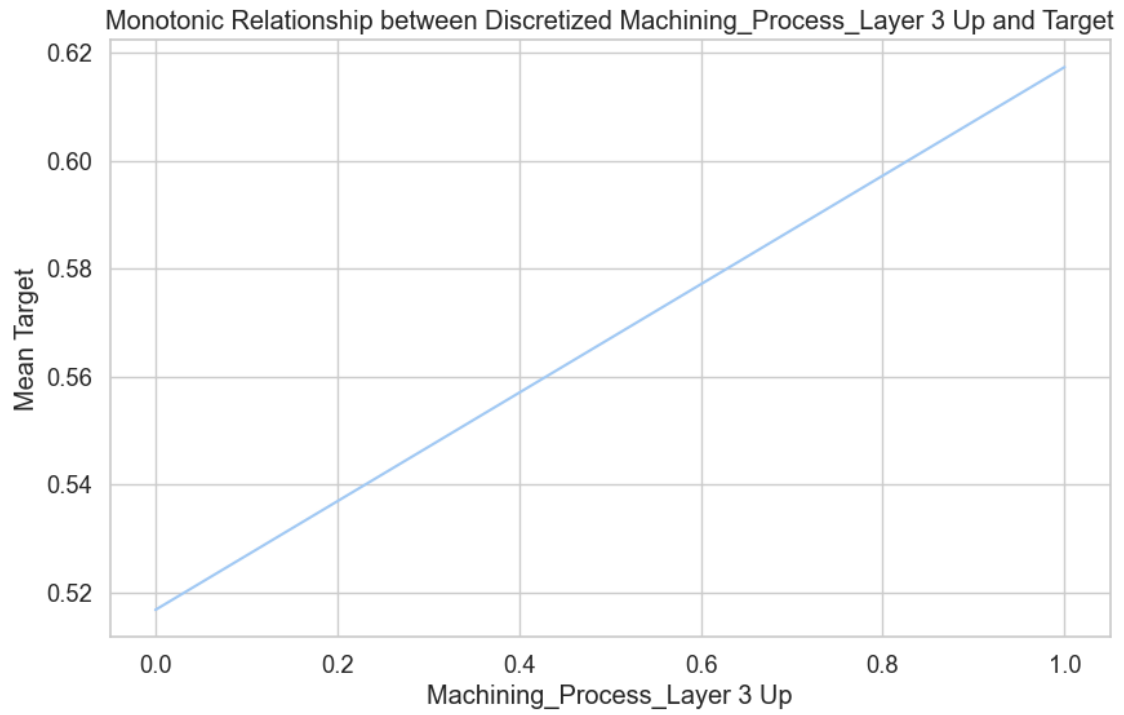


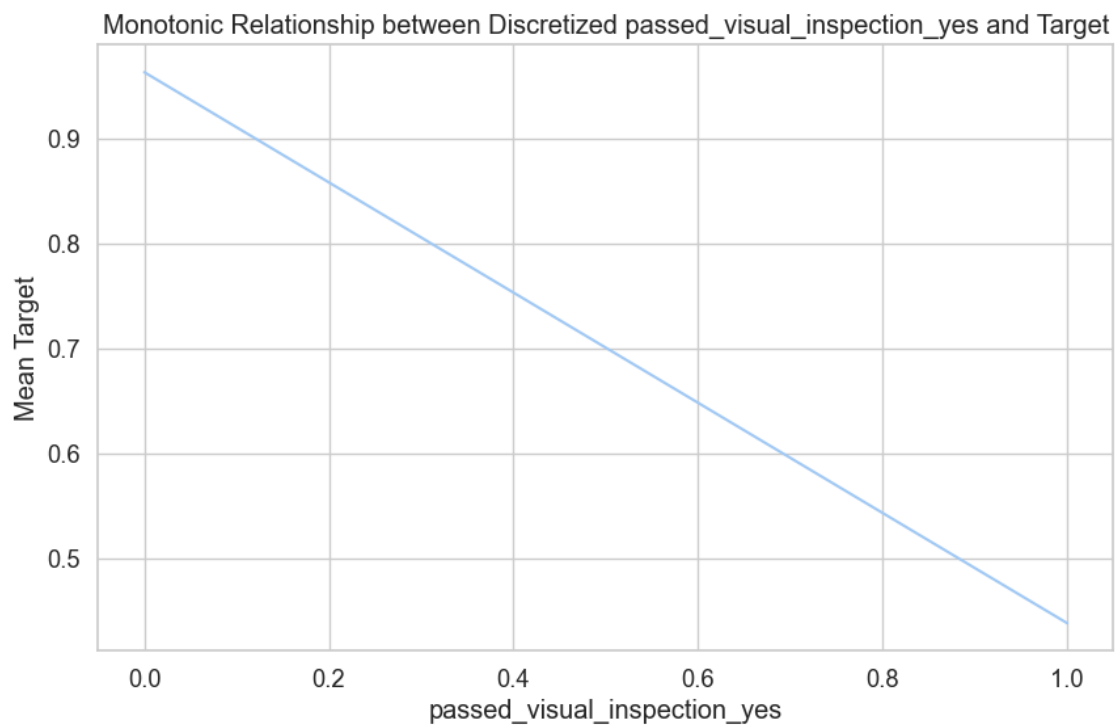
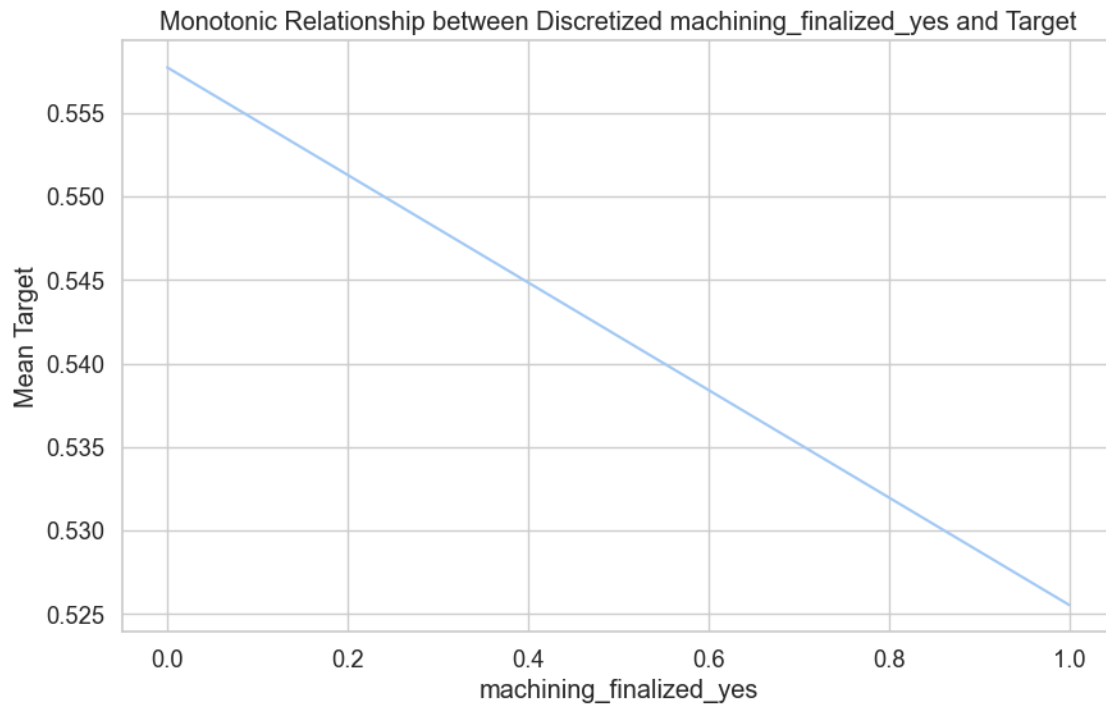








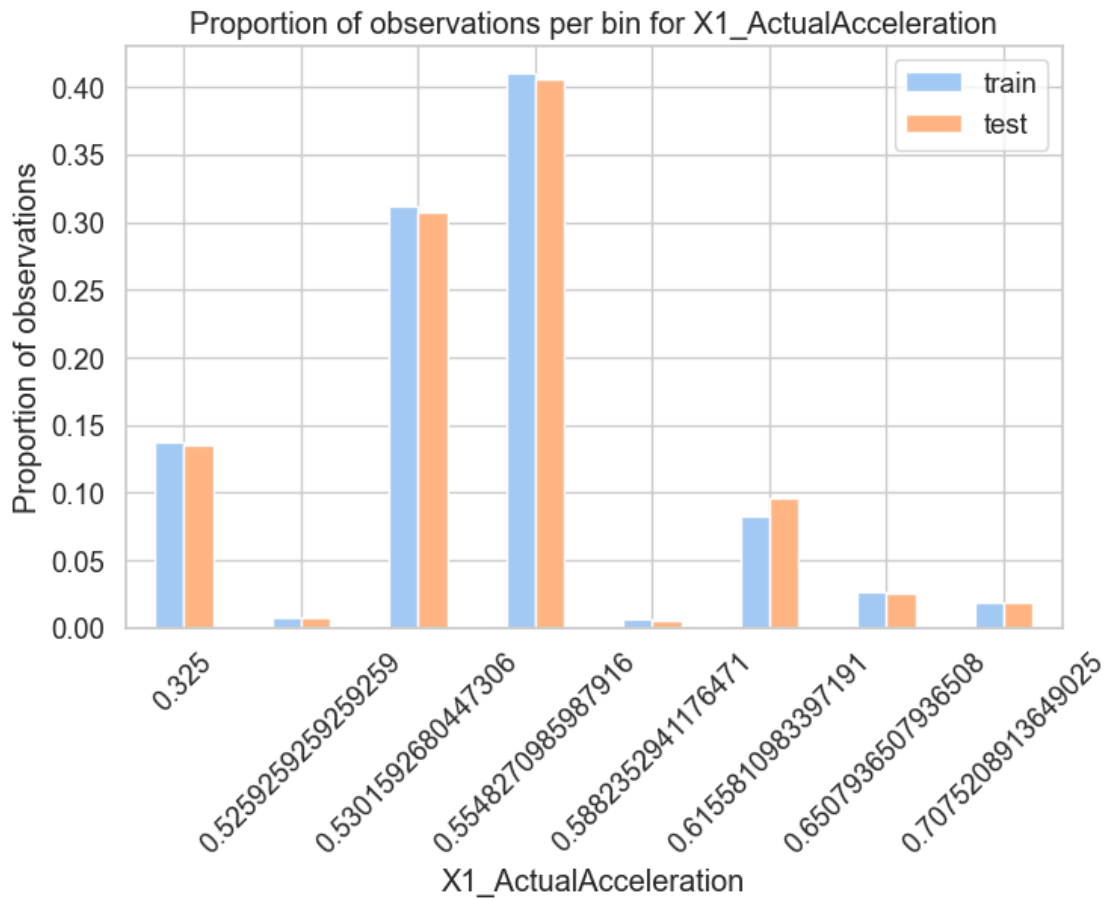


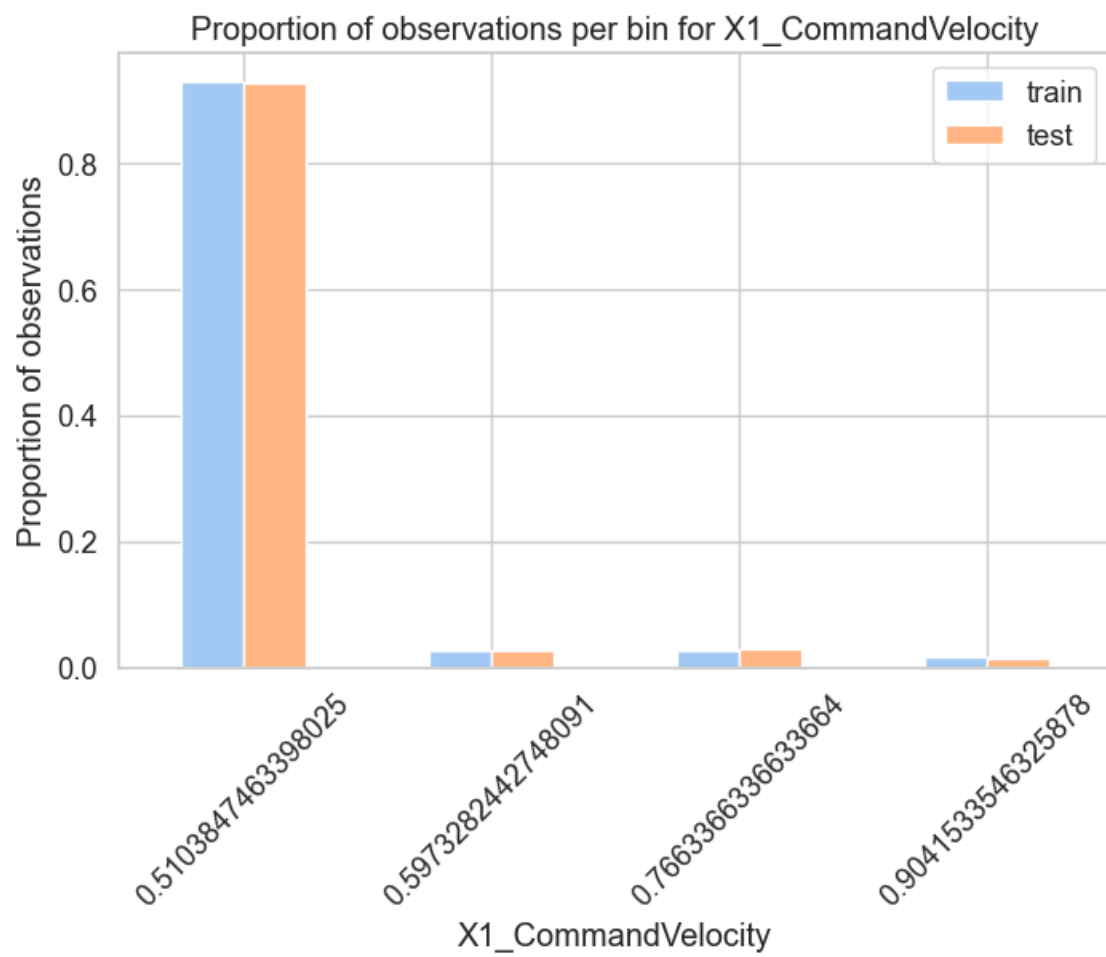


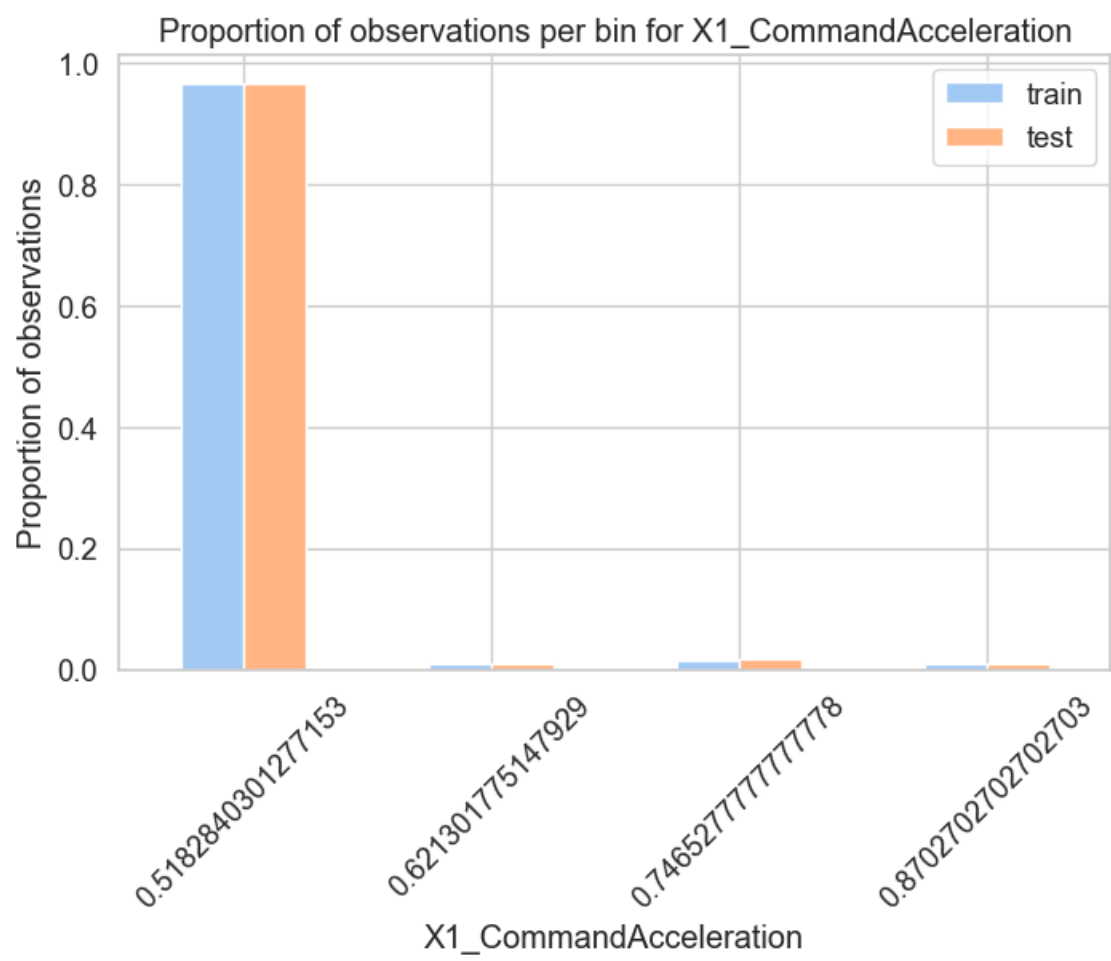
```
[46]: for col in train_t.columns:
    # take bins for each variables by train and test sets
    t1 = train_t.groupby([col])[col].count() / len(train_t)
    t2 = test_t.groupby([col])[col].count() / len(test_t)

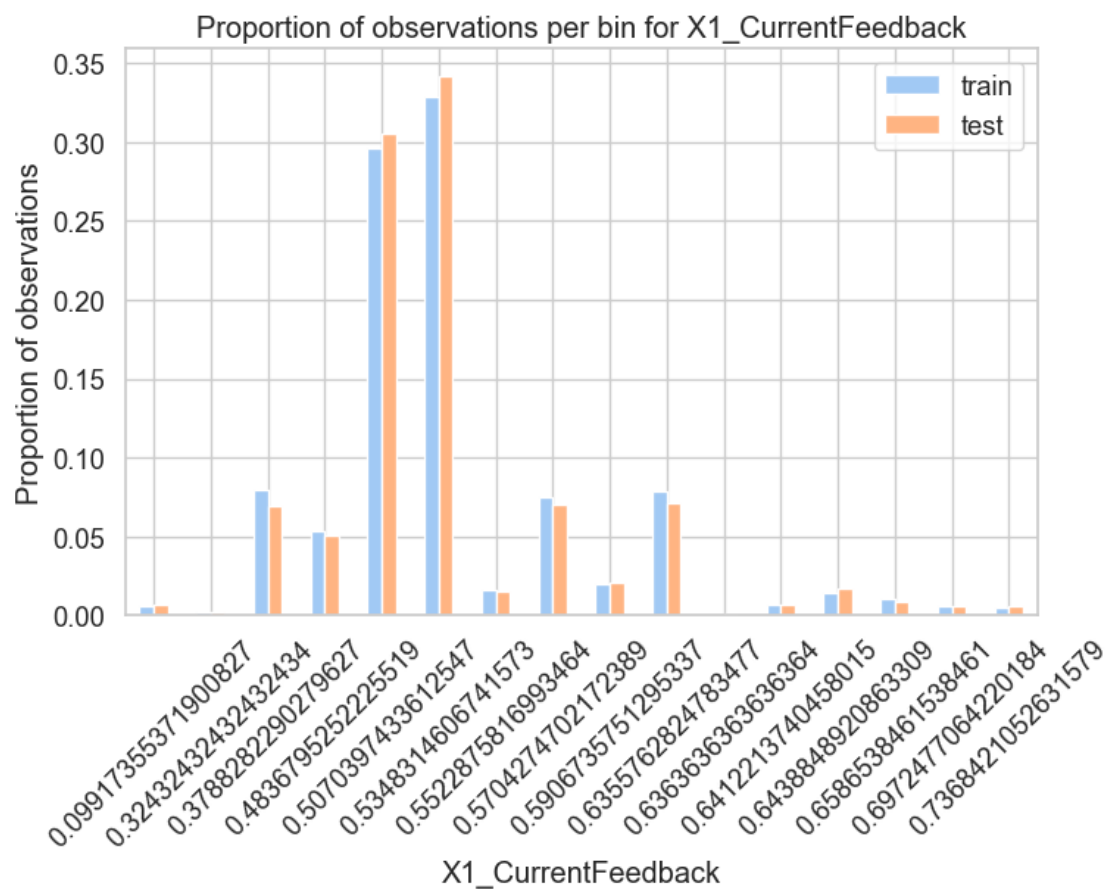
    # create dataframe for (tmp) var
    tmp = pd.concat([t1, t2], axis=1)
    tmp.columns = ['train', 'test']

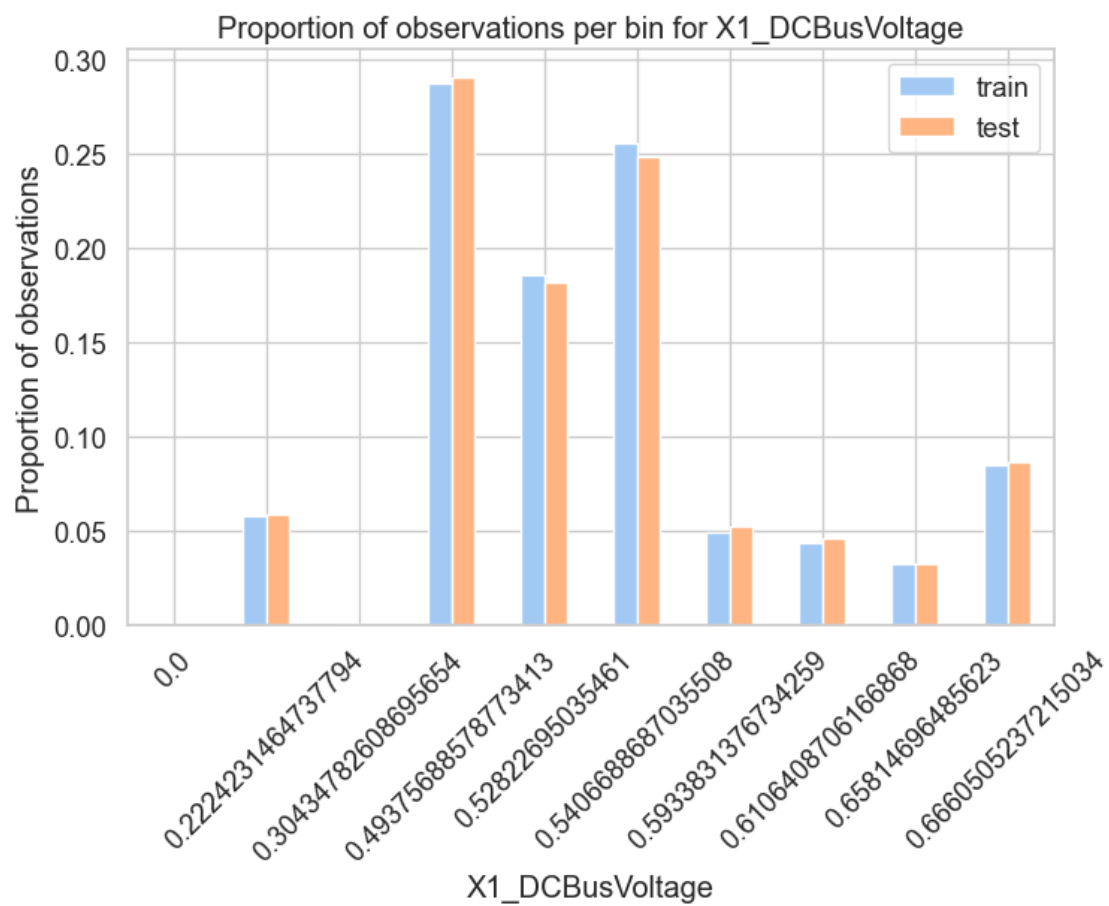
    # build graph
    tmp.plot(kind='bar', figsize=(8, 5))
    plt.xticks(rotation=45)
    plt.ylabel('Proportion of observations')
    plt.title(f'Proportion of observations per bin for {col}')
    plt.show()
```

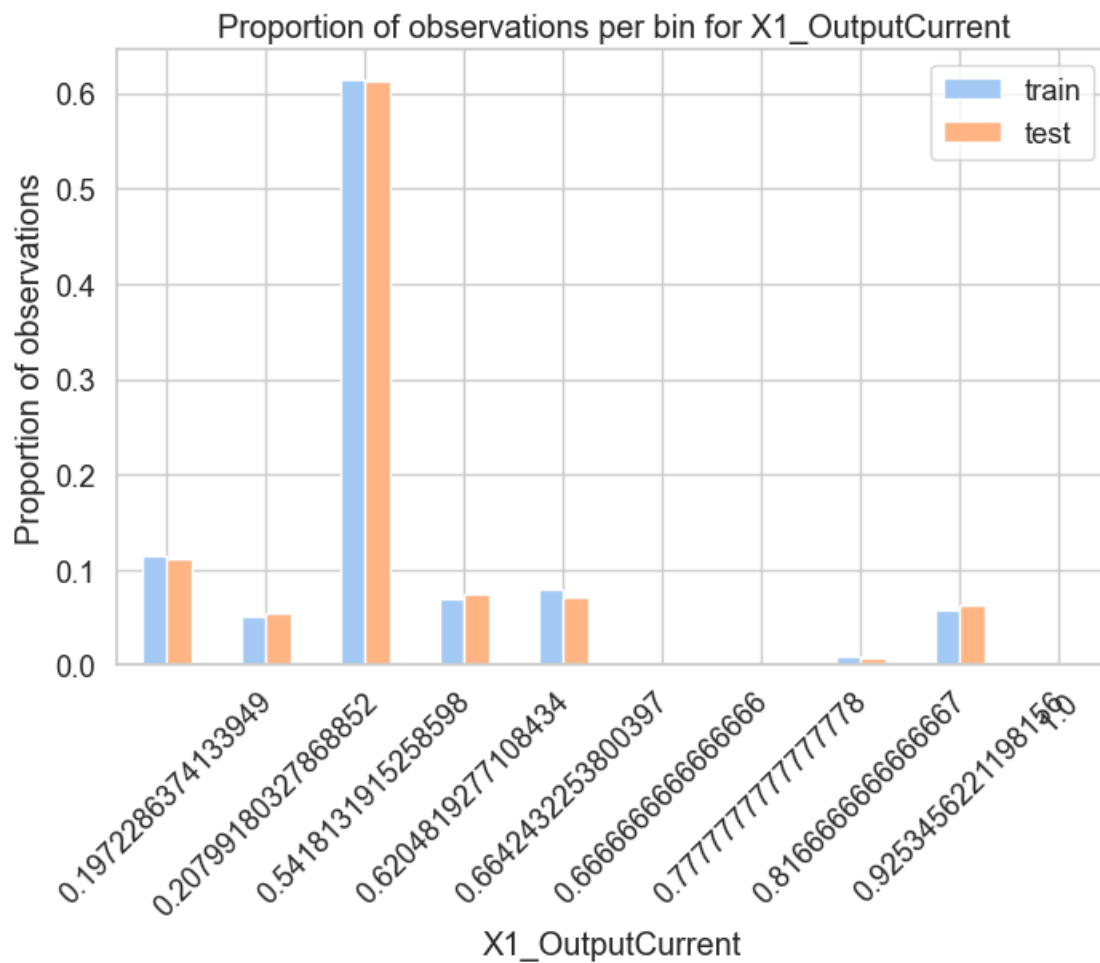


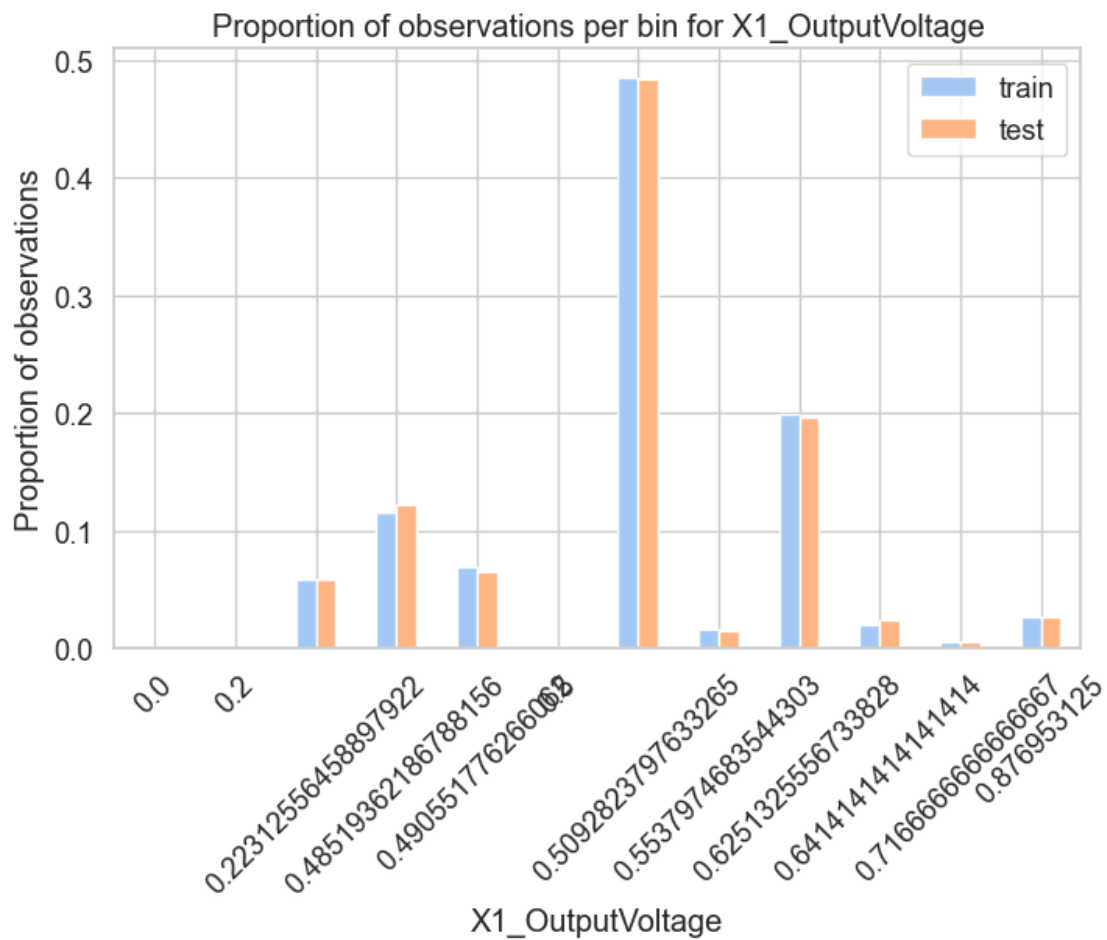


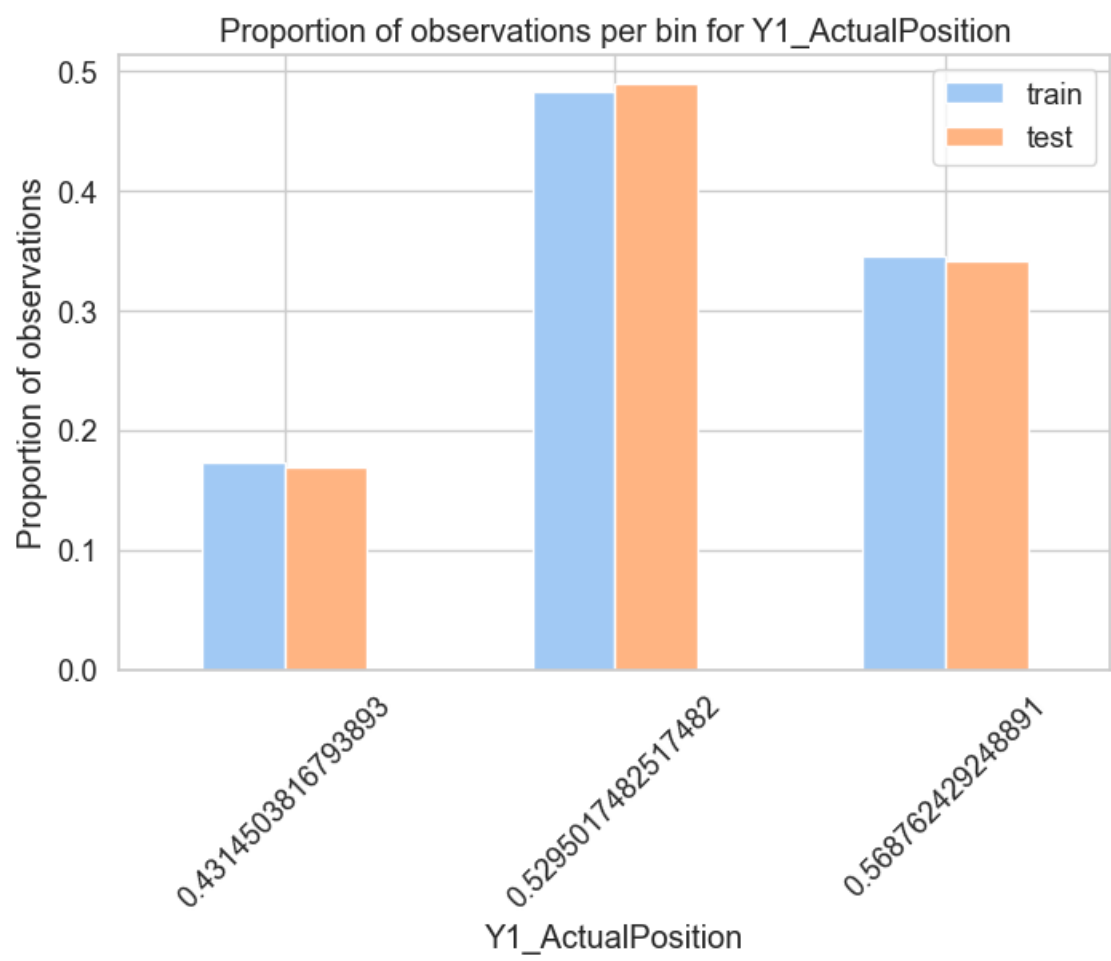


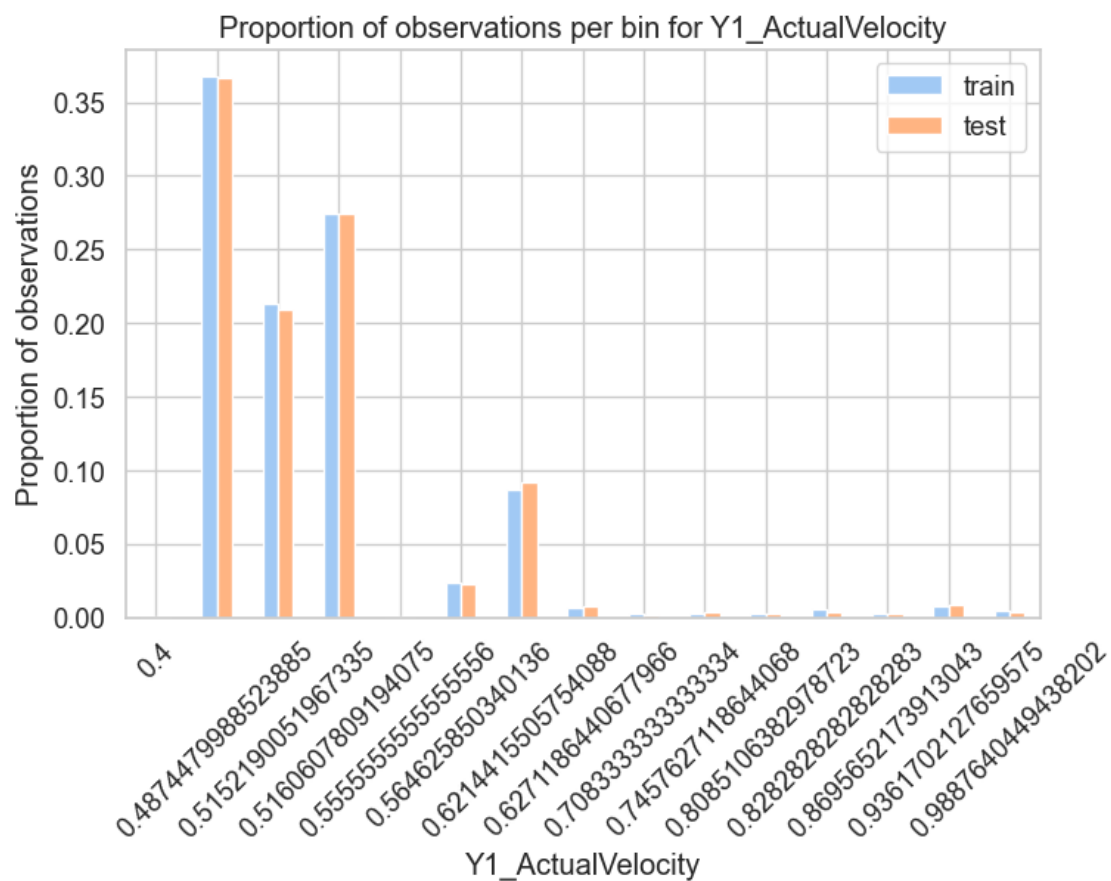


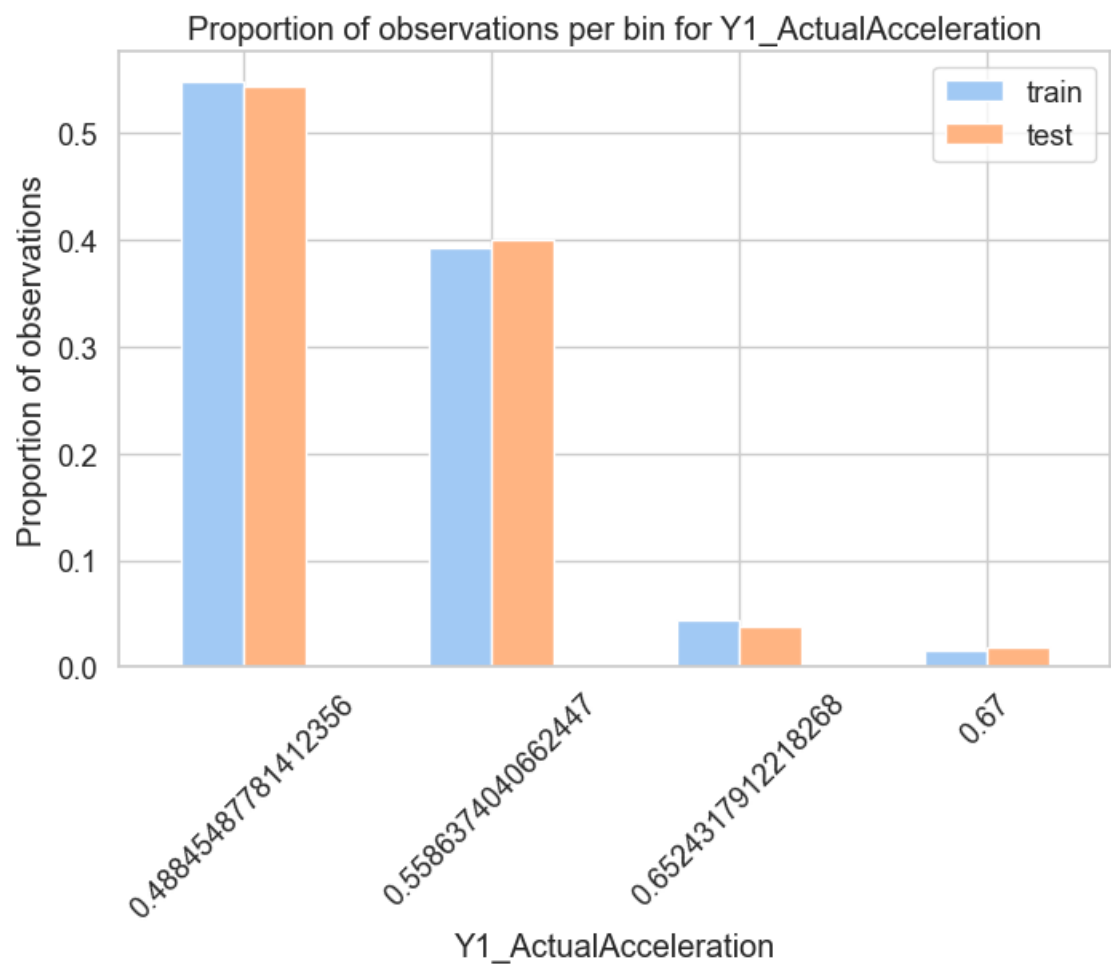


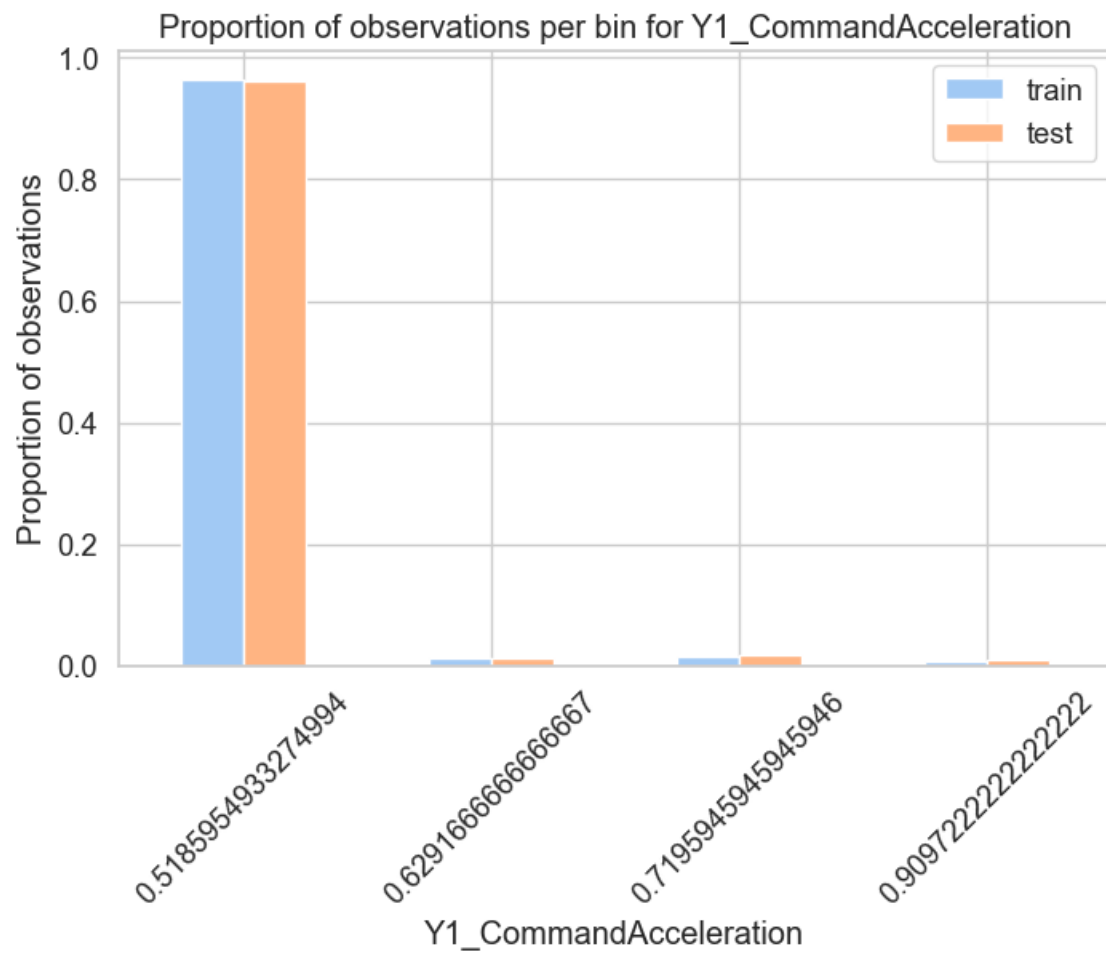


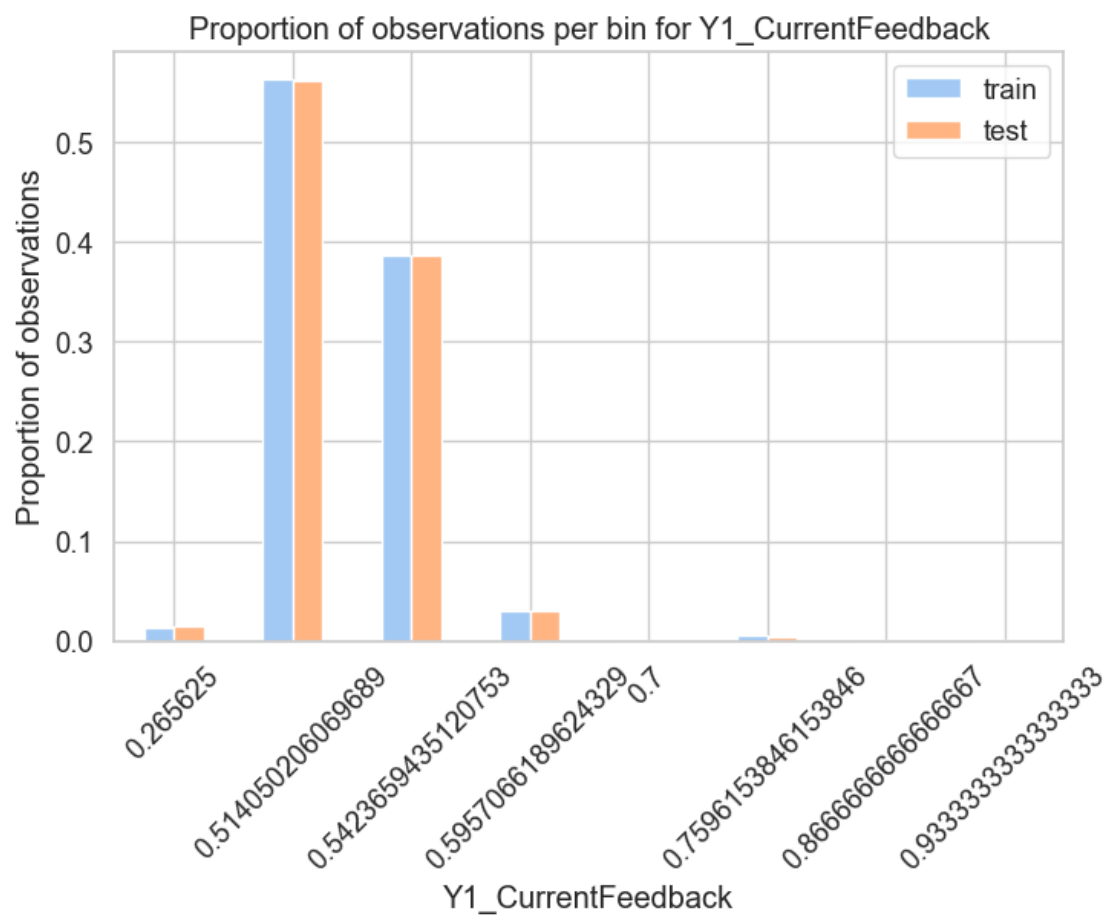


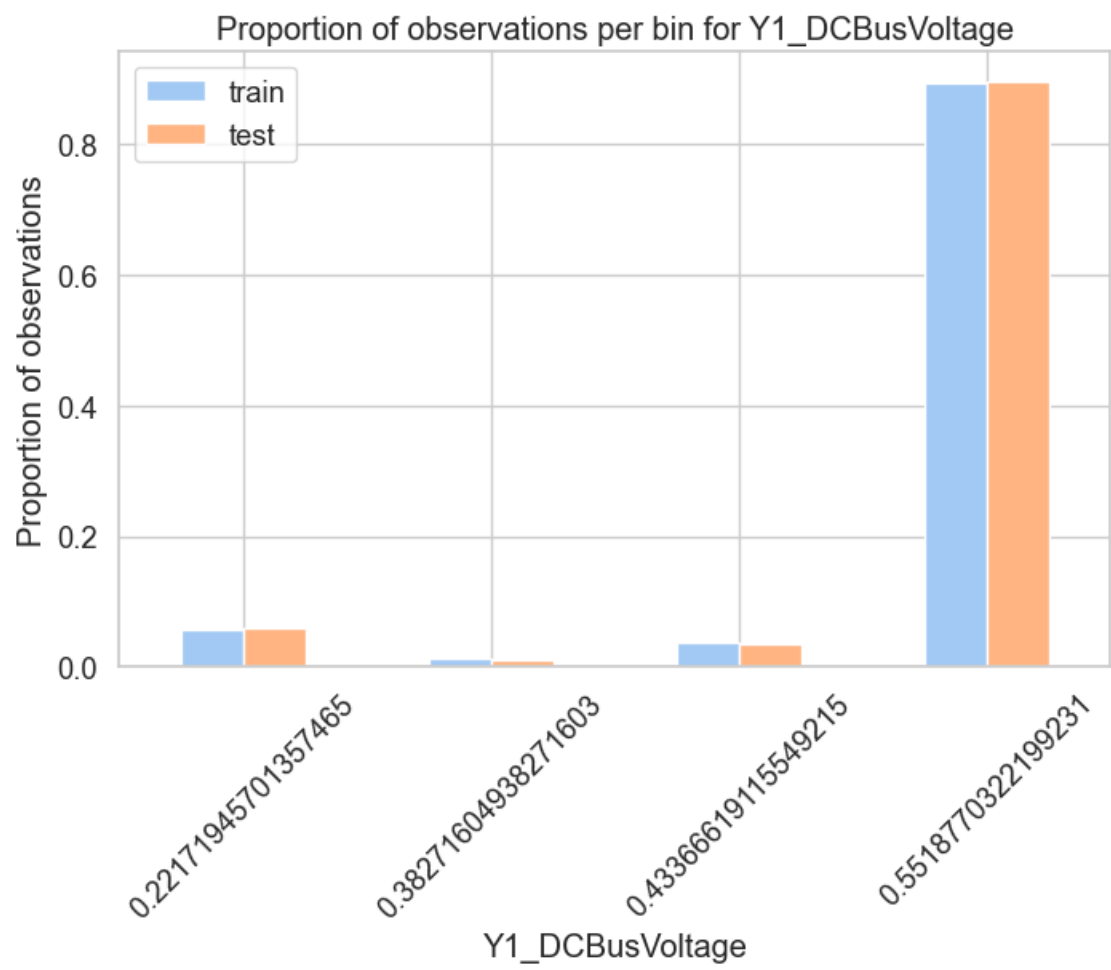


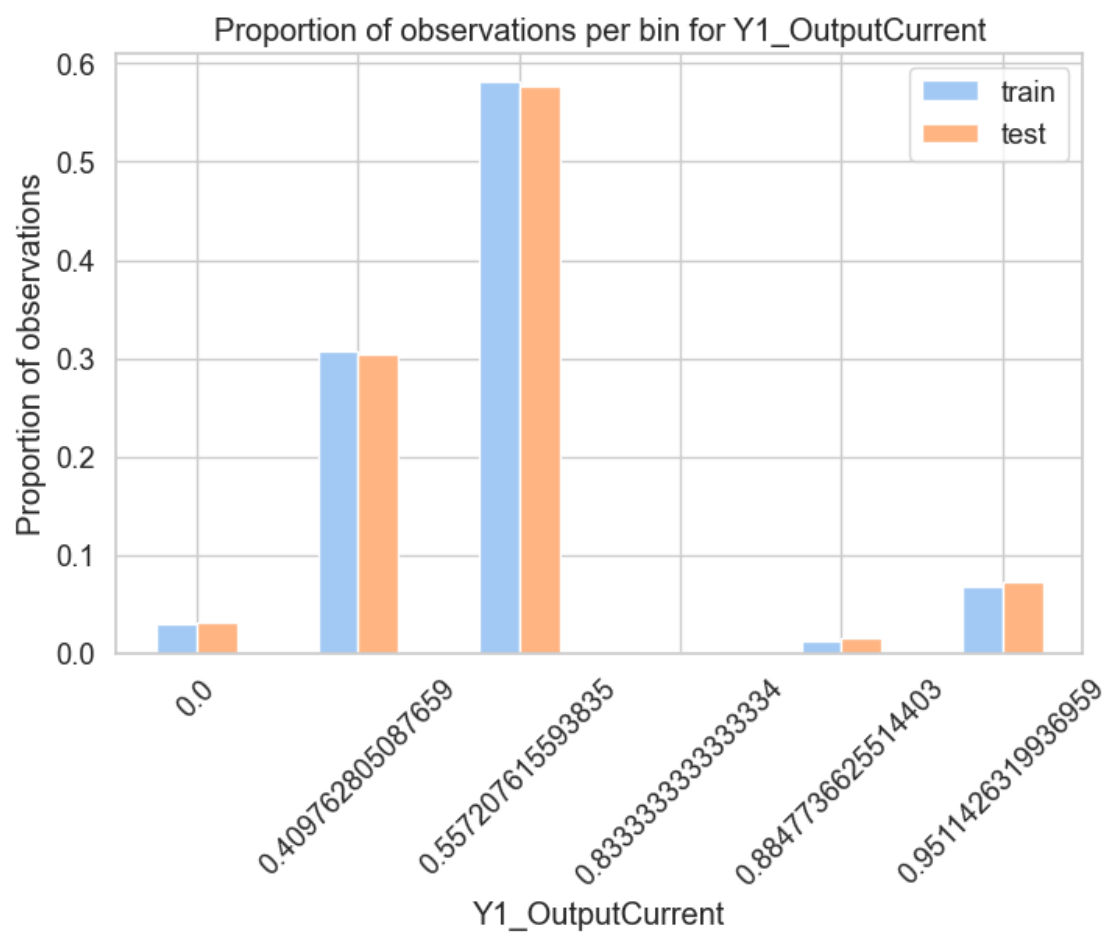


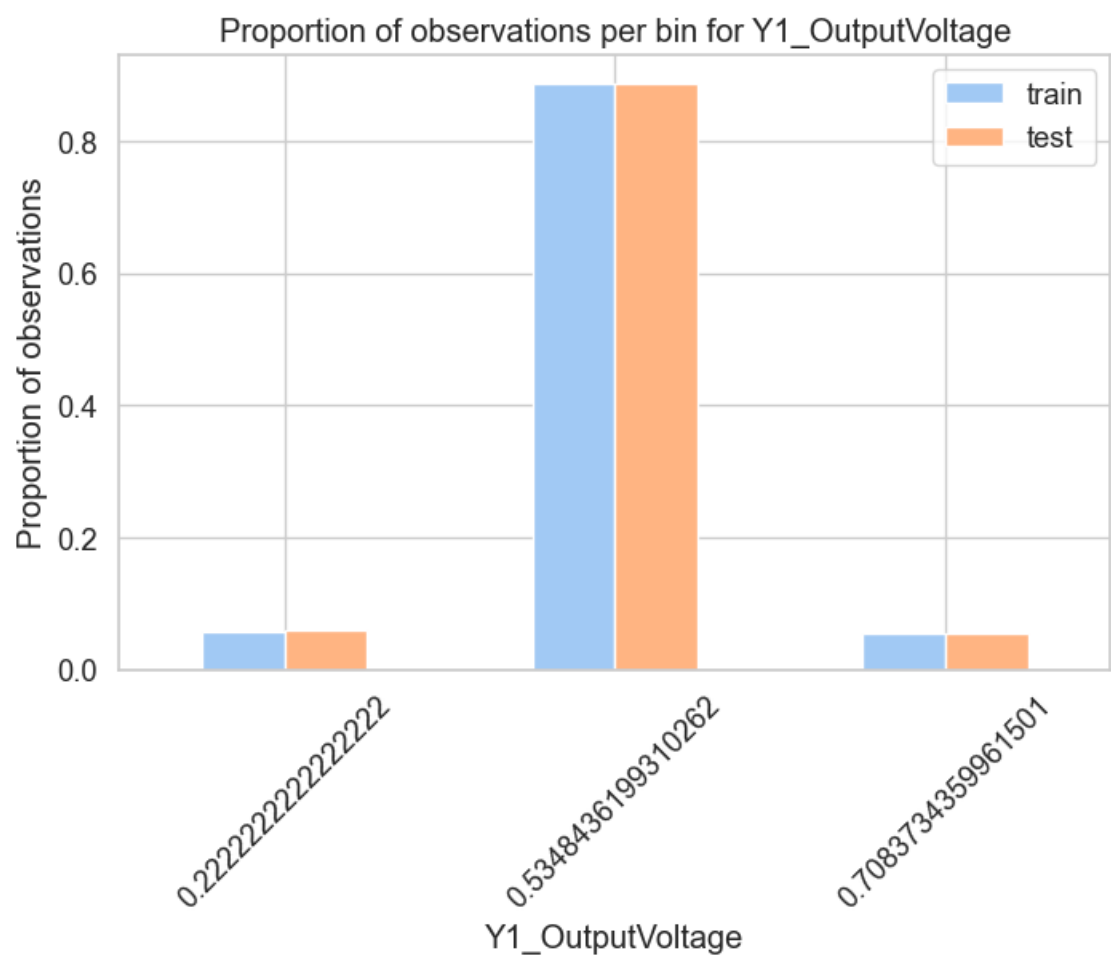


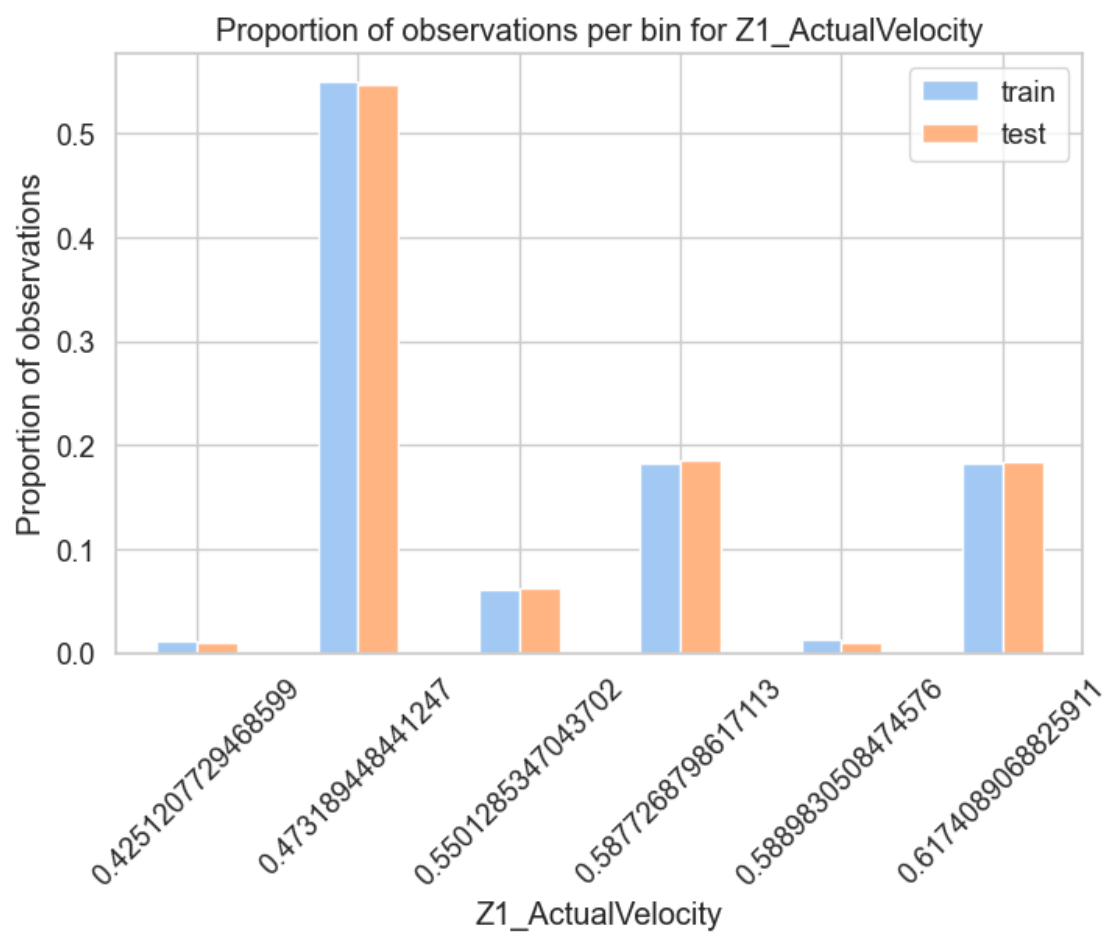


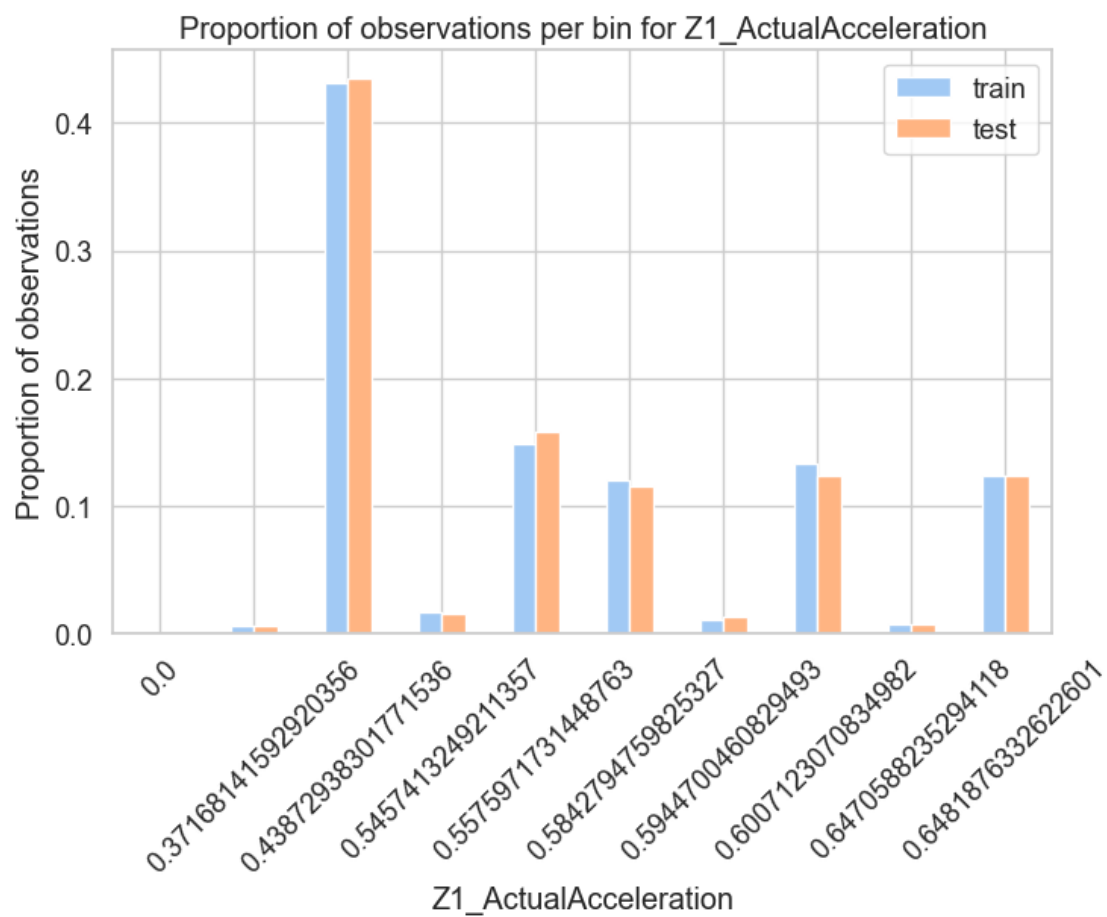


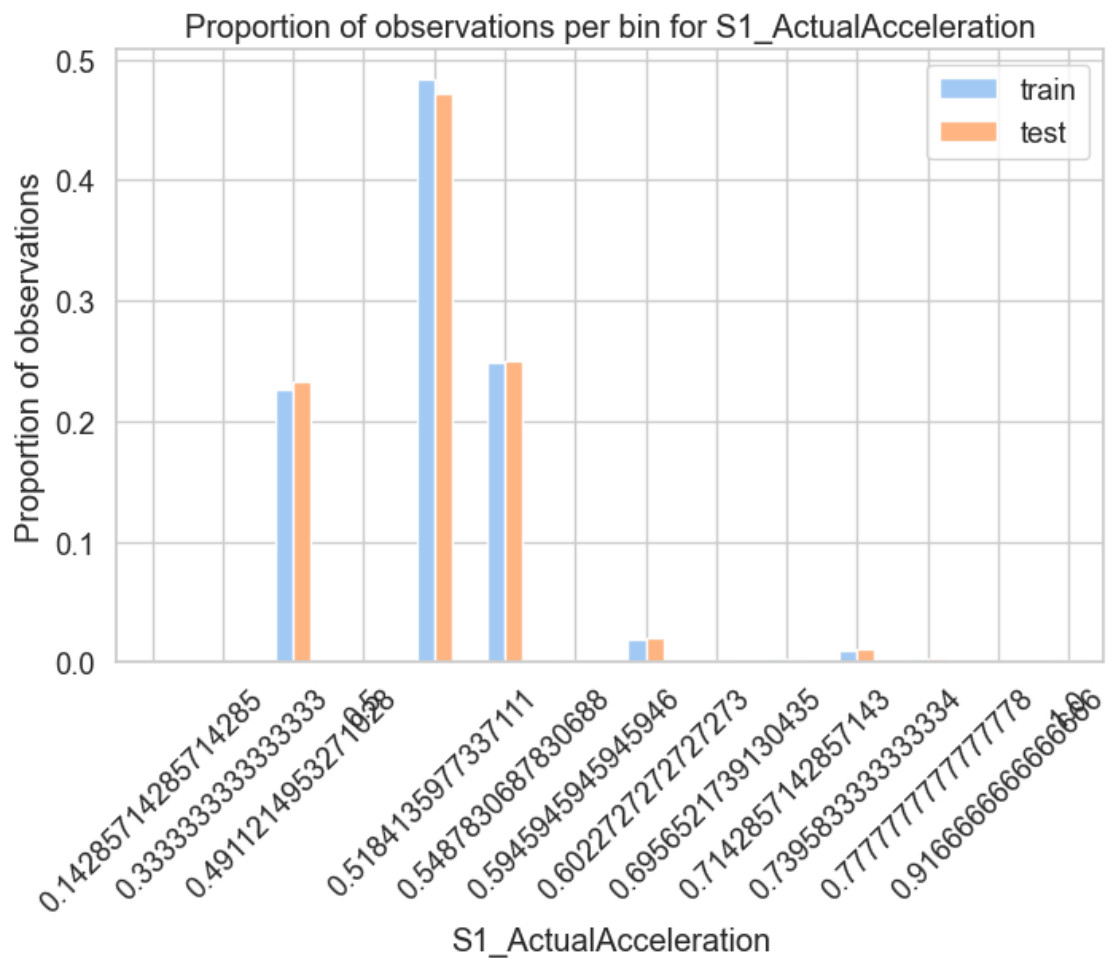


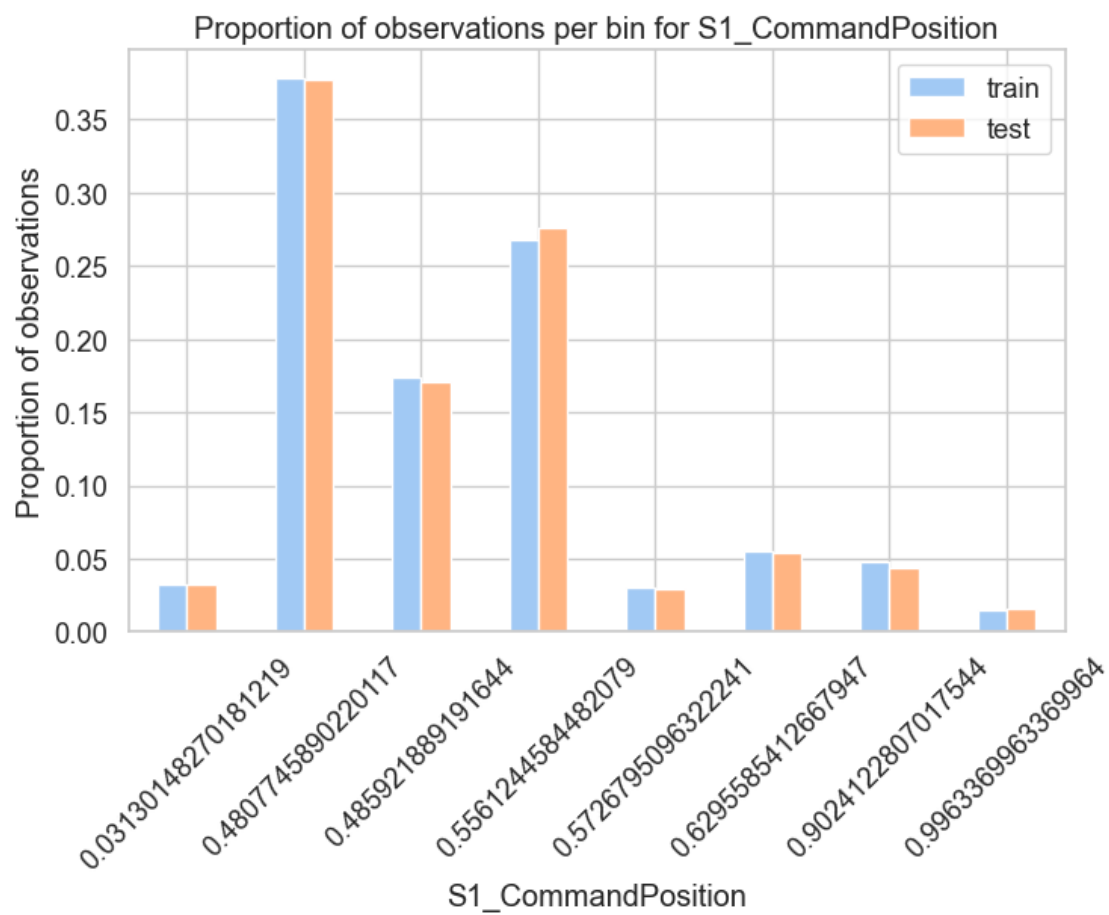


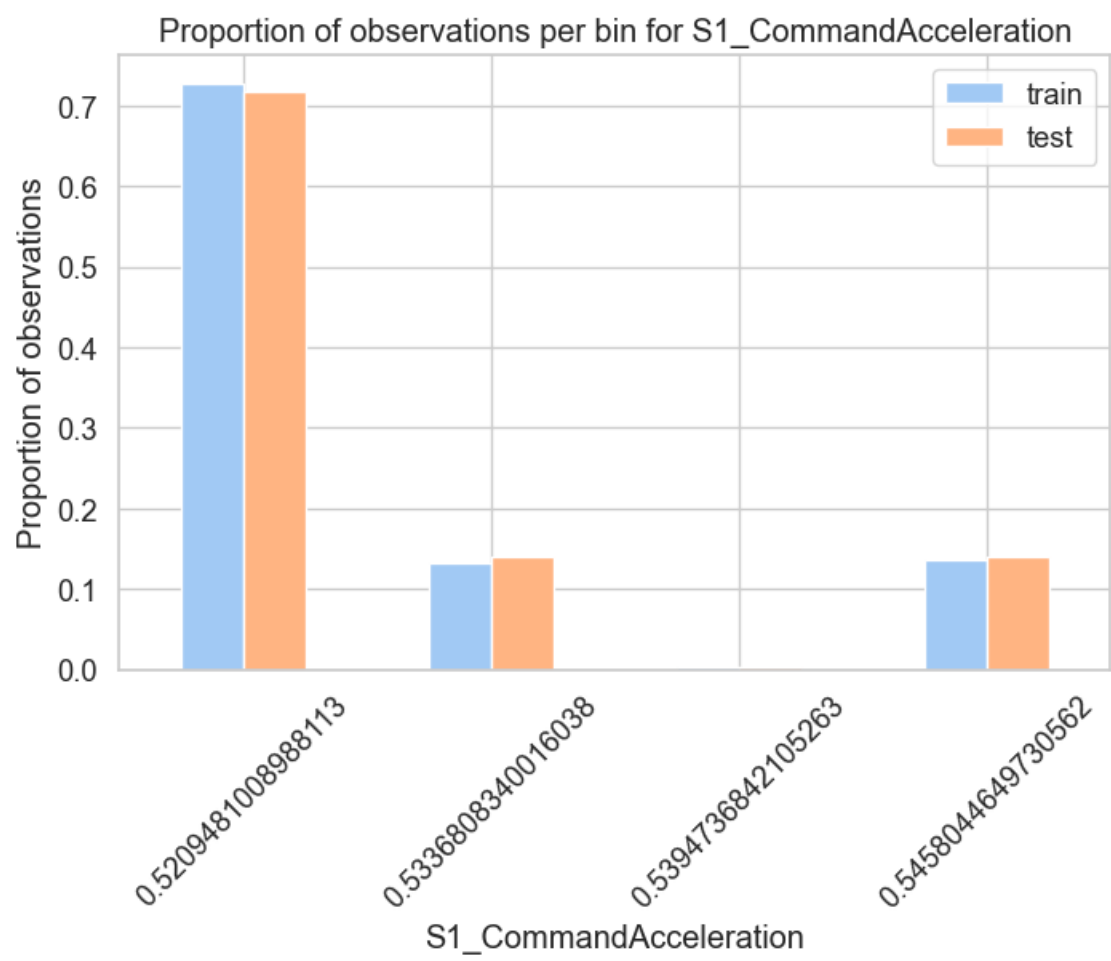


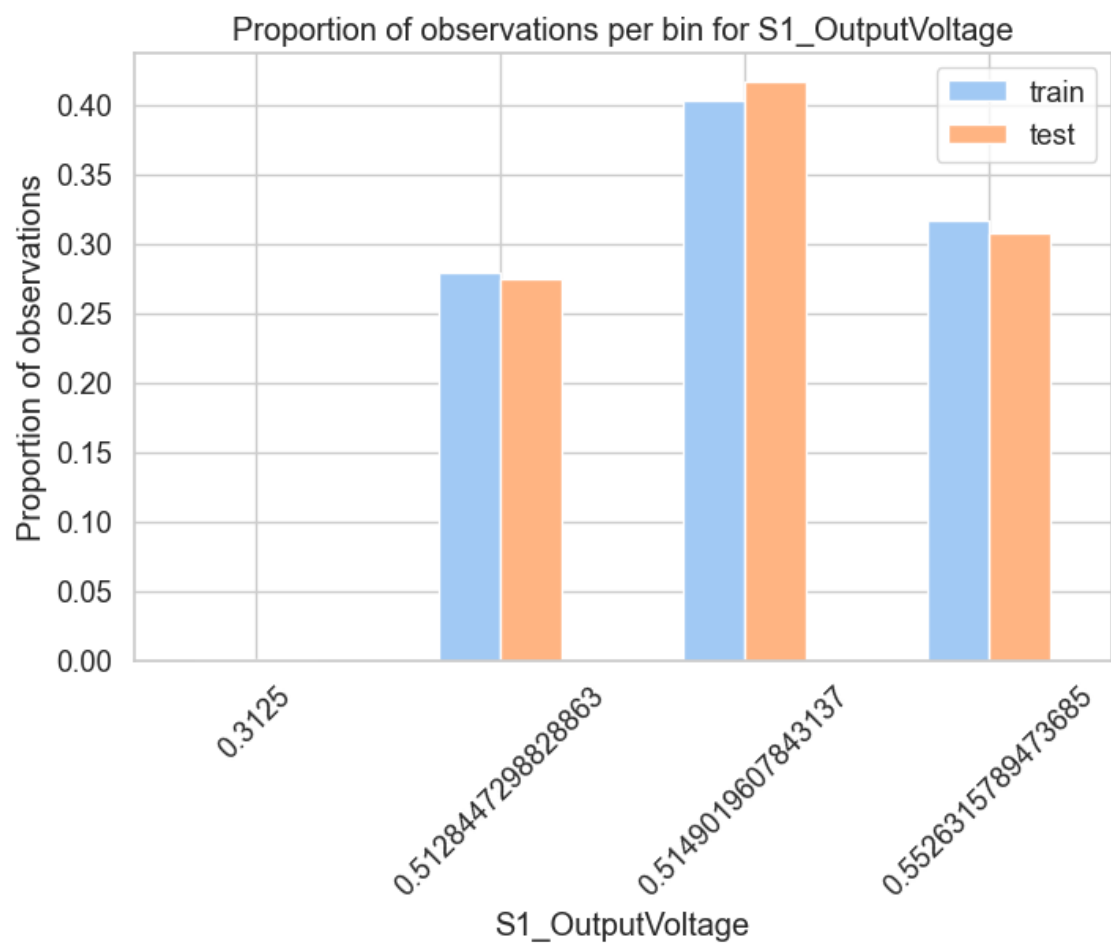


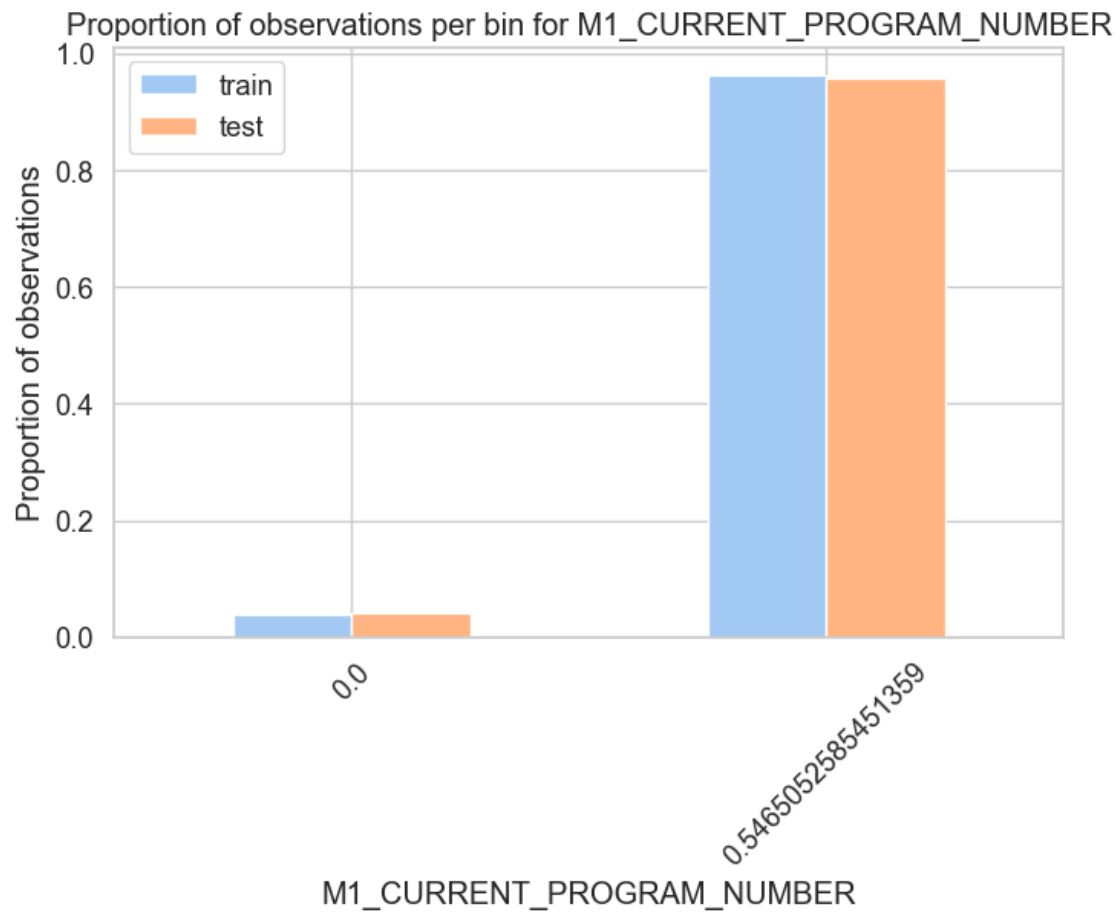


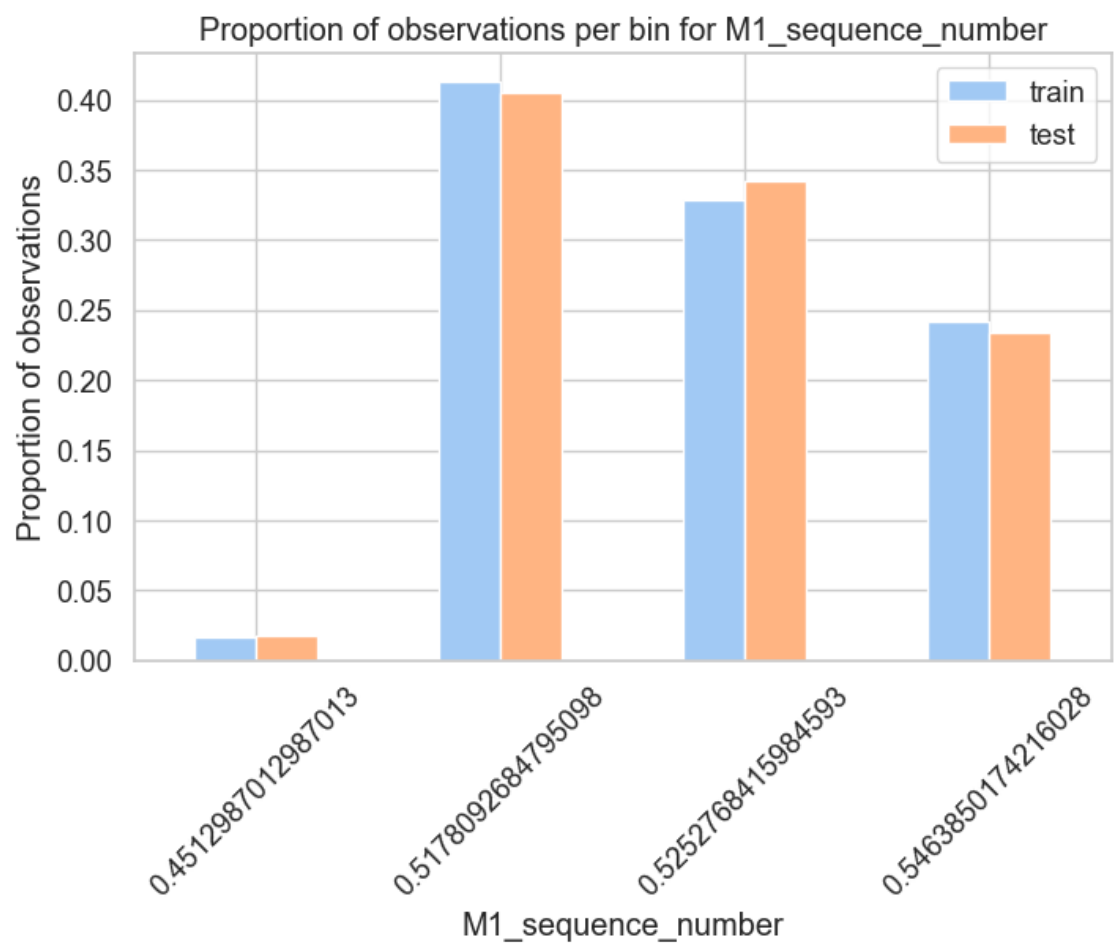


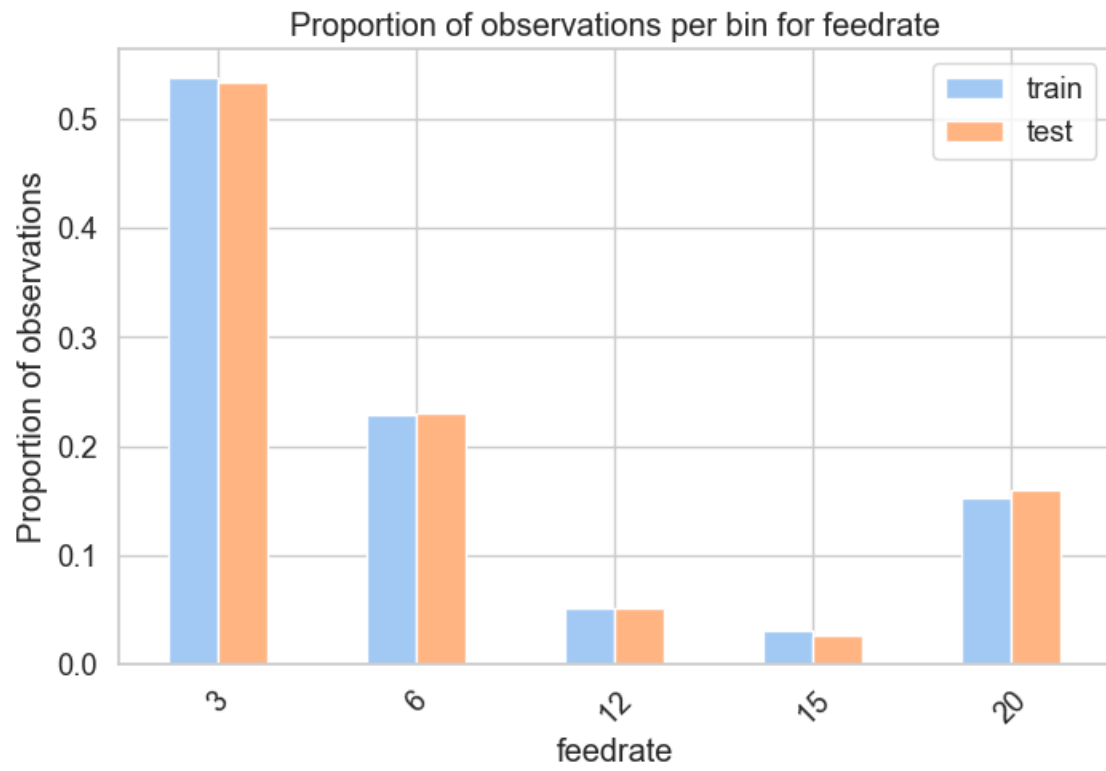


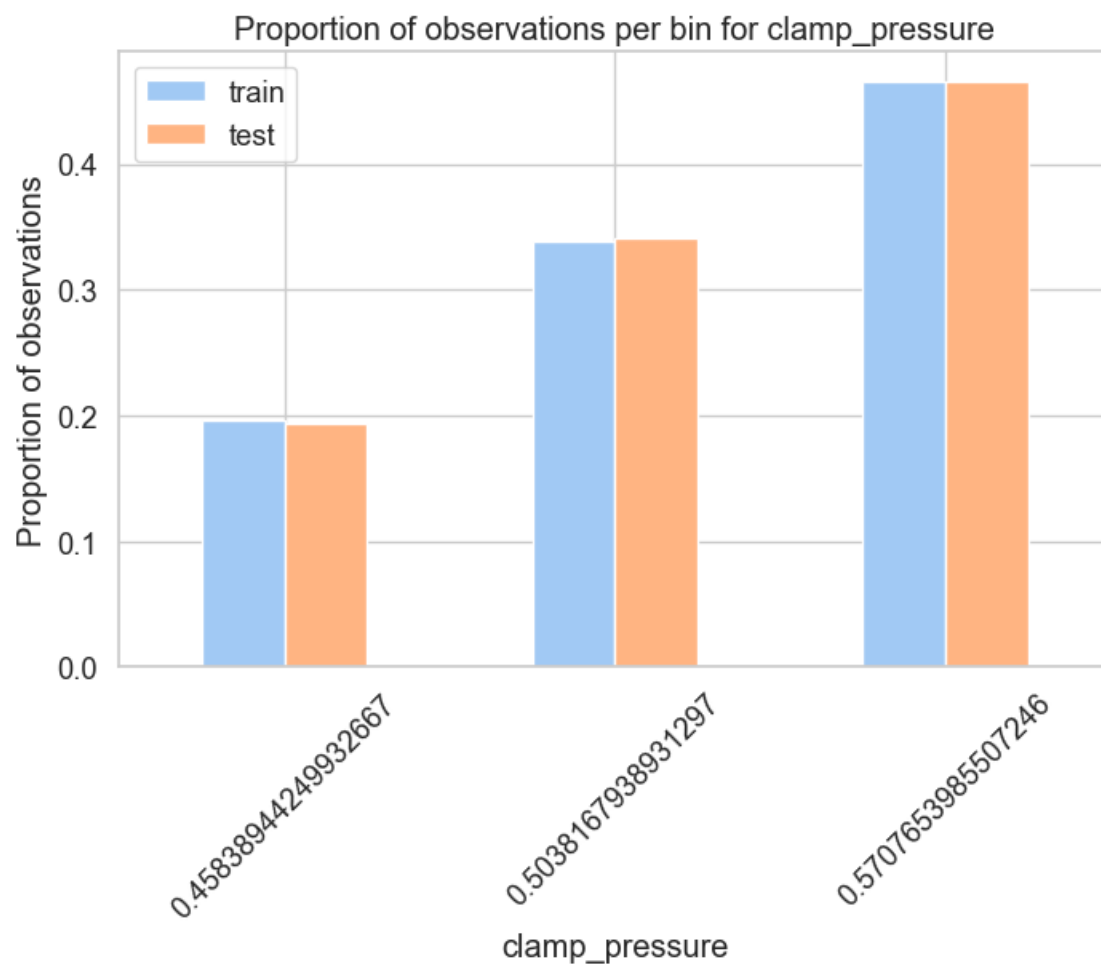


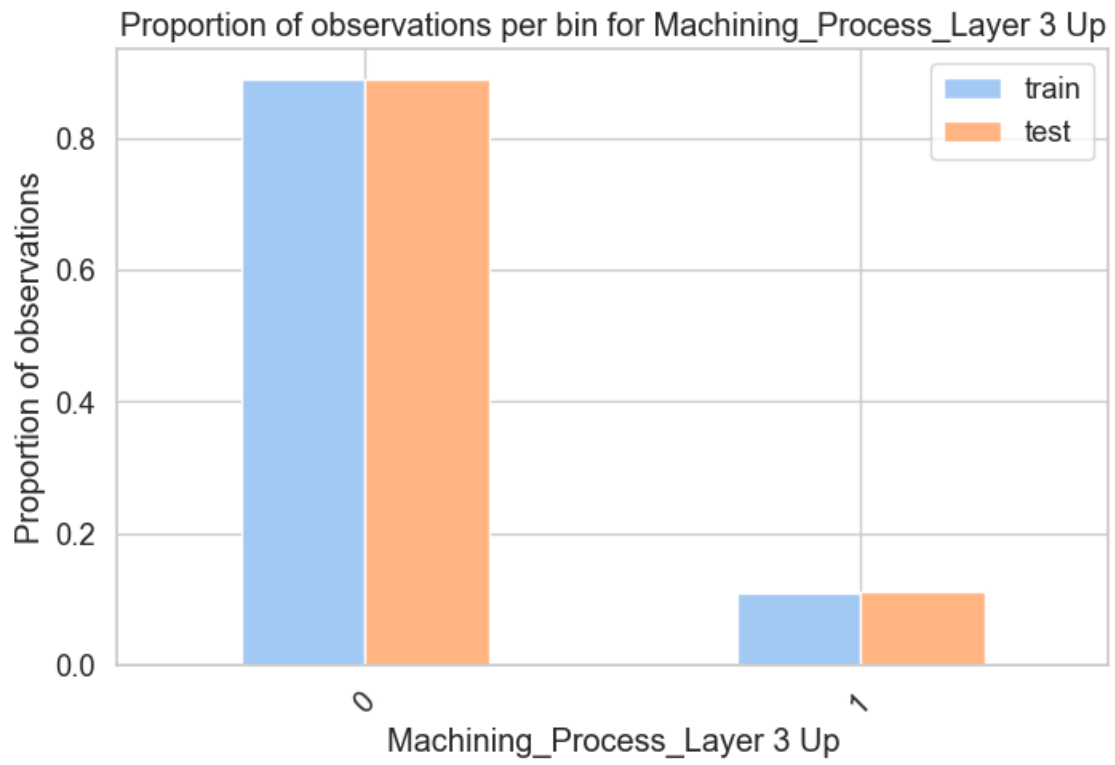
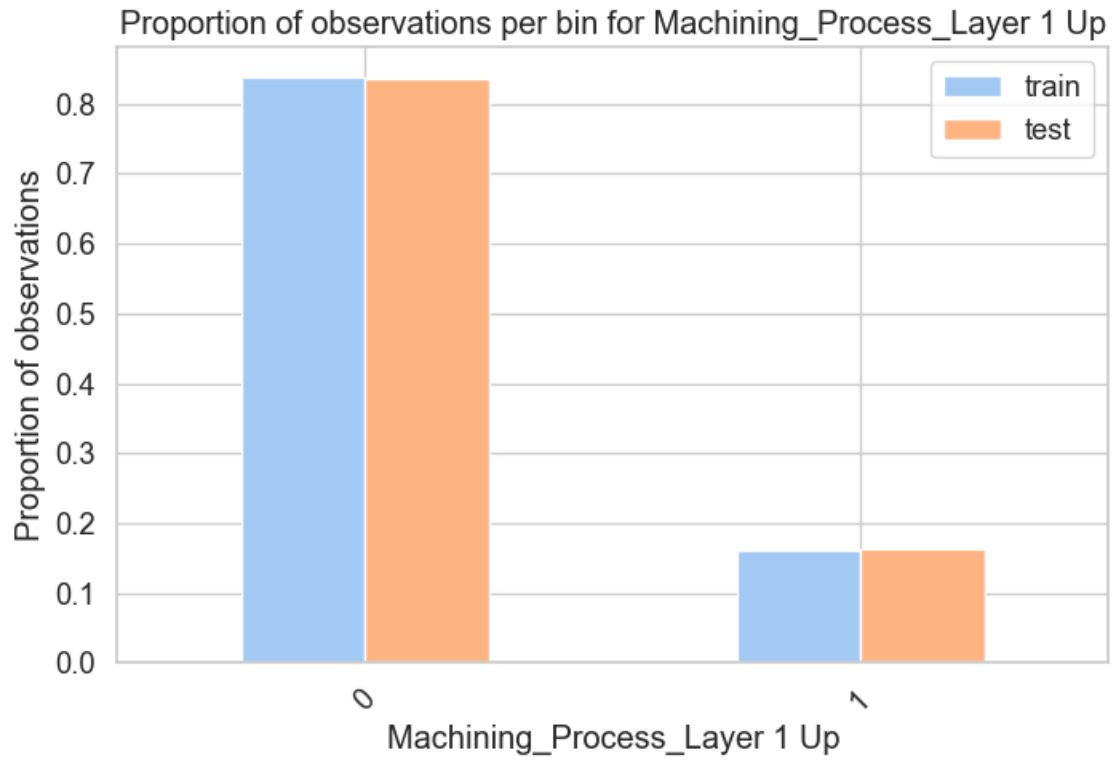


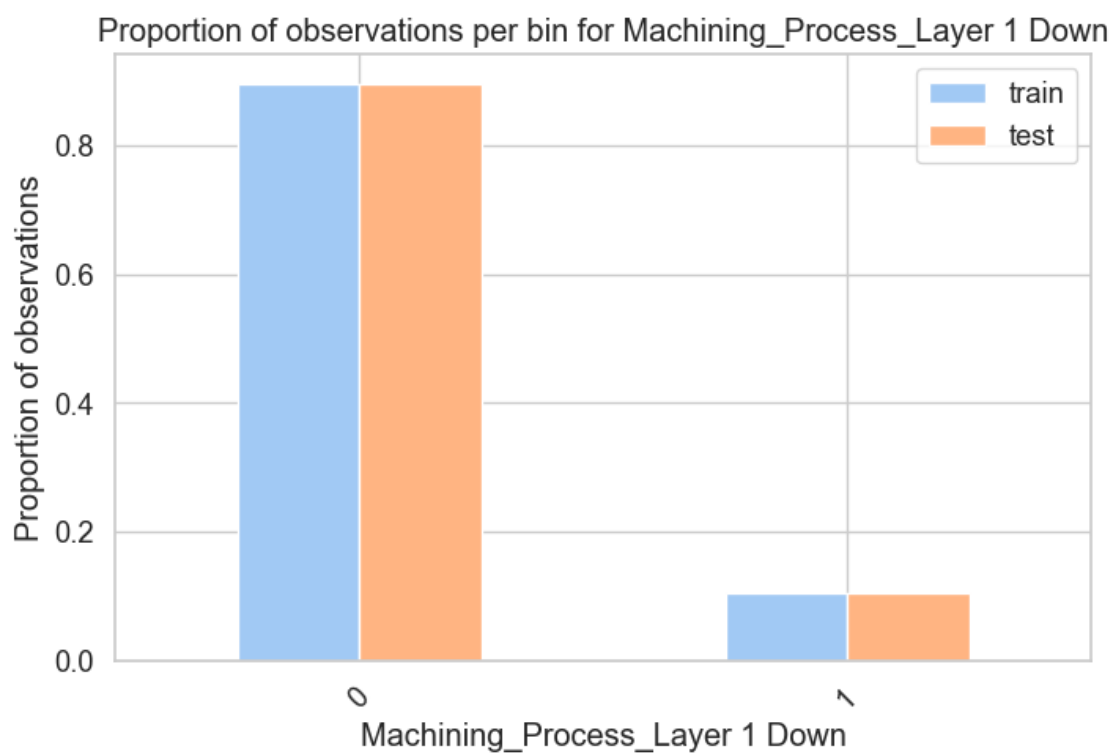
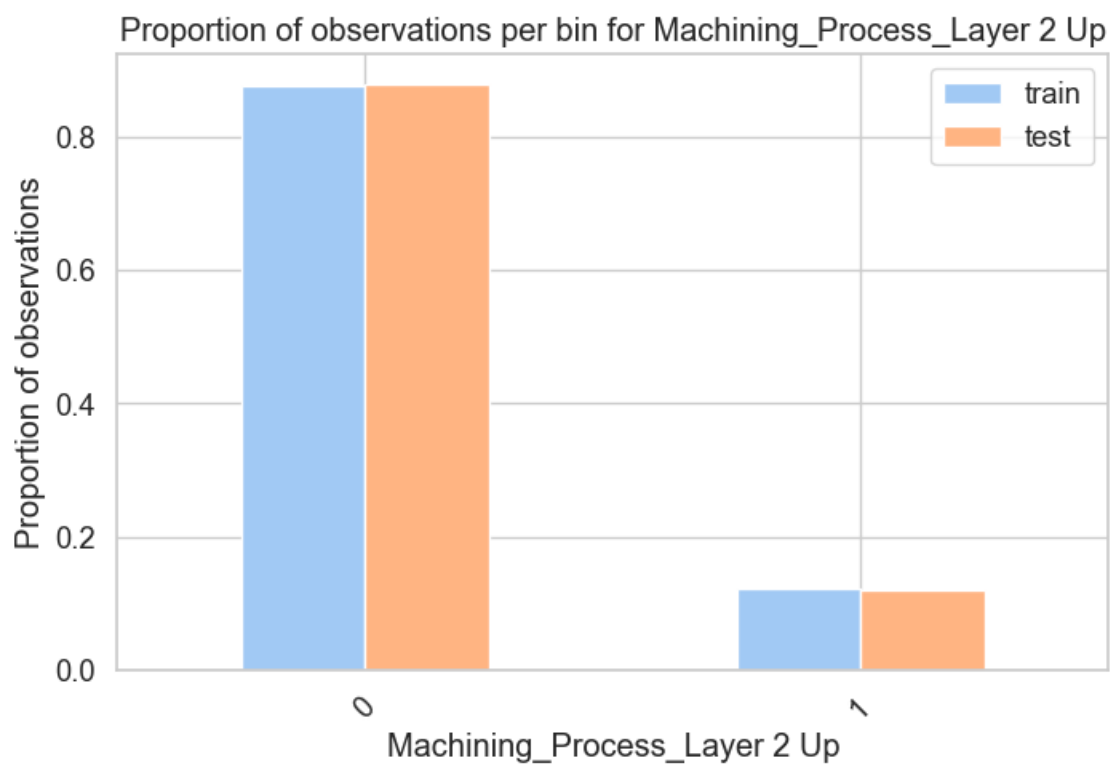


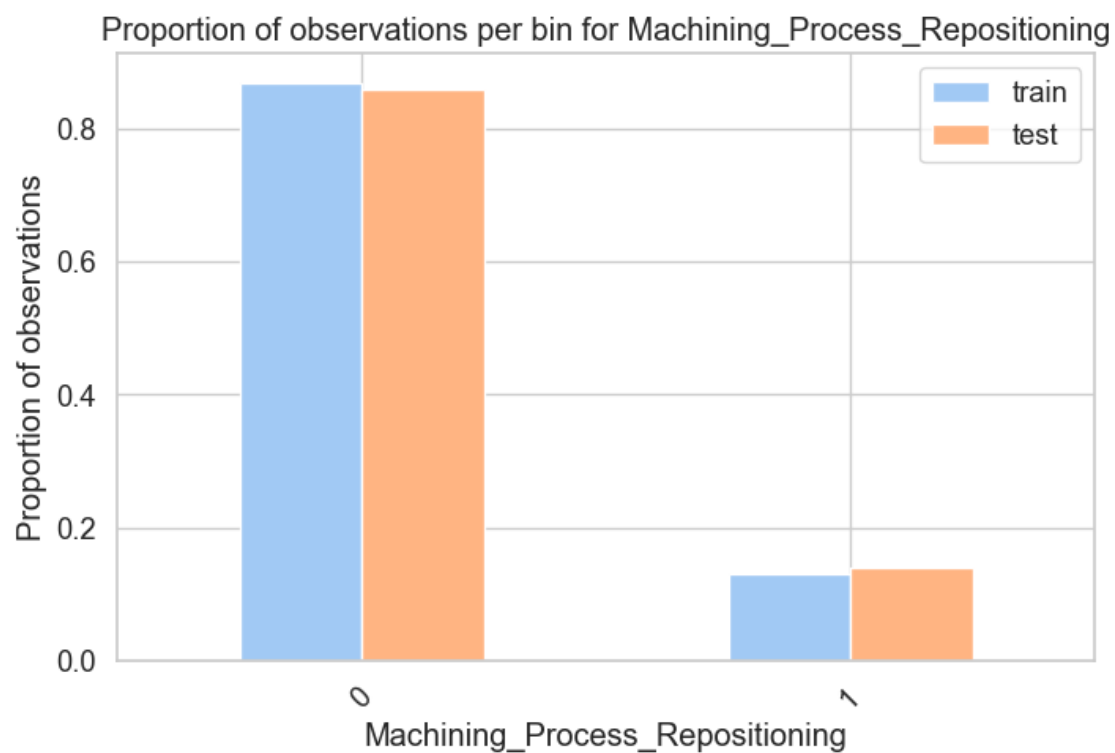
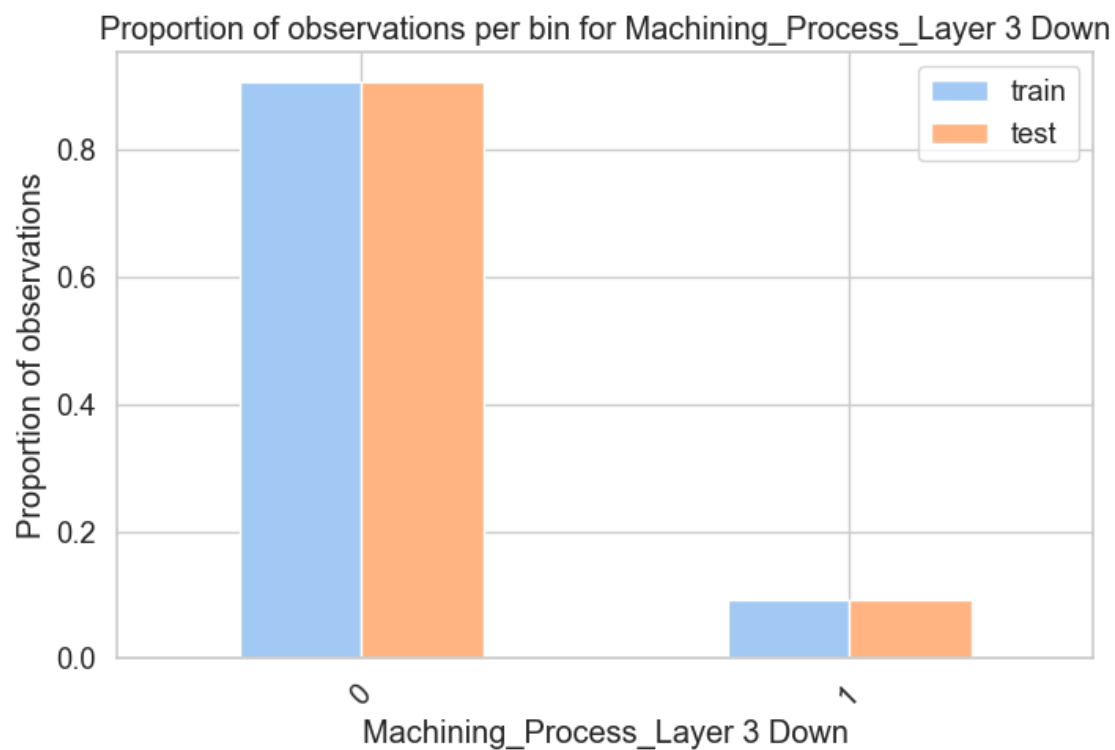


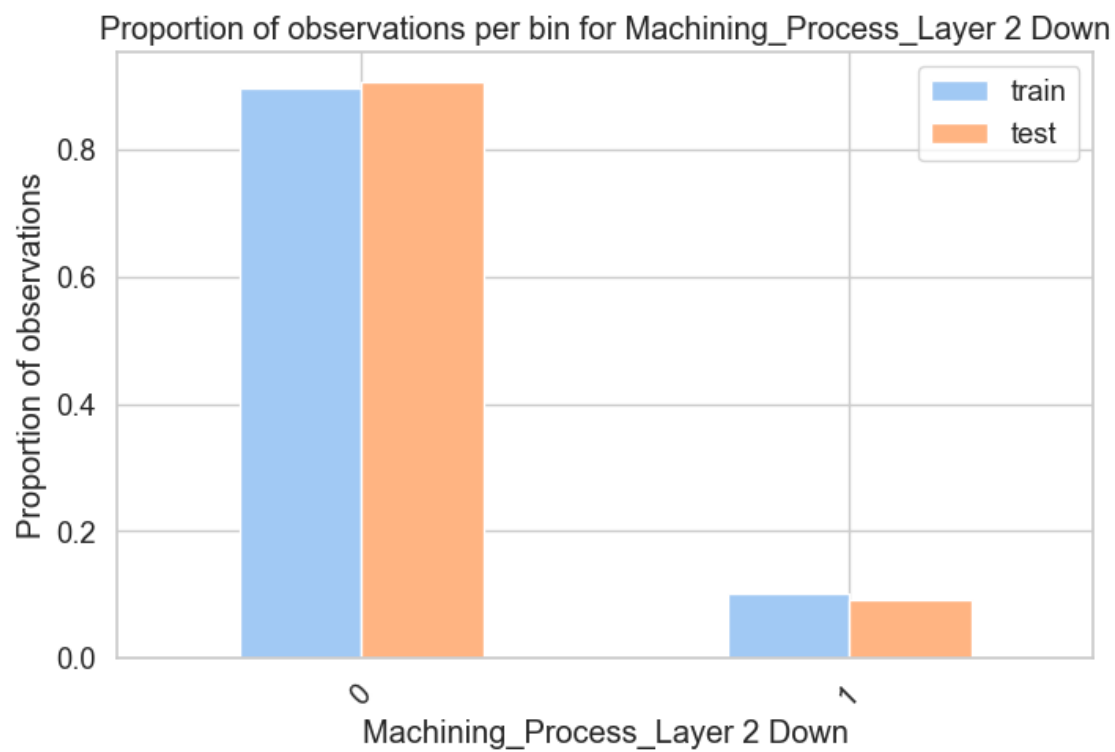


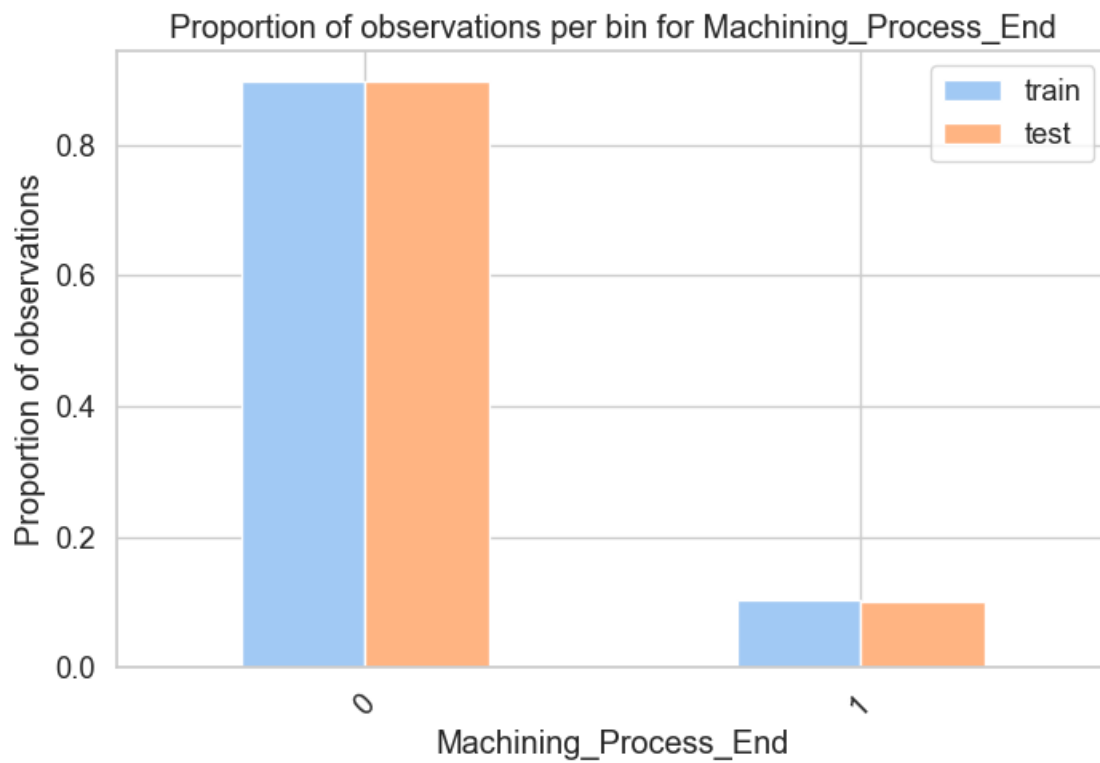
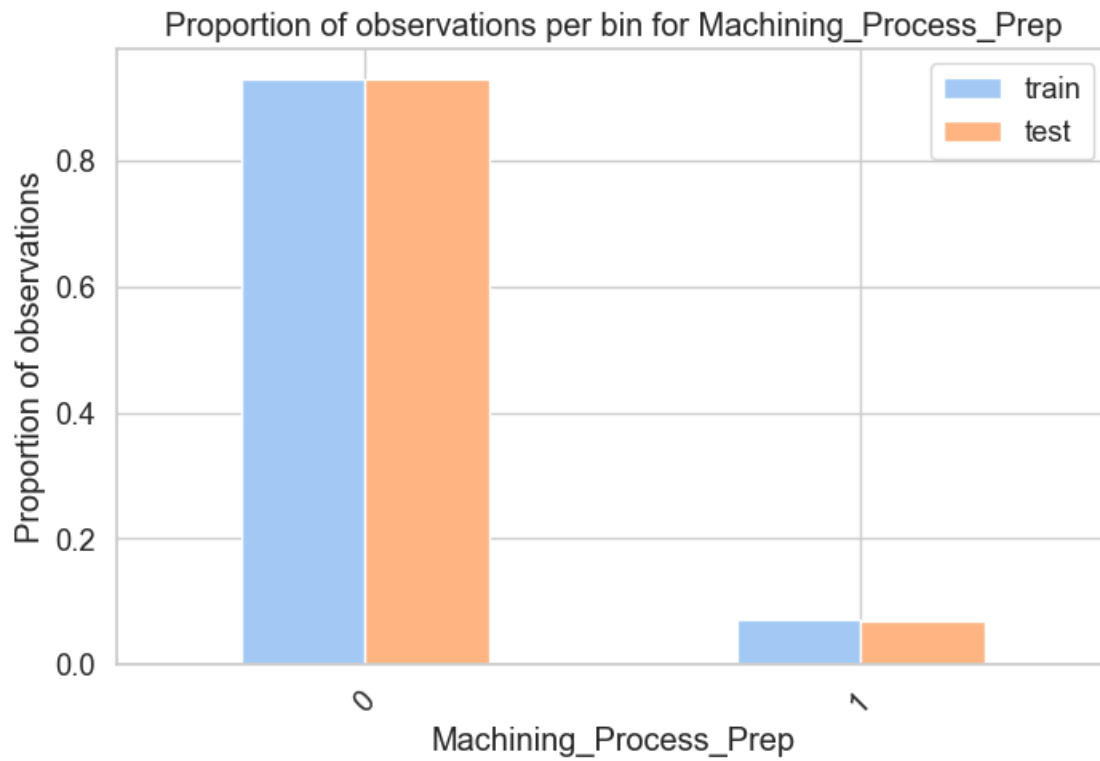


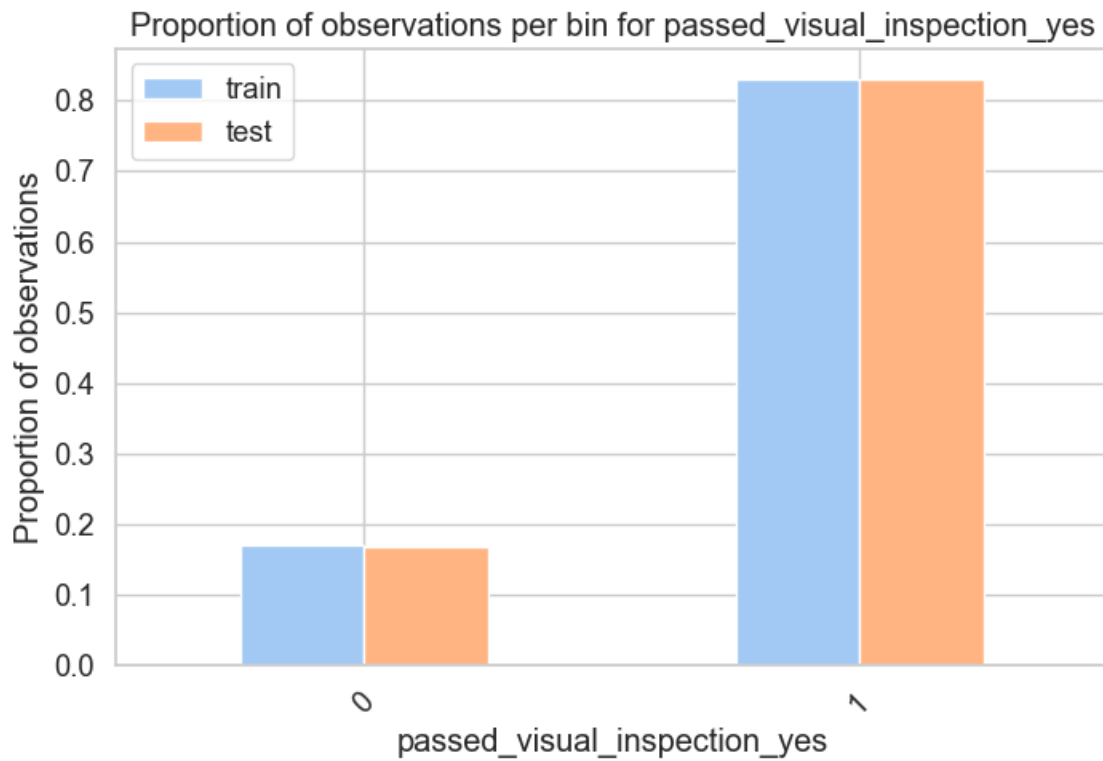
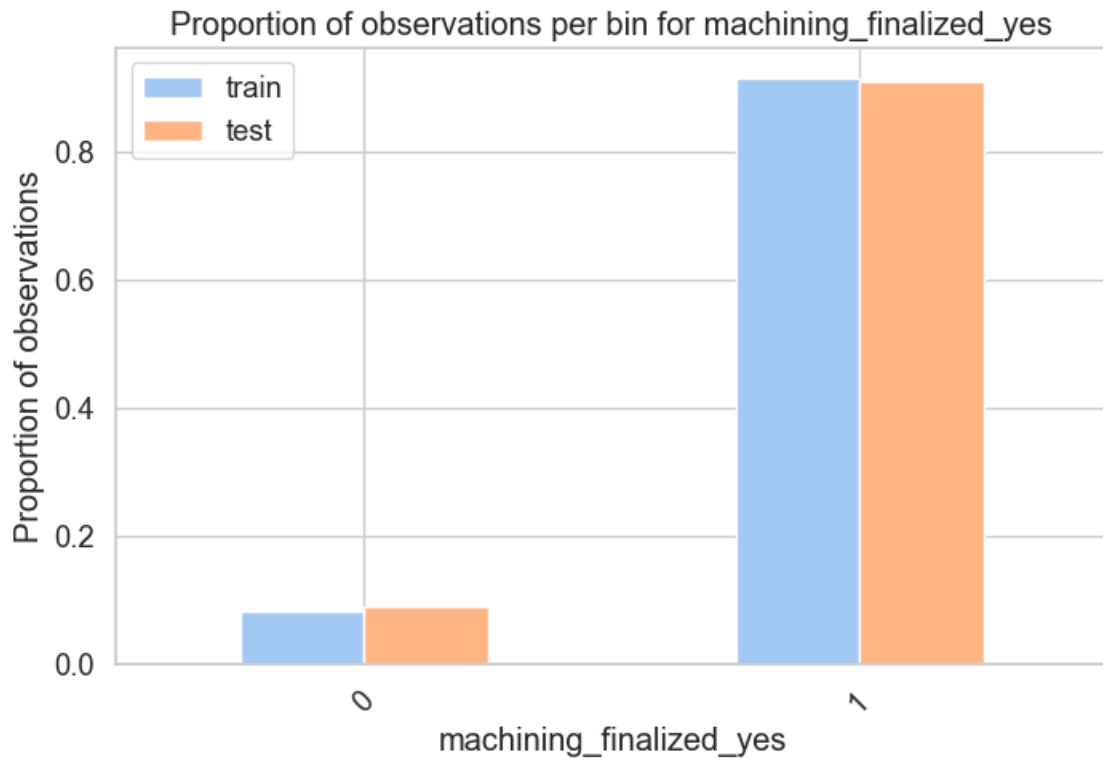




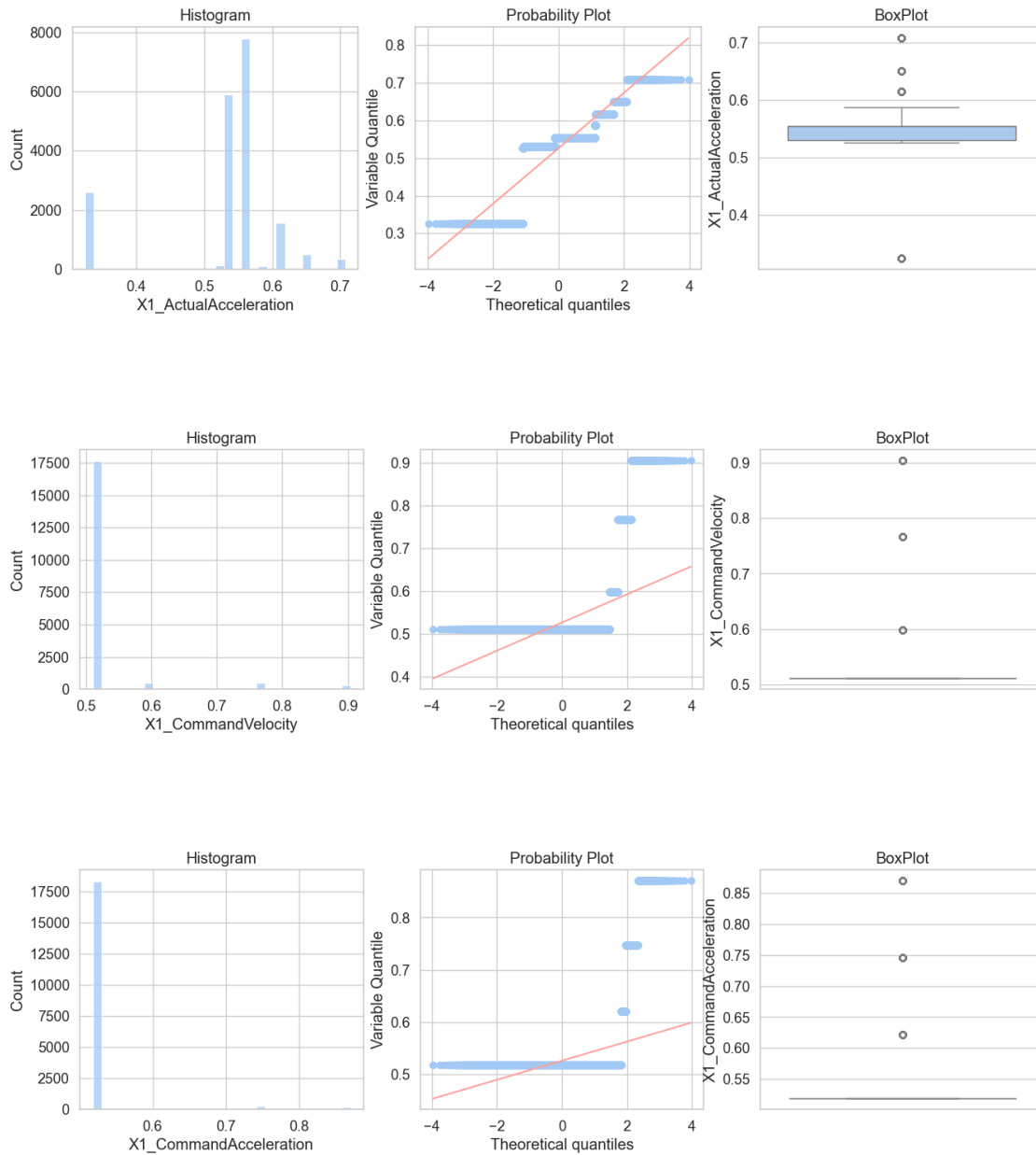


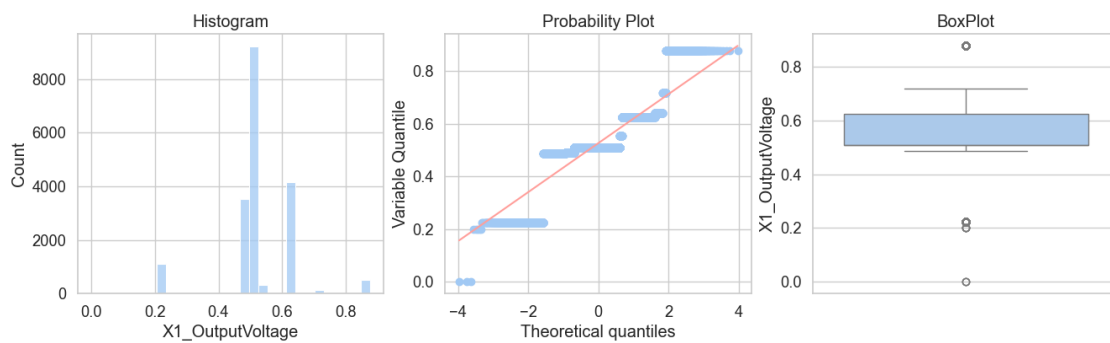
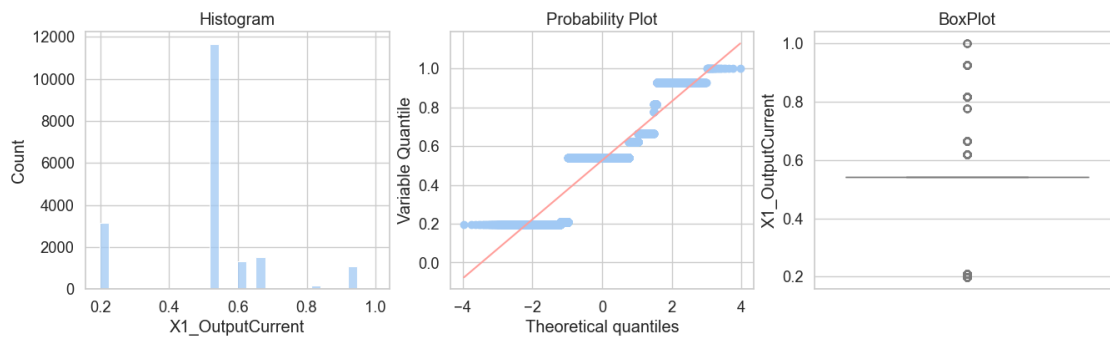
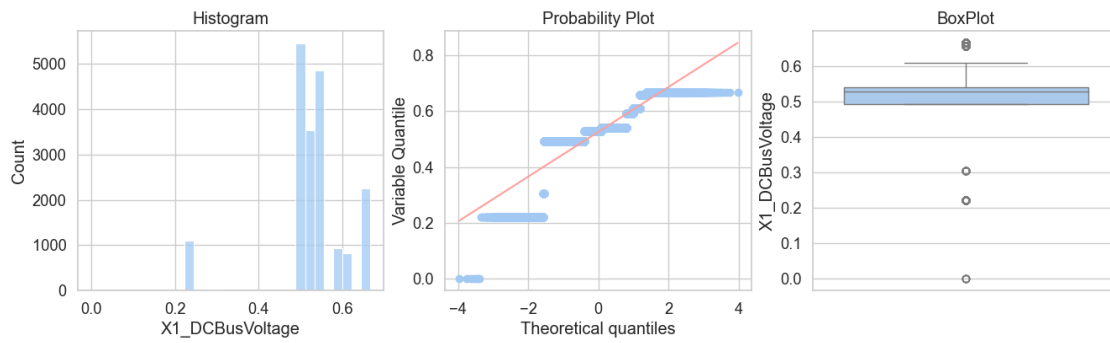
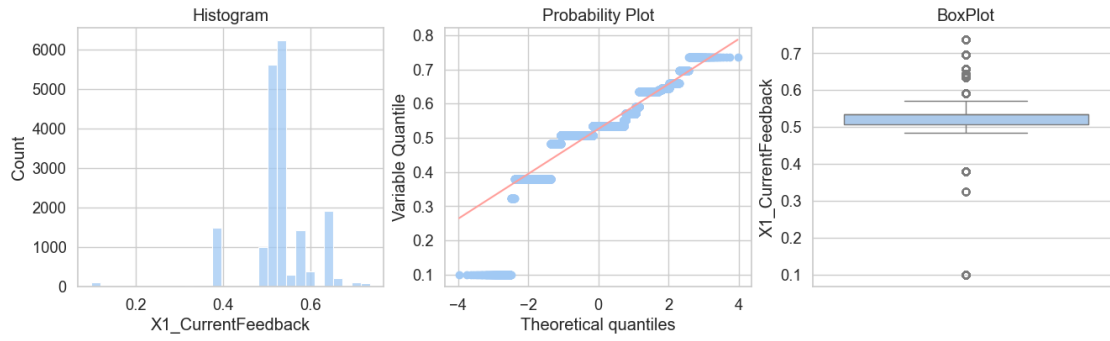


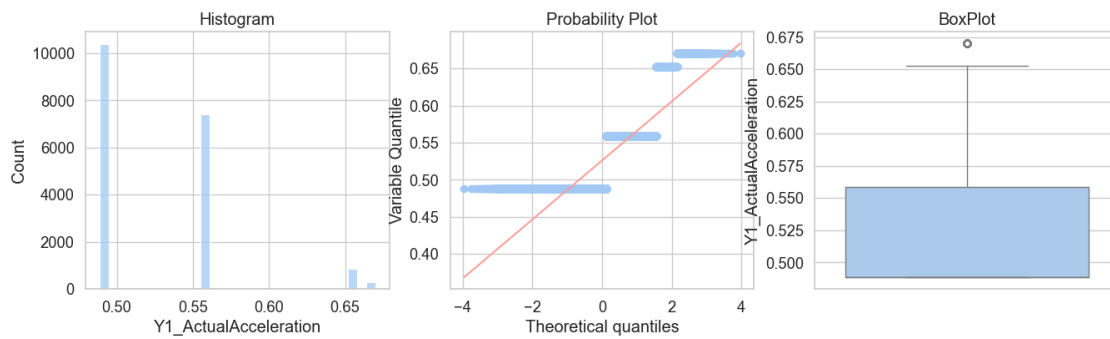
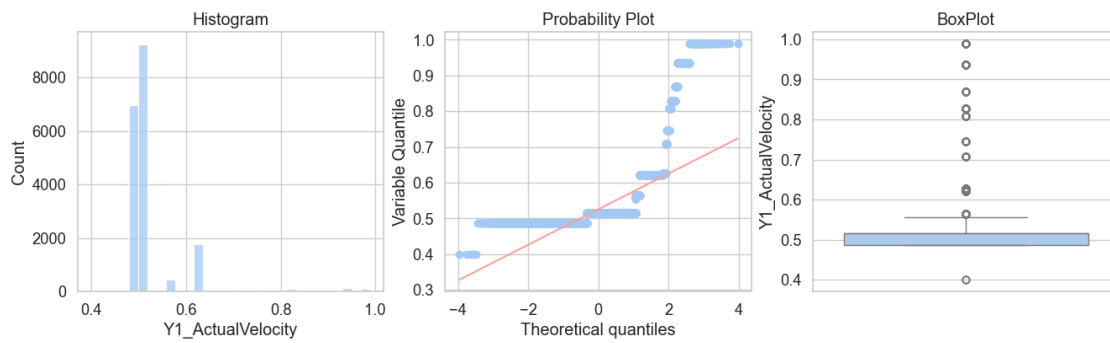
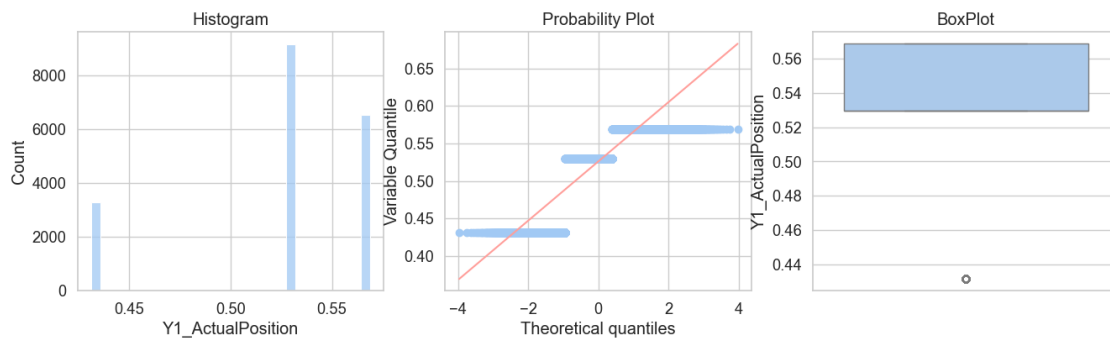


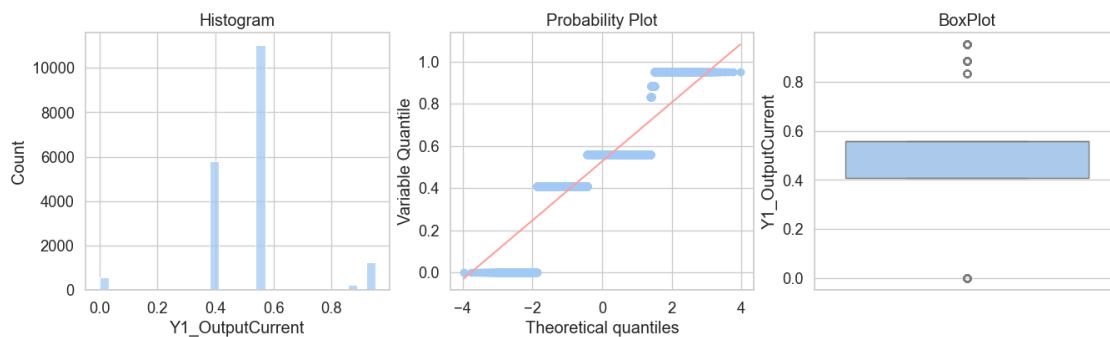
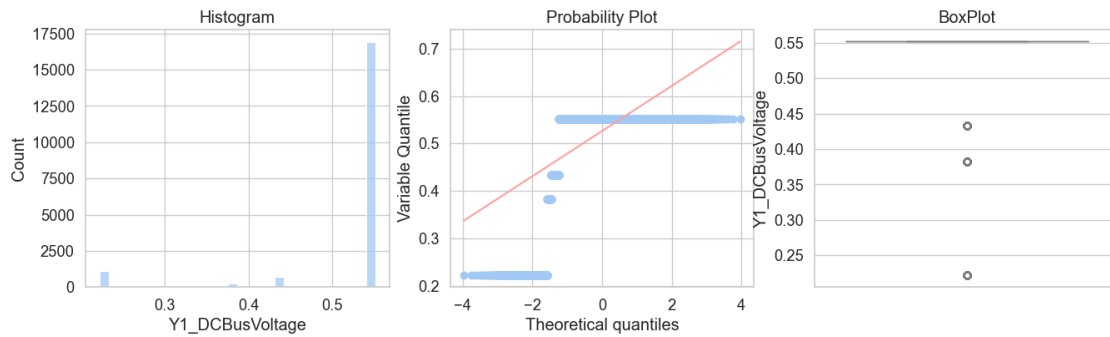
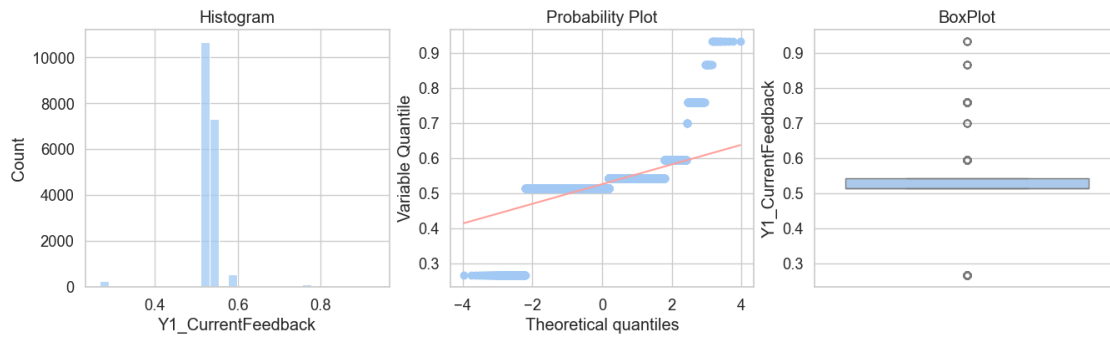
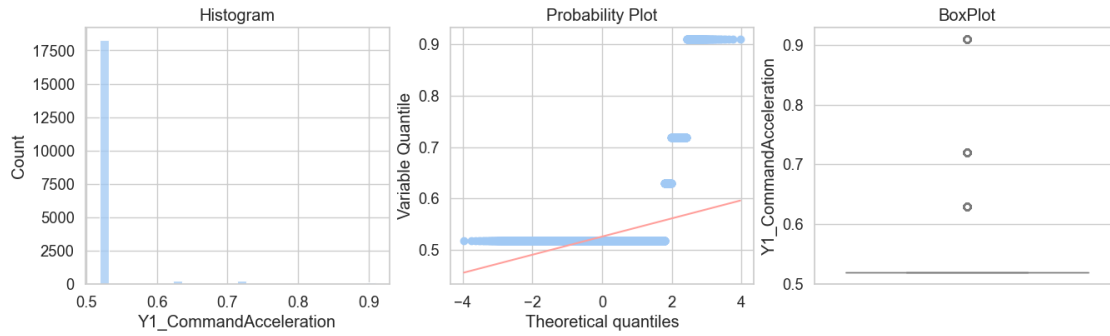


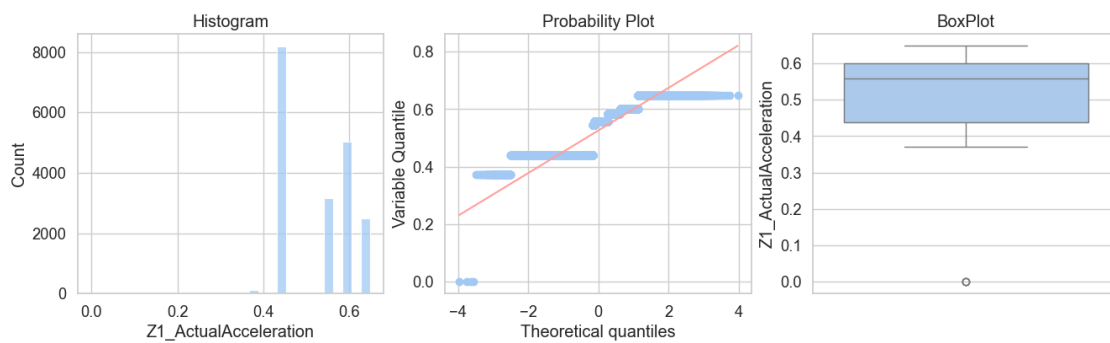
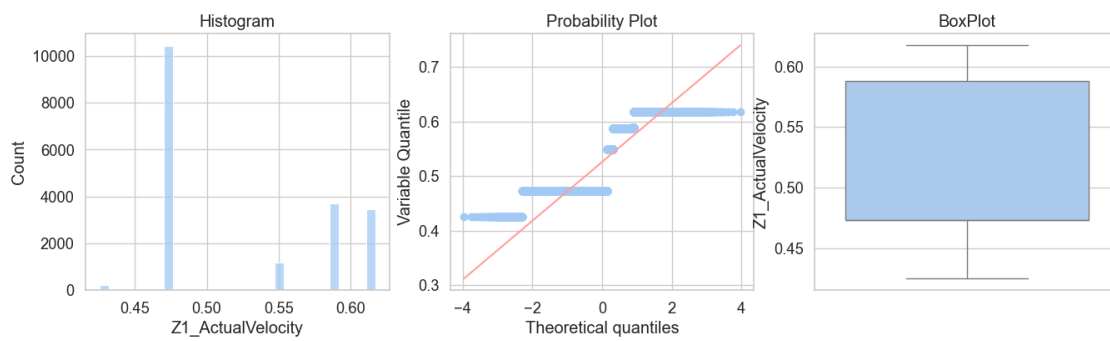
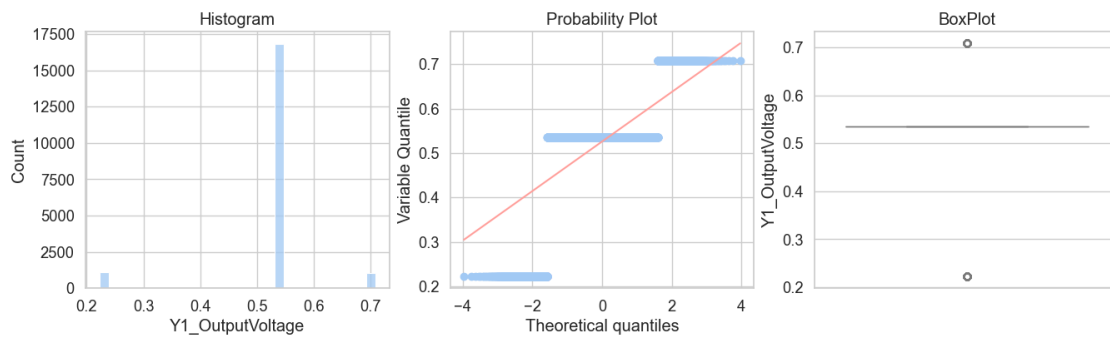
```
[47]: for col in train_t.columns:
        diagnostic_plots(train_t, col)
```

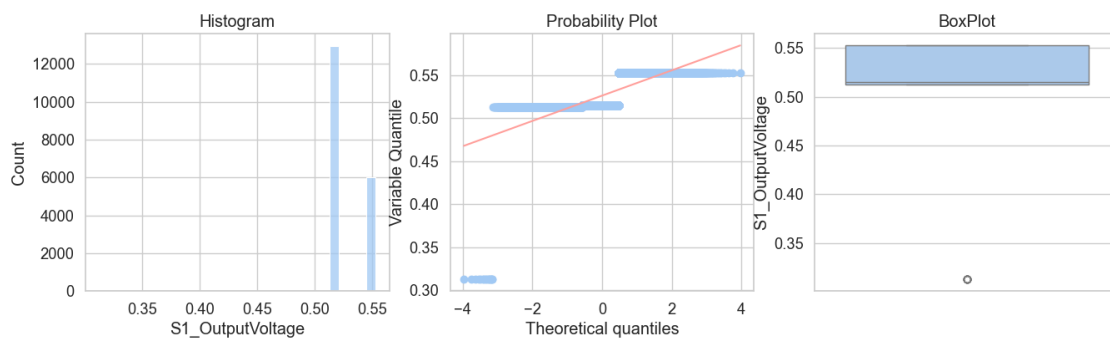
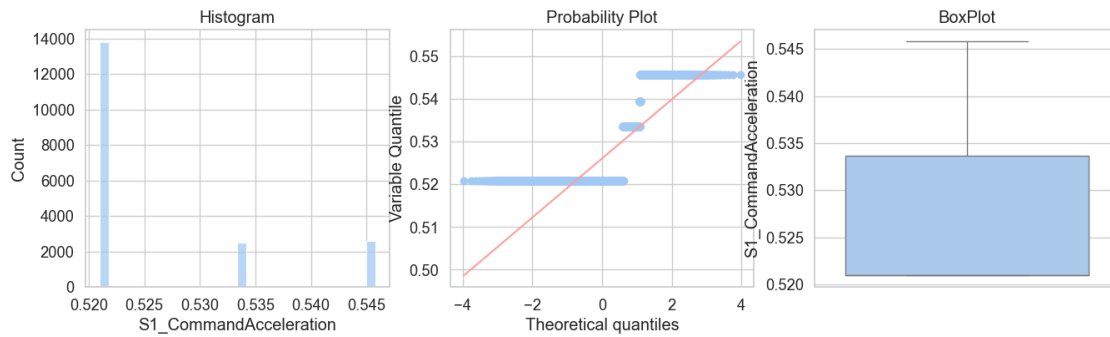
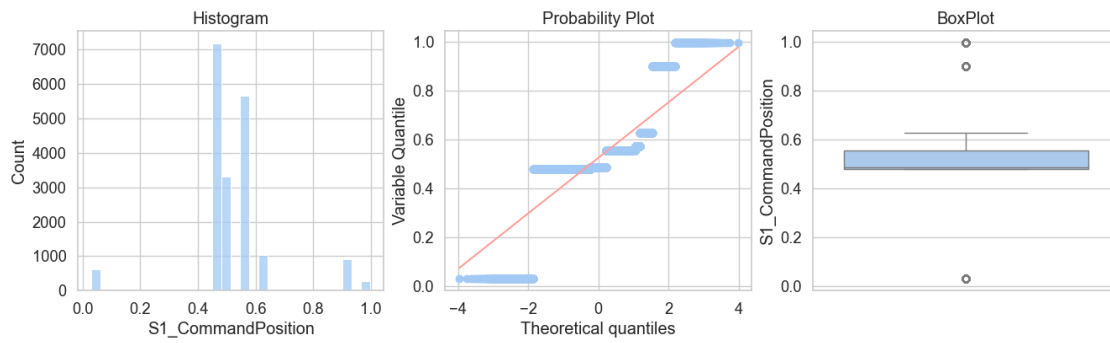
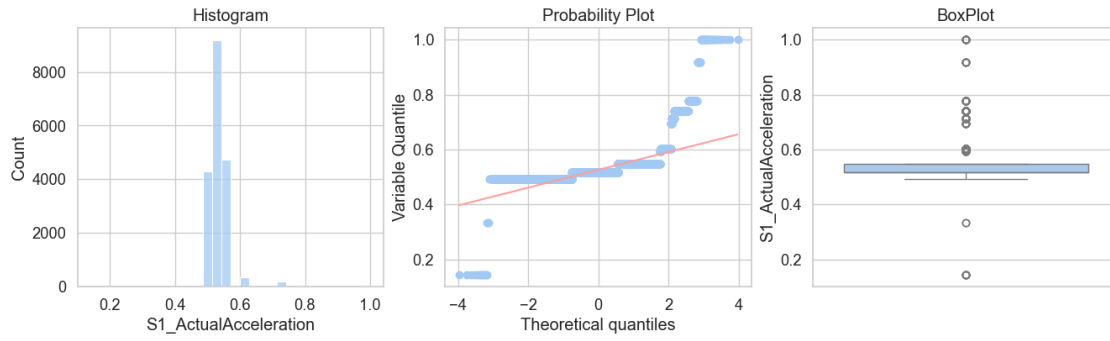


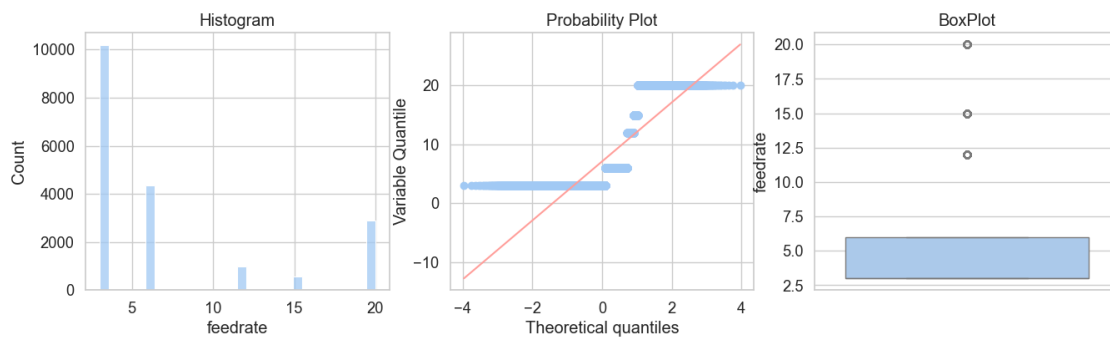
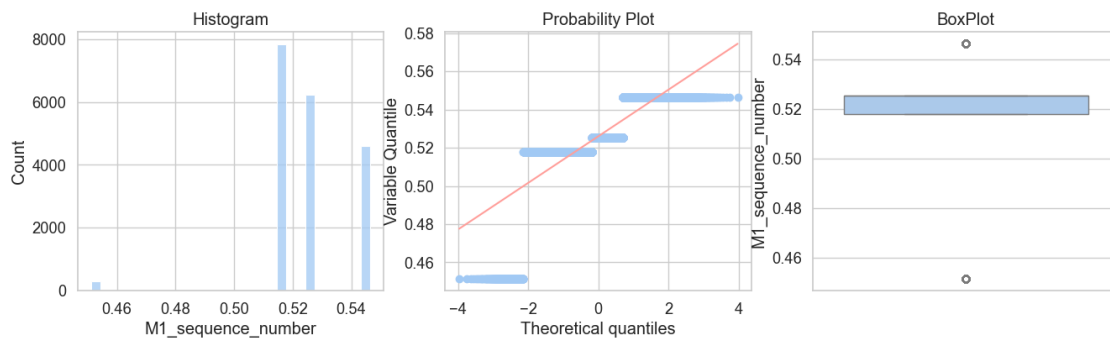
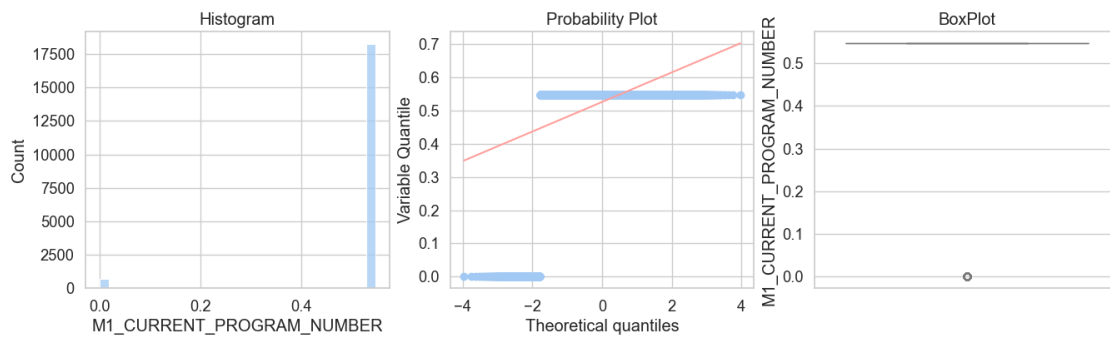


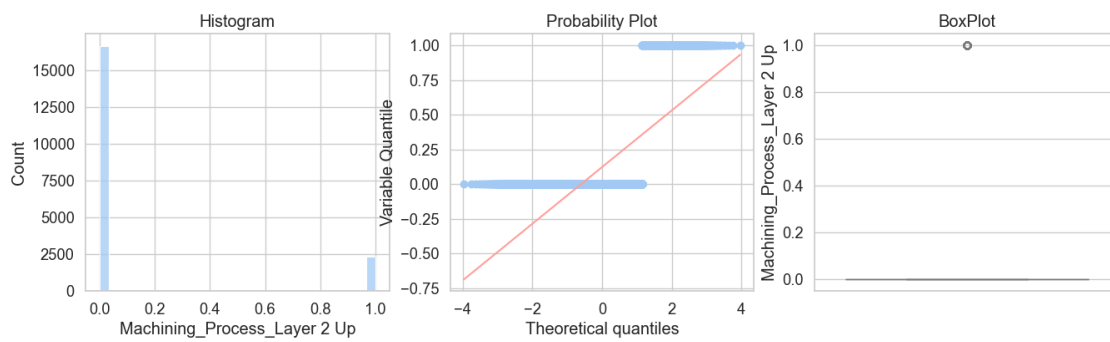
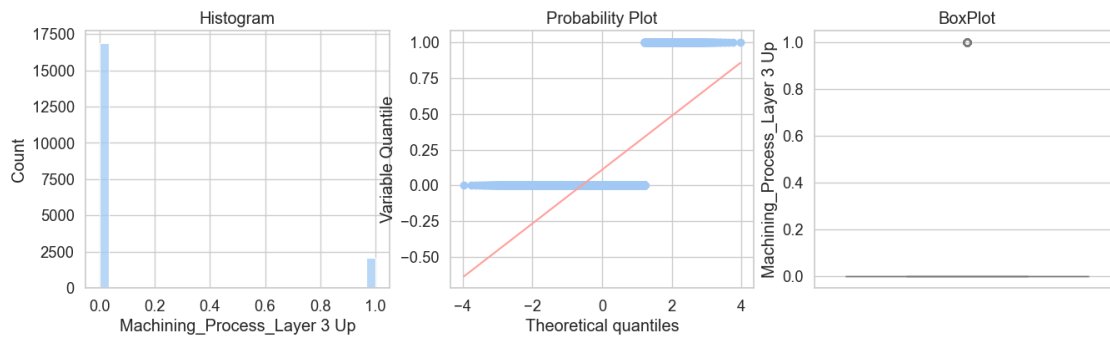
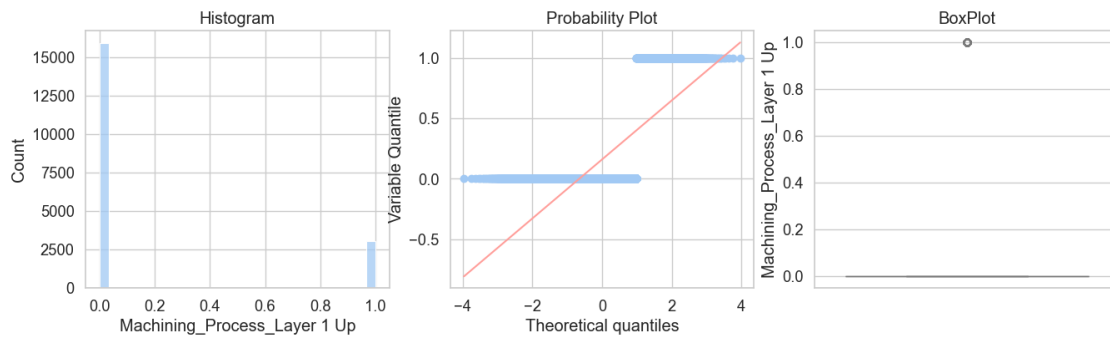
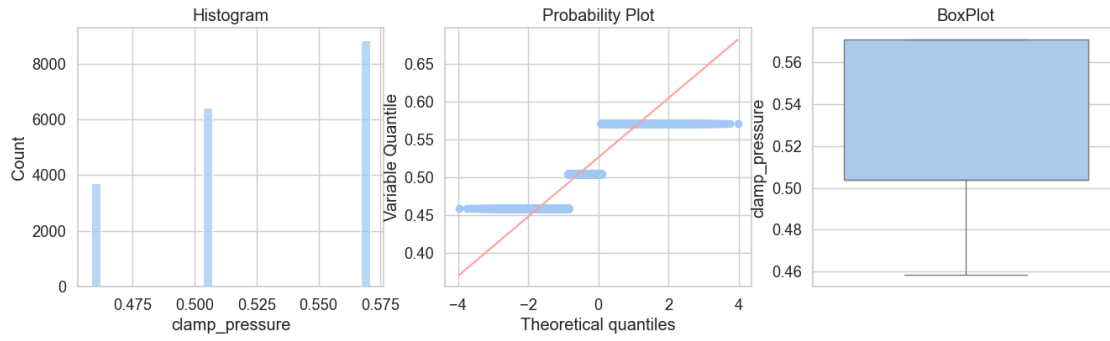


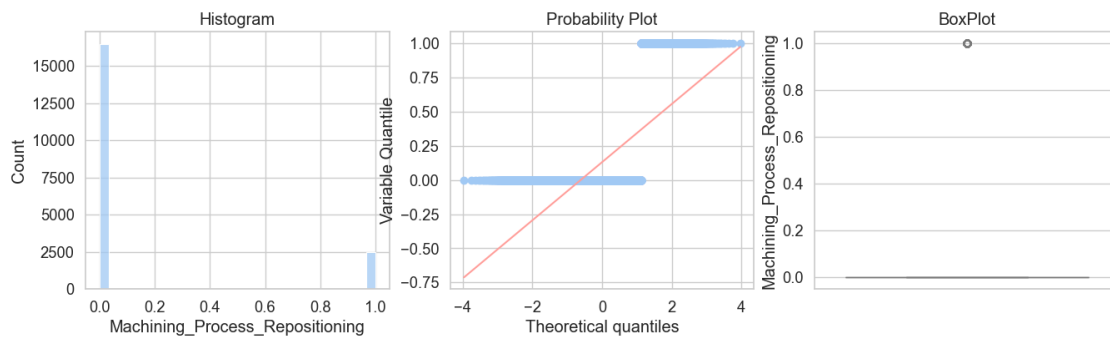
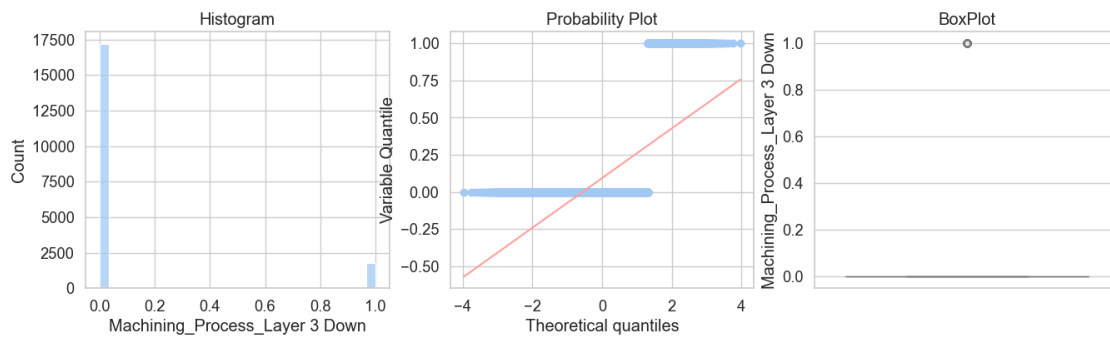
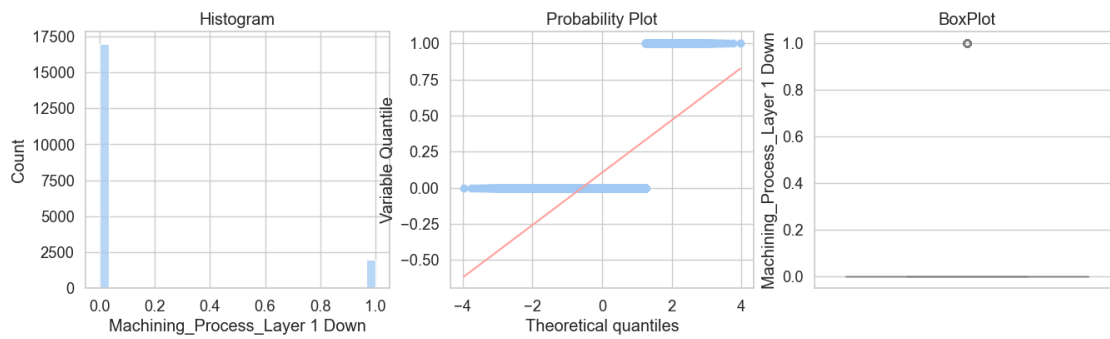


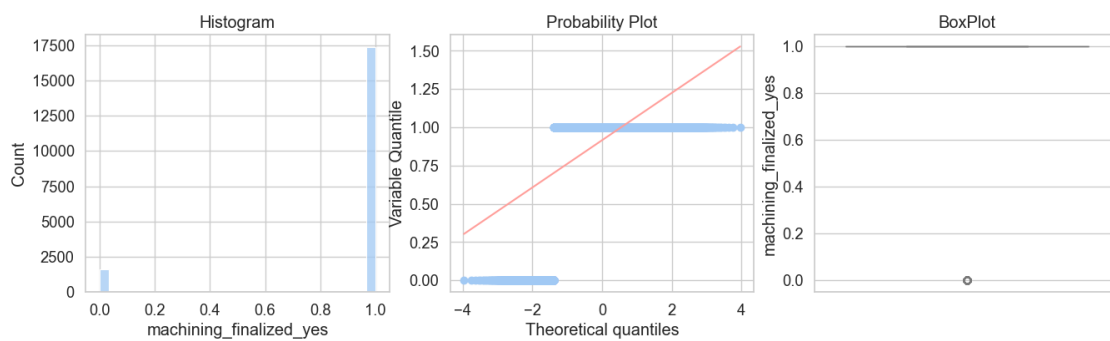
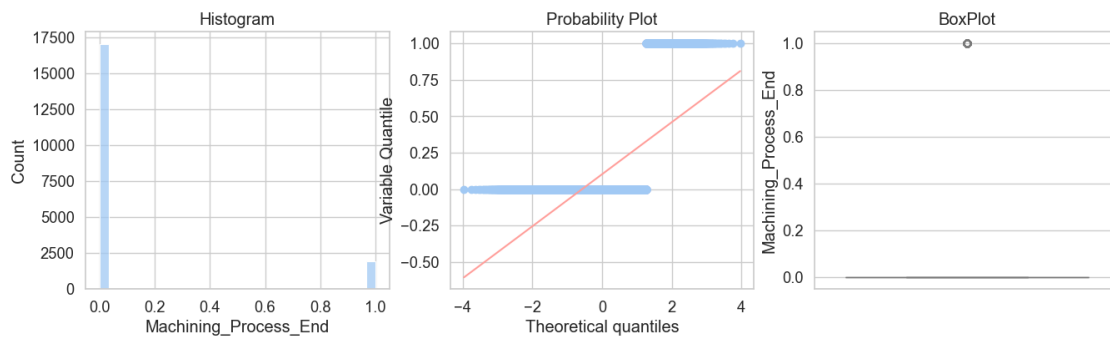
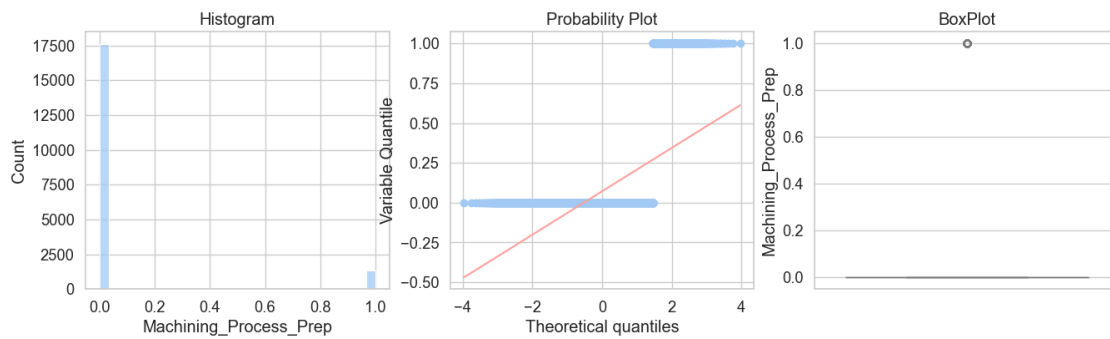
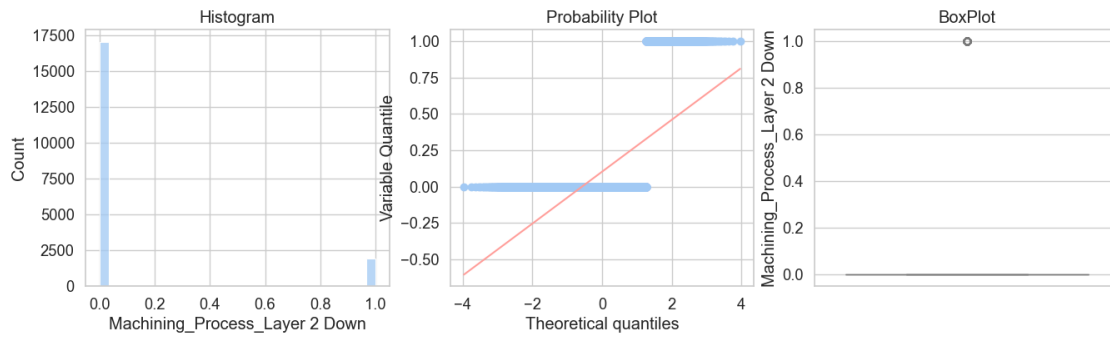


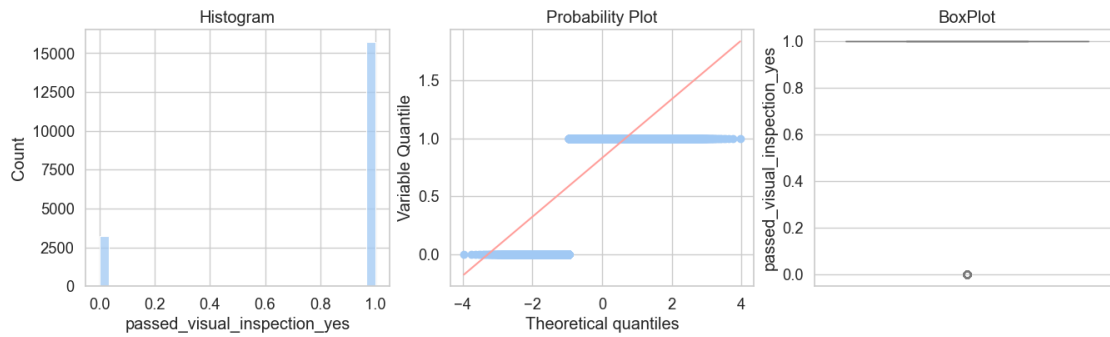












0.1 6. Training algorithms with default parameters

```
[48]: classifiers = {
    'KNeighbor': KNeighborsClassifier(),
    'RandomForest': RandomForestClassifier(),
    'SVC': SVC(),
    'Decision Tree': DecisionTreeClassifier(),
    'AdaBoost': AdaBoostClassifier(),
    'BaggingClf': BaggingClassifier(),
    'ExtraTreeClf': ExtraTreesClassifier(),
    'XGBoost': XGBClassifier(),
    'GaussianNB': GaussianNB(),
    'SGDC': SGDCClassifier(),
}
```

```
[49]: results = {} # Dictionary for save results

for name, clf in classifiers.items():
    clf.fit(train_X, y_train)
    score = clf.score(test_X, y_test)
    results[name] = score

sorted_results = dict(sorted(results.items(), key=lambda item: item[1],
    ↪reverse=True))

print("\nResults:")
for name, score in sorted_results.items():
    print(f"{name}: {score:.2f}")
```

Function for evaluate model performance with classification metrics

0.2 7. Build pipeline and function for performance metrics

```
[50]: # Function for evaluate performance metrics
def evaluate_performance(model, X_train, y_train, X_test, y_test):

    # train model
    model.fit(X_train, y_train)

    # predict label
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    # evaluate metrics performance
    roc_auc_train = roc_auc_score(y_train, y_pred_train)
    roc_auc_test = roc_auc_score(y_test, y_pred_test)

    accuracy_train = accuracy_score(y_train, y_pred_train)
    accuracy_test = accuracy_score(y_test, y_pred_test)

    recall_train = recall_score(y_train, y_pred_train)
    recall_test = recall_score(y_test, y_pred_test)

    f1_train = f1_score(y_train, y_pred_train)
    f1_test = f1_score(y_test, y_pred_test)

    precision_train = precision_score(y_train, y_pred_train)
    precision_test = precision_score(y_test, y_pred_test)

    # Output metrics
    print("ROC AUC Train:", roc_auc_train)
    print("ROC AUC Test:", roc_auc_test)
    print()
    print("Accuracy Train:", accuracy_train)
    print("Accuracy Test:", accuracy_test)
    print()
    print("Recall Train:", recall_train)
    print("Recall Test:", recall_test)
    print()
    print("F1 Score Train:", f1_train)
    print("F1 Score Test:", f1_test)
    print()
    print("Precision Train:", precision_train)
    print("Precision Test:", precision_test)
    print()

    # evaluate mean metrics with cross_val_score
```

```

    roc_auc_cv_scores = cross_val_score(model, X_train, y_train, cv=10,
↪scoring='roc_auc')
    accuracy_cv_scores = cross_val_score(model, X_train, y_train, cv=10,
↪scoring='accuracy')

    roc_auc_cv_mean = roc_auc_cv_scores.mean()
    accuracy_cv_mean = accuracy_cv_scores.mean()

    # evaluate Standard Deviation
    roc_auc_cv_std = roc_auc_cv_scores.std()
    accuracy_cv_std = accuracy_cv_scores.std()

    # Output mean metrics and standard deviation
    print("ROC AUC (cross-validation): Mean =", roc_auc_cv_mean)
    print("Accuracy (cross-validation): Mean =", accuracy_cv_mean)
    print()
    print("ROC AUC Std =", roc_auc_cv_std)
    print("Accuracy Std =", accuracy_cv_std)
    print()

    # Confusion Matrix
    print("Confusion Matrix for Train:")
    print(confusion_matrix(y_train, y_pred_train))
    print()
    print("Confusion Matrix for Test:")
    print(confusion_matrix(y_test, y_pred_test))
    print()

```

Function to build main preprocessor pipeline

```

[51]: def build_pipeline(classifier):
    pipeline = Pipeline(steps = [
        ('imputer', RandomSampleImputer()),
        ('encoder', OneHotEncoder()),
        ('constant', DropConstantFeatures(tol = 0.97)),
        ('duplicated', DropDuplicateFeatures()),
        ('correlated', SmartCorrelatedSelection(selection_method = 'variance')),
        ('scaler', StandardScaler()),
        ('winsorizer', Winsorizer(capping_method = 'gaussian', tail = 'both',
↪fold = 1.5)),
        ('clf', classifier)
    ])
    return pipeline

```

Function for fine-tuning model

```

[52]: def tune_model(X_train, y_train, classifier, param_grid, cv = 10):

```



```

    pipeline = build_pipeline(classifier)
    grid_search = GridSearchCV(pipeline, param_grid, cv = cv, n_jobs = -1,
↪scoring = 'accuracy')
    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_
    best_model.fit(X_train, y_train)

    return best_model

```

Function for get best model

```

[53]: def get_best_model(X_train, y_train, classifier, param_grid, cv=10):
        return tune_model(X_train, y_train, classifier, param_grid, cv)

```

0.3 7.1 Training Models

0.3.1 Random Forest Classifier

```

[54]: classifier_RF = RandomForestClassifier()

param_RF = {
    'constant__tol': [0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'clf__n_estimators': [50, 100, 150],
    'clf__criterion': ['gini', 'entropy'],
    'clf__min_samples_split': [2, 3],
    'clf__min_samples_leaf': [2, 3]
}
best_model_RF = get_best_model(X_train, y_train, classifier_RF, param_RF)

```

```

[55]: evaluate_performance(best_model_RF, X_train, y_train, X_test, y_test)

```

```

[56]: # Save model
with open('best_model_RF', 'wb') as f:
    pickle.dump(best_model_RF, f)

```

0.3.2 Decision Tree Classifier

```

[63]: classifier_DT = DecisionTreeClassifier()

param_DT = {
    'constant__tol': [0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'clf__criterion': ['gini', 'entropy', 'log_loss'],
    'clf__splitter': ['best', 'random'],
    'clf__min_samples_split': [2, 3]
}
best_model_DT = get_best_model(X_train, y_train, classifier_DT, param_DT)

```

```
[64]: evaluate_performance(best_model_DT, X_train, y_train, X_test, y_test)
```

```
[65]: # Save model
with open('best_model_DT', 'wb') as f:
    pickle.dump(best_model_DT, f)
```

0.3.3 SGDClassifier

```
[104]: classifier_SGD = SGDClassifier(loss = 'log')

param_SGD = {
    'constant__tol': [0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'clf__loss': ['hinge', 'log_loss'],
    'clf__penalty': ['l1', 'l2'],
}
best_model_SGDC = get_best_model(X_train, y_train, classifier_SGD, param_SGD)
```

```
[105]: evaluate_performance(best_model_SGDC, X_train, y_train, X_test, y_test)
```

```
[106]: # Save model
with open('best_model_SGDC', 'wb') as f:
    pickle.dump(best_model_SGDC, f)
```

1 Gradient Boosting Classifier

```
[69]: classifier_GBC = GradientBoostingClassifier()

param_GBC = {
    'constant__tol': [0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'winsorizer__fold': [0.5, 0.8, 1],
    'clf__n_estimators': [100, 200, 300],
    'clf__criterion': ['friedman_mse', 'squared_error']
}
best_model_GBC = get_best_model(X_train, y_train, classifier_GBC, param_GBC)
```

```
[70]: evaluate_performance(best_model_GBC, X_train, y_train, X_test, y_test)
```

```
[71]: # Save model
with open('best_model_GBC', 'wb') as f:
    pickle.dump(best_model_GBC, f)
```

2 Ada Boost Classifier

```
[72]: classifier_ADA = AdaBoostClassifier()

param_ADA = {
    'constant__tol': [0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'winsorizer__fold': [0.5, 0.8, 1],
    'clf__n_estimators': [100, 150, 200, 300],
    'clf__learning_rate': [0.1, 0.2, 0.3],
    'clf__algorithm': ['SAMME']
}
best_model_ADA = get_best_model(X_train, y_train, classifier_ADA, param_ADA)

[73]: evaluate_performance(best_model_ADA, X_train, y_train, X_test, y_test)

[74]: # Save model
with open('best_model_ADA', 'wb') as f:
    pickle.dump(best_model_ADA, f)
```

3 Bagging Classifier

```
[75]: classifier_BAGGING = BaggingClassifier(n_jobs = -1)

param_BAG = {
    'constant__tol': [0.87, 0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'winsorizer__fold': [0.5, 0.8, 1],
    'clf__n_estimators': [80, 90, 100, 120],
    'clf__max_samples': [1, 2, 3]
}
best_model_BAGGING = get_best_model(X_train, y_train, classifier_BAGGING,
    ↪param_BAG)

[76]: evaluate_performance(best_model_BAGGING, X_train, y_train, X_test, y_test)

[77]: # Save model
with open('best_model_BAGGING', 'wb') as f:
    pickle.dump(best_model_BAGGING, f)
```

4 Extra Trees Classifier

```
[78]: classifier_EXTRA = ExtraTreesClassifier(n_jobs = -1)

param_EXTRA = {
    'constant__tol': [0.93, 0.96],
```

```

        'winsorizer__tail': ['left', 'right'],
        'winsorizer__fold': [0.5, 0.8, 1],
        'clf__n_estimators': [90, 100, 120],
        'clf__criterion': ['gini', 'entropy', 'log_loss'],
        'clf__min_samples_split': [2, 3],
        'clf__min_samples_leaf': [2, 3]
    }
    best_model_EXTRA = get_best_model(X_train, y_train, classifier_EXTRA,
    ↪param_EXTRA)

```

```
[79]: evaluate_performance(best_model_EXTRA, X_train, y_train, X_test, y_test)
```

```
[80]: # Save model
with open('best_model_EXTRA', 'wb') as f:
    pickle.dump(best_model_EXTRA, f)

```

5 KNeighborsClassifier

```
[82]: classifier_KNeighbors = KNeighborsClassifier(n_jobs = -1)

param_KN = {
    'constant__tol': [0.87, 0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'winsorizer__fold': [0.5, 0.8, 1],
    'clf__n_neighbors': [120, 150, 180, 200],
    'clf__algorithm': ['auto', 'ball_tree', 'brute']
}
best_model_KNeighbors = get_best_model(X_train, y_train, classifier_KNeighbors,
    ↪param_KN)

```

```
[83]: evaluate_performance(best_model_KNeighbors, X_train, y_train, X_test, y_test)
```

```
[84]: # Save model
with open('best_model_KNeighbors', 'wb') as f:
    pickle.dump(best_model_KNeighbors, f)

```

6 XGBoost Classifier

```
[85]: classifier_XGB = XGBClassifier(n_jobs = -1)

param_XGB = {
    'constant__tol': [0.87, 0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'winsorizer__fold': [0.5, 0.8, 1],
    'clf__n_estimators': [170, 200, 220],
    'clf__max_depth': [3, 4, 5]
}

```

```
}
best_model_XGB = get_best_model(X_train, y_train, classifier_XGB, param_XGB)
```

```
[86]: evaluate_performance(best_model_XGB, X_train, y_train, X_test, y_test)
```

```
[87]: # Save model
with open('best_model_XGB', 'wb') as f:
    pickle.dump(best_model_XGB, f)
```

7 GaussianNB

```
[88]: classifier_Gaussian = GaussianNB()

param_GAUS = {
    'constant__tol': [0.87, 0.93, 0.96],
    'scaler': [StandardScaler()],
    'winsorizer__tail': ['left', 'right'],
    'winsorizer__fold': [0.5, 0.8, 1]
}
best_model_GAUS = get_best_model(X_train, y_train, classifier_Gaussian,
    ↪param_GAUS)
```

```
[89]: evaluate_performance(best_model_GAUS, X_train, y_train, X_test, y_test)
```

```
[90]: # Save model
with open('best_model_GAUS', 'wb') as f:
    pickle.dump(best_model_GAUS, f)
```

8 SVC

```
[108]: classifier_SVC = SVC(probability = True)

param_SVC = {
    'constant__tol': [0.93, 0.96],
    'winsorizer__tail': ['left', 'right'],
    'scaler': [StandardScaler()],
    'clf__C': [0.5, 1],
    'clf__kernel': ['linear', 'rbf']
}
best_model_SVC = get_best_model(X_train, y_train, classifier_SVC, param_SVC)
```

```
[109]: evaluate_performance(best_model_SVC, X_train, y_train, X_test, y_test)
```

```
[110]: # Save model
with open('best_model_SVC', 'wb') as f:
    pickle.dump(best_model_SVC, f)
```

8.1 8. Create lists for visualization results

```
[196]: models = [
    best_model_ADA,
    best_model_BAGGING,
    best_model_DT,
    best_model_EXTRA,
    best_model_GAUS,
    best_model_GBC,
    best_model_KNeighbors,
    best_model_RF,
    best_model_SGDC,
    best_model_SVC,
    best_model_XGB
]

model_names = [
    'AdaBoostClf',
    'BaggingClf',
    'DecisionTreeClf',
    'ExtraTreeClf',
    'GaussianNB',
    'GradientBoostingClf',
    'KNeighbors',
    'RandomForestClf',
    'SGDC',
    'SVC',
    'XGBoostClf'
]

[200]: def calculate_performance_metrics(models, X_train, y_train, X_test, y_test,
    ↪model_names):
    metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score', 'ROC AUC']
    metric_scores = {metric: [] for metric in metrics}

    for model in models:
        train_predictions = model.predict(X_train)
        test_predictions = model.predict(X_test)

        accuracy = accuracy_score(y_test, test_predictions)
        precision = precision_score(y_test, test_predictions)
        recall = recall_score(y_test, test_predictions)
        f1 = f1_score(y_test, test_predictions)
        roc_auc = roc_auc_score(y_test, test_predictions)

        metric_scores['Accuracy'].append(accuracy)
        metric_scores['Precision'].append(precision)
```

```

        metric_scores['Recall'].append(recall)
        metric_scores['F1 Score'].append(f1)
        metric_scores['ROC AUC'].append(roc_auc)

    return pd.DataFrame(metric_scores, index=model_names).
    ↪sort_values(by='Accuracy', ascending=False)

calculate_performance_metrics(models, X_train, y_train, X_test, y_test,
    ↪model_names)

```

8.2 8.1 Visualization of results by metrics Accuracy, ROC AUC, Precision, Recall, F1 and confusion matrix

```

[198]: def plot_cm(model, X_train, X_test, y_train, y_test, model_name):
    # Creating subgraphs for confusion matrix for train and test set
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

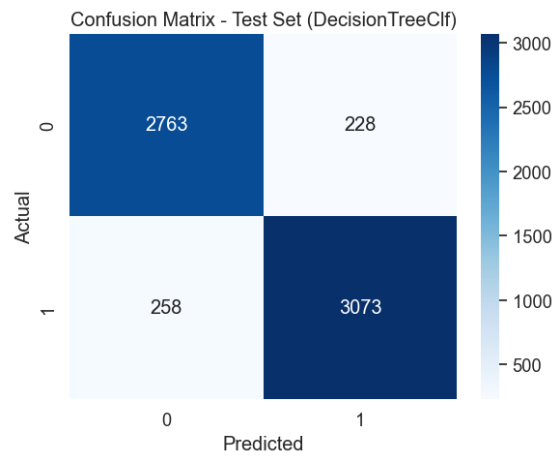
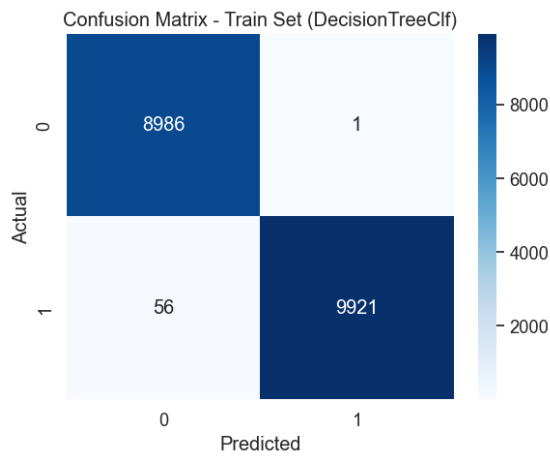
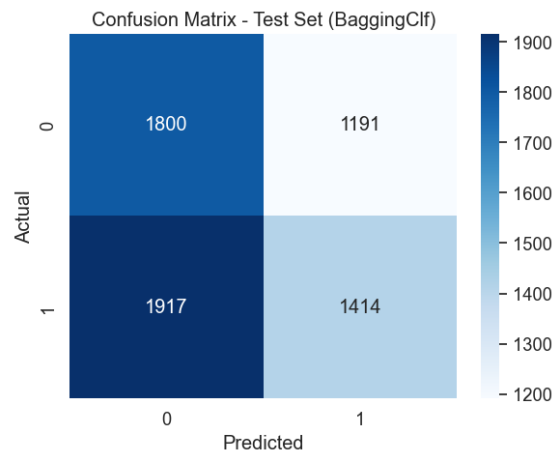
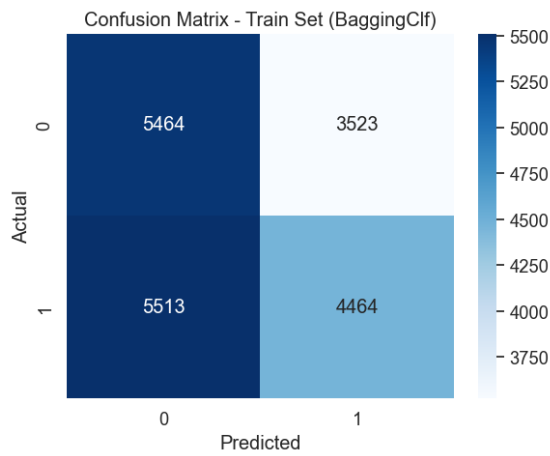
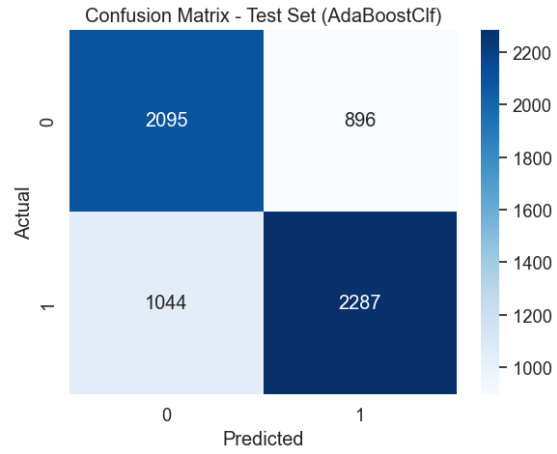
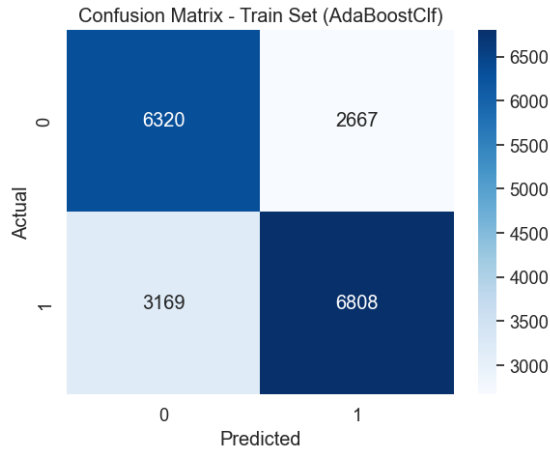
    # Confusion Matrix for train set
    train_cm = confusion_matrix(y_train, model.predict(X_train))
    sns.heatmap(train_cm, annot=True, cmap='Blues', fmt='d', ax=axes[0])
    axes[0].set_title('Confusion Matrix - Train Set ({}).format(model_name))
    axes[0].set_xlabel('Predicted')
    axes[0].set_ylabel('Actual')

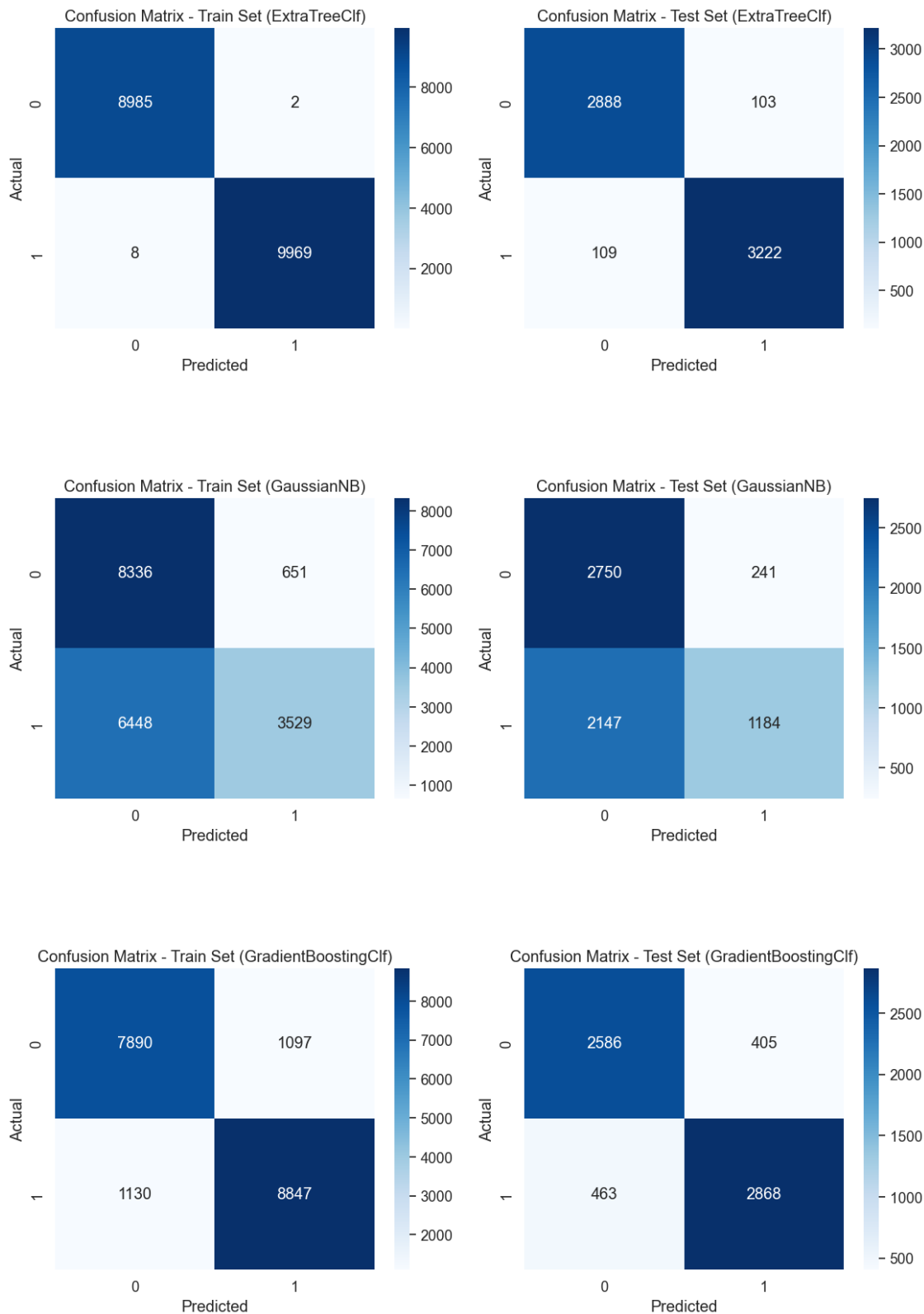
    # Confusion Matrix for test set
    test_cm = confusion_matrix(y_test, model.predict(X_test))
    sns.heatmap(test_cm, annot=True, cmap='Blues', fmt='d', ax=axes[1])
    axes[1].set_title('Confusion Matrix - Test Set ({}).format(model_name))
    axes[1].set_xlabel('Predicted')
    axes[1].set_ylabel('Actual')

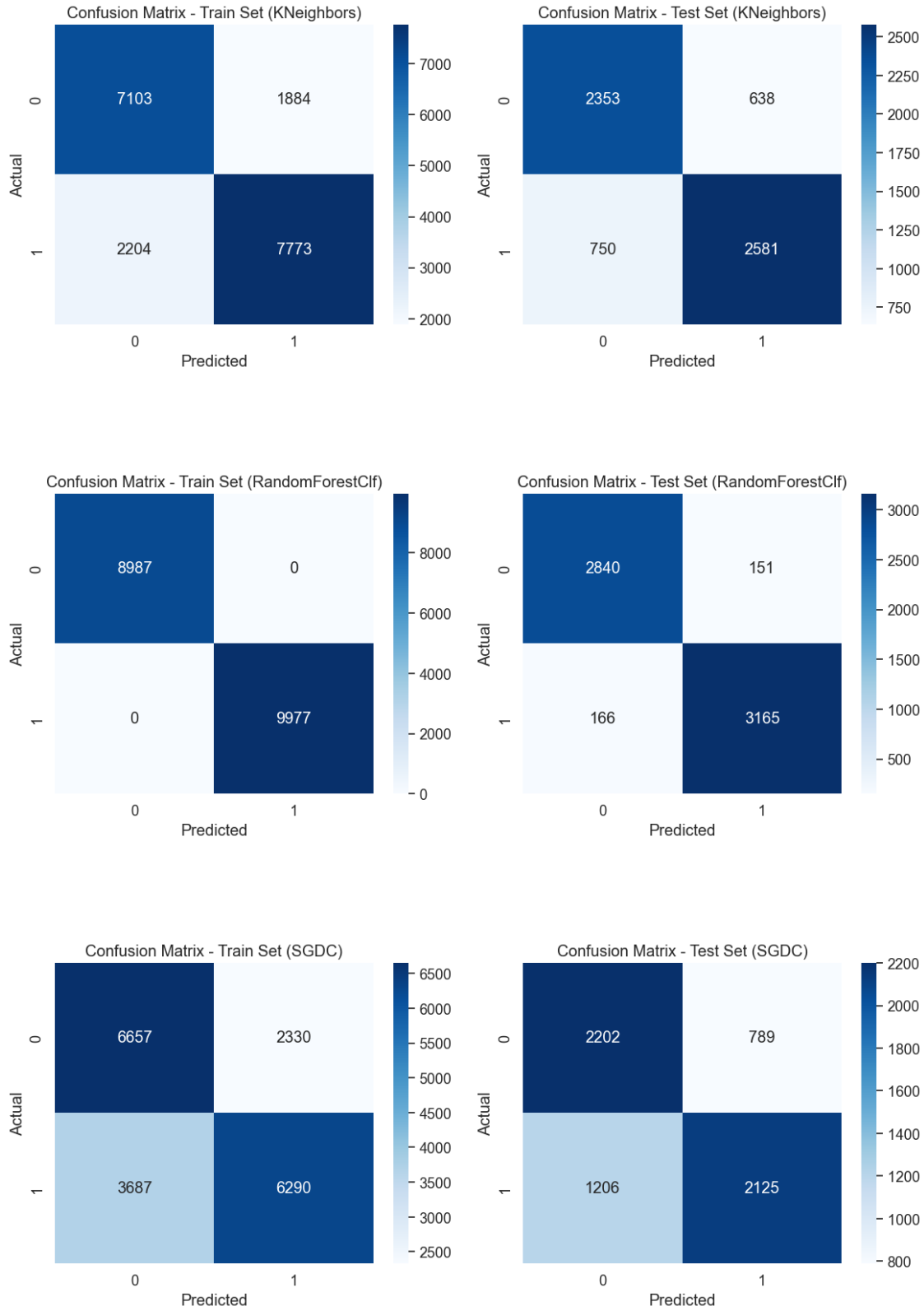
    plt.tight_layout()
    plt.show()

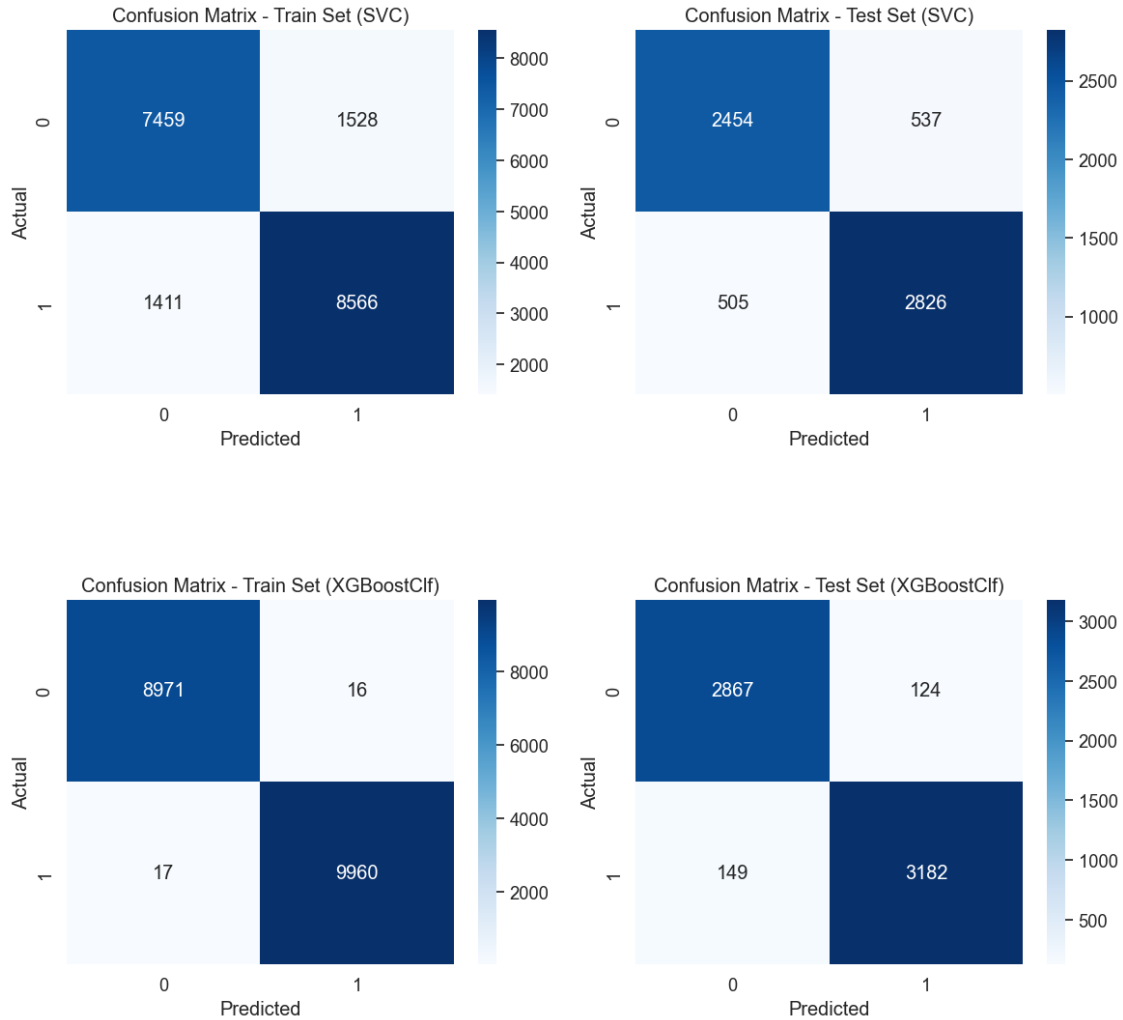
    # Build confusion matrix for each models
    for model, model_name in zip(models, model_names):
        plot_cm(model, X_train, X_test, y_train, y_test, model_name)

```









```
[201]: def plot_all_metrics(models, X_train, y_train, X_test, y_test, model_names):
    metrics = {
        'ROC AUC': ([], []),
        'Accuracy': ([], []),
        'Precision': ([], []),
        'Recall': ([], []),
        'F1 Score': ([], [])
    }

    for model in models:
        y_train_pred = model.predict(X_train)
        y_test_pred = model.predict(X_test)

        y_train_proba = model.predict_proba(X_train)[: , 1]
        y_test_proba = model.predict_proba(X_test)[: , 1]
```

```

metrics['ROC AUC'][0].append(roc_auc_score(y_train, y_train_proba))
metrics['ROC AUC'][1].append(roc_auc_score(y_test, y_test_proba))

metrics['Accuracy'][0].append(accuracy_score(y_train, y_train_pred))
metrics['Accuracy'][1].append(accuracy_score(y_test, y_test_pred))

metrics['Precision'][0].append(precision_score(y_train, y_train_pred))
metrics['Precision'][1].append(precision_score(y_test, y_test_pred))

metrics['Recall'][0].append(recall_score(y_train, y_train_pred))
metrics['Recall'][1].append(recall_score(y_test, y_test_pred))

metrics['F1 Score'][0].append(f1_score(y_train, y_train_pred))
metrics['F1 Score'][1].append(f1_score(y_test, y_test_pred))

# Sorting models by test ROC AUC
sorted_indices = np.argsort(metrics['ROC AUC'][1])[:, :-1]

model_names_sorted = [model_names[i] for i in sorted_indices]

for metric in metrics:
    metrics[metric] = ([metrics[metric][0][i] for i in sorted_indices],
                       [metrics[metric][1][i] for i in sorted_indices])

x = np.arange(len(model_names_sorted))
width = 0.35

fig, axes = plt.subplots(3, 2, figsize=(16, 20))
axes = axes.flatten()

for idx, (metric, (train_scores, test_scores)) in enumerate(metrics.
↪items()):
    ax = axes[idx]
    rects1 = ax.bar(x - width/2, train_scores, width, label=f'Train_
↪{metric}', color='green')
    rects2 = ax.bar(x + width/2, test_scores, width, label=f'Test_
↪{metric}', color='blue')

    ax.set_ylabel(f'{metric} Scores')
    ax.set_title(f'{metric} Scores')
    ax.set_xticks(x)
    ax.set_xticklabels(model_names_sorted, rotation=45, ha='right')
    ax.legend()

    autolabel(rects1, ax)
    autolabel(rects2, ax)

```

```

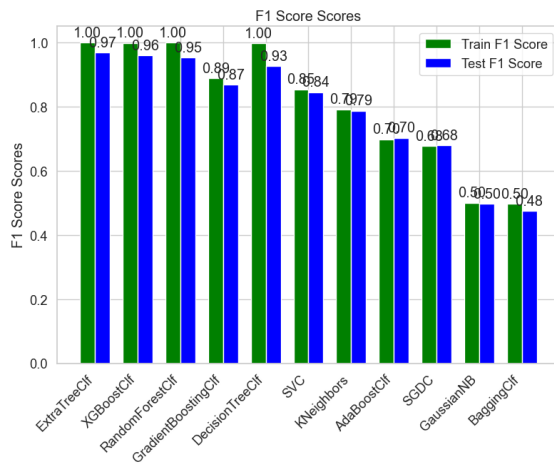
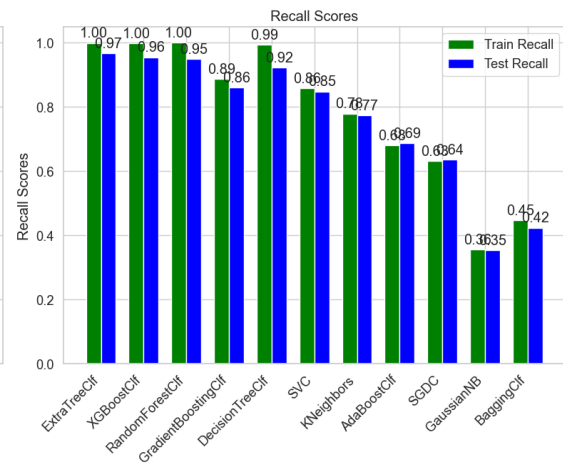
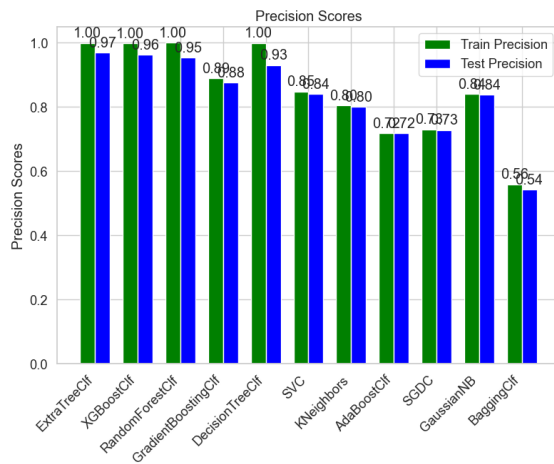
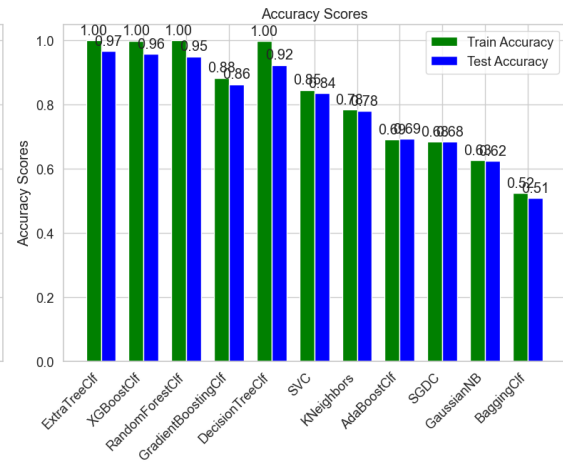
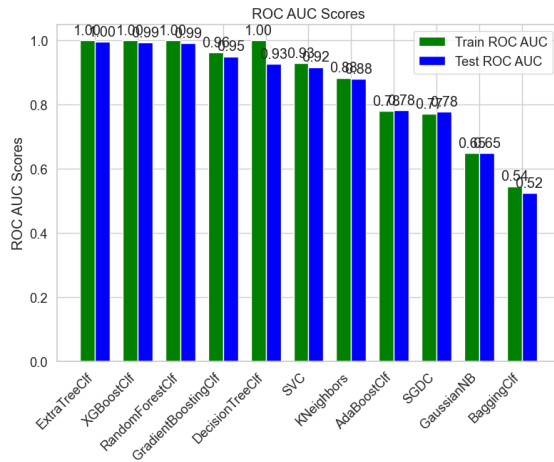
# Remove any empty subplots (if there are)
if len(metrics) % 2 != 0:
    fig.delaxes(axes[-1])

fig.tight_layout()
plt.show()

def autolabel(rects, ax):
    for rect in rects:
        height = rect.get_height()
        ax.annotate(f'{{height:.2f}}',
                    xy=(rect.get_x() + rect.get_width() / 2, height),
                    xytext=(0, 3),
                    textcoords="offset points",
                    ha='center', va='bottom')

plot_all_metrics(models, X_train, y_train, X_test, y_test, model_names)

```



9 ANN Model

```
[132]: # Function for demonstrate model performance
def plot_performance(history):
    plt.figure(figsize=(12, 6))

    # Loss
    plt.subplot(1, 2, 1)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Loss')
    plt.legend()

    # Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Accuracy')
    plt.legend()

    plt.show()

[140]: # Preprocessing for ANN model
pipeline = Pipeline(steps = [
    ('imputer', RandomSampleImputer()),
    ('encoder', OneHotEncoder()),
    ('constant', DropConstantFeatures(tol = 0.97)),
    ('duplicated', DropDuplicateFeatures()),
    ('correlated', SmartCorrelatedSelection(selection_method = 'variance')),
    ('scaler', StandardScaler()),
    ('winsorizer', Winsorizer(capping_method = 'gaussian', tail = 'both', fold_u
↵= 1.5))
])

X_train_preprocessed = pipeline.fit_transform(X_train)
X_test_preprocessed = pipeline.transform(X_test)

[141]: # Early stopping
early_stopping = EarlyStopping(
    monitor = 'val_accuracy',
    min_delta = 0.001,
    patience = 50,
    restore_best_weights = True,
```

```

        verbose = 1,
        mode = 'auto'
    )
    callbacks = early_stopping

```

```

[142]: # Build model
input_shape = (X_train_preprocessed.shape[1],)

def create_model(optimizer = 'adam'):
    classifier = Sequential()

    classifier.add(Input(shape = input_shape))

    classifier.add(Dense(units = 300, kernel_initializer = 'he_uniform',
↪activation = 'relu', kernel_regularizer = l2(0.001)))
    classifier.add(BatchNormalization())
    classifier.add(Dropout(0.2))

    classifier.add(Dense(units = 300, kernel_initializer = 'he_uniform',
↪activation = 'relu', kernel_regularizer = l2(0.001)))
    classifier.add(BatchNormalization())
    classifier.add(Dropout(0.2))

    classifier.add(Dense(units = 250, kernel_initializer = 'he_uniform',
↪activation = 'relu', kernel_regularizer = l2(0.001)))
    classifier.add(BatchNormalization())
    classifier.add(Dropout(0.3))

    classifier.add(Dense(units = 200, kernel_initializer = 'he_uniform',
↪activation = 'relu', kernel_regularizer = l2(0.001)))
    classifier.add(BatchNormalization())
    classifier.add(Dropout(0.2))

    classifier.add(Dense(units = 1, activation = 'sigmoid', kernel_initializer
↪= 'glorot_uniform'))
    classifier.compile(optimizer = optimizer, loss = 'binary_crossentropy',
↪metrics = ['accuracy'])
    return classifier

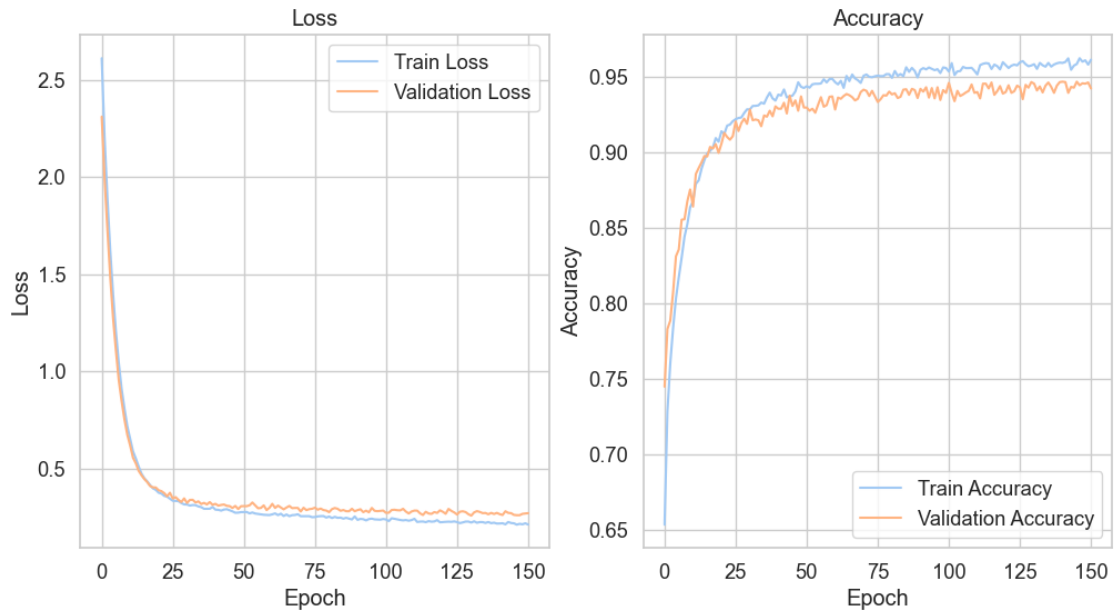
hist_model = create_model()
history1 = hist_model.fit(
    X_train_preprocessed,
    y_train,
    epochs = 300,
    batch_size = 128,
    validation_split = 0.2,
    callbacks = callbacks

```



```
)
hist_model.summary()
```

```
[144]: plot_performance(history1)
```



10 GridSearchCV

```
[145]: pipeline = Pipeline(steps=[
    ('preprocessing', pipeline),
    ('model', KerasClassifier(model=create_model,
                              epochs = 100,
                              batch_size = 128,
                              optimizer = 'adam',
                              metrics=['val_accuracy'],
                              validation_split = 0.2,
                              callbacks = callbacks
                              ))
])

# GridSearch params
parameters = {
    'model__batch_size': [64, 128, 256],
    'model__epochs': [100, 200],
    'model__optimizer': ['adam', 'SGD']
}
```

```

grid = GridSearchCV(
    estimator = pipeline,
    param_grid = parameters,
    n_jobs = -1,
    cv = 3,
    error_score = 'raise'
)

grid_results = grid.fit(X_train, y_train, model__validation_split = 0.2,
    ↪model__callbacks = callbacks)

best_model = grid_results.best_estimator_
print("Best Parameters:", grid_results.best_params_)

```

```

[180]: best_params = grid_results.best_params_
print('Best Parameters:', best_params)

```

```

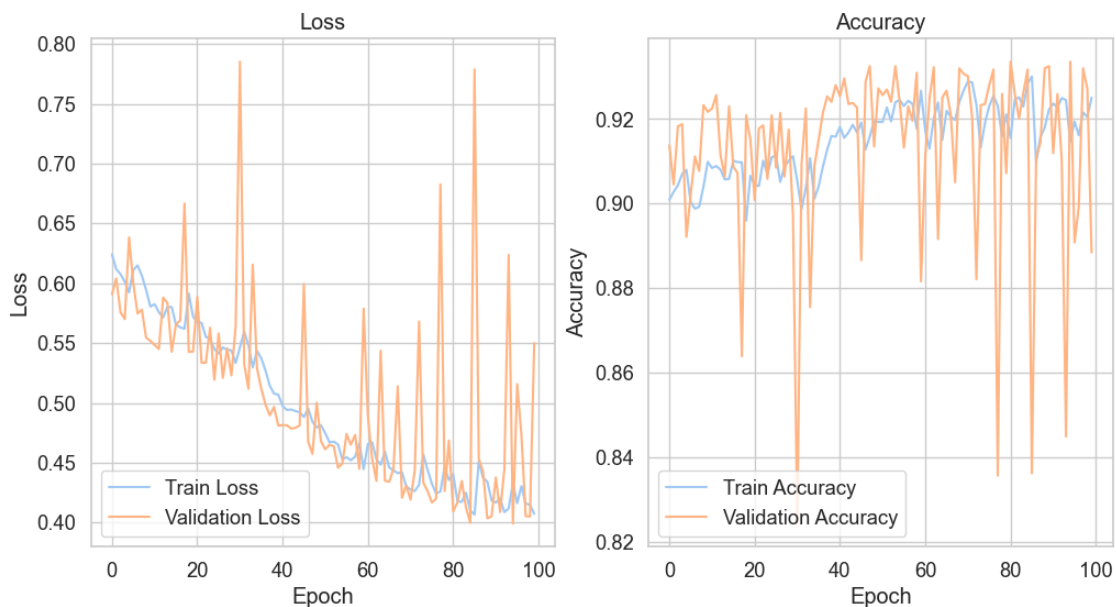
[185]: best_model_ANN = best_model_ANN.fit(X_train_preprocessed, y_train,
    validation_split = 0.2,
    batch_size = best_params['model__batch_size'],
    epochs = best_params['model__epochs'],
    callbacks = [EarlyStopping(monitor = 'val_loss',
    ↪patience = 10, verbose = 1)])

```

```

[186]: plot_performance(best_model_ANN)

```



```

[215]: y_train_pred_proba = best_model.predict(X_train_preprocessed)
y_train_pred = (y_train_pred_proba > 0.5).astype(int)

y_test_pred_proba = best_model.predict(X_test_preprocessed)
y_test_pred = (y_test_pred_proba > 0.5).astype(int)

train_accuracy = accuracy_score(y_train, y_train_pred)
train_roc_auc = roc_auc_score(y_train, y_train_pred_proba)
train_recall = recall_score(y_train, y_train_pred)
train_precision = precision_score(y_train, y_train_pred)
train_f1 = f1_score(y_train, y_train_pred)

test_accuracy = accuracy_score(y_test, y_test_pred)
test_roc_auc = roc_auc_score(y_test, y_test_pred_proba)
test_recall = recall_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred)

train_metrics = [train_accuracy, train_roc_auc, train_recall, train_precision,
                 ↪train_f1]
test_metrics = [test_accuracy, test_roc_auc, test_recall, test_precision,
                 ↪test_f1]
metric_names = ['Accuracy', 'ROC AUC', 'Recall', 'Precision', 'F1 Score']

x = np.arange(len(metric_names))
width = 0.35 #

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, train_metrics, width, label='Train',
               ↪color='skyblue')
bars2 = ax.bar(x + width/2, test_metrics, width, label='Test',
               ↪color='lightgreen')

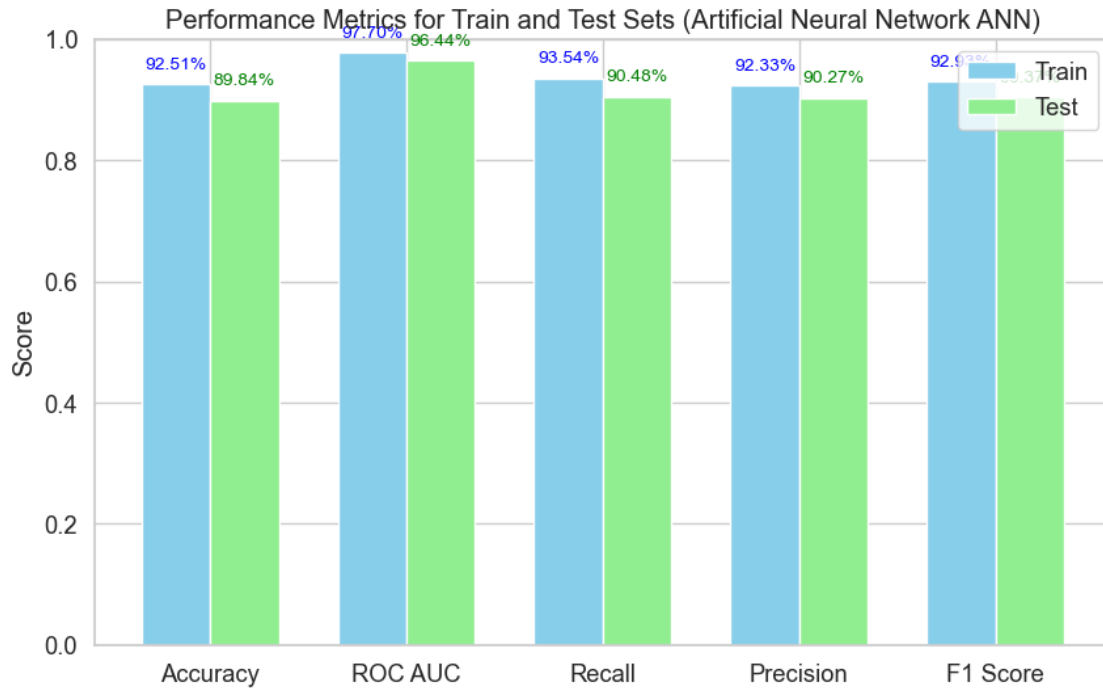
for bar in bars1:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, yval + 0.02, f'{yval:.2%}',
            ↪ha='center', va='bottom', color='blue', fontsize=10)
for bar in bars2:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, yval + 0.02, f'{yval:.2%}',
            ↪ha='center', va='bottom', color='green', fontsize=10)

ax.set_ylabel('Score')
ax.set_title('Performance Metrics for Train and Test Sets (Artificial Neural
             ↪Network ANN)')
ax.set_xticks(x)

```

```
ax.set_xticklabels(metric_names)
ax.set_ylim(0, 1)
ax.legend()

plt.show()
```



Build the graph that will show us the performance of models based on the ROC AUC and ACCURACY metrics

10.1 9. Conclusion

During the project on predicting CNC machine tool part failures, we trained several classification models, including Extra Tree, XGBoost, Random Forest, GradientBoostingClf, and Decision Tree. All of them demonstrated good performance, allowing us to effectively predict potential part failures and take timely measures to prevent them.

We also trained an Artificial Neural Network (ANN) model, which demonstrated the following metric values:

- *Accuracy*: 92.51% (Train), 89.84% (Test)
- *ROC AUC*: 97.70% (Train), 96.44% (Test)
- *Recall*: 93.54% (Train), 90.48% (Test)
- *Precision*: 92.33% (Train), 90.27% (Test)
- *F1 Score*: 92.87% (Train), 90.36% (Test)

To improve the results, we plan to conduct further research. We will consider alternative data

processing methods, including outlier handling, and experiment with algorithm parameters. Additionally, we will explore the possibility of using ensemble methods to increase the stability and accuracy of predictions.

Overall, our project demonstrates the potential of applying machine learning in manufacturing processes, allowing for increased efficiency and reduced operational risks.

:
 , Extra Tree, XGBoost,
 Random Forest, GradientBoostingClf Decision Tree.

- (ANN), :
- *Accuracy*: 92.51% (Train), 89.84% (Test)
 - *ROC AUC*: 97.70% (Train), 96.44% (Test)
 - *Recall*: 93.54% (Train), 90.48% (Test)
 - *Precision*: 92.33% (Train), 90.27% (Test)
 - *F1 Score*: 92.87% (Train), 90.36% (Test)

[]: